

04

The Teaching Guide

Every tool in the system, explained for a beginner, with a free way to learn each one

Understand the project.

Know what deserves your attention.

COLORFUL READING EDITION

The complete original text, with reading maps, priority labels and selective highlights. Start with the map on the next page.

Your reading map

A manual to visit when the work needs it

Start with the links below. Then read the full text at your own pace.

- **Chapter 0 / Learn with an AI tool** - Start with the habits for understanding changes.
- **Chapter 2 / Your daily editor** - Learn the workflow and the review controls.
- **Chapter 15 / Watch and read order** - Choose a lesson for the concept in front of you.
- **Chapter 16 / The schedule** - Use stages, time budgets, and checkpoints.
- **Chapter 17 / Glossary** - Look up a term and return to your page.

Keep for later. The other chapters are for their practical stages. Read security before handling accounts or keys. Each exercise is followed by a shaded completion checklist.

The four labels

Label	How to use it
READ FIRST	Blue. Essential to understanding the project or a decision.
KEY IDEA	Teal. A concept or checkpoint to remember; yellow marks short phrases.
WATCH OUT	Orange. A limitation, assumption, or common mistake to read carefully.
OPTIONAL - FIRST PASS	Purple. Return when you need this detail; the text is still here.

Reading labels are added guidance. The original text, figures, dates, and source references are retained. Page numbers in this edition may differ from the originals.

Find a section

How to read this document	7
Chapter 0. How to learn with an AI coding tool without becoming dependent on it	7
What it is (plain words and an analogy)	7
Why this matters now (for this project)	8
How it is used in this project	8
The ten operational rules, each with a 5-minute exercise	8
Learn it	10
10-minute exercise	11
Done when	12
Chapter 1. Git, GitHub, branches, pull requests and CI checks	12
What it is (plain words and an analogy)	12
Why this tool now (for this project)	13
How it is used in this project	13
Learn it	14
10-minute exercise	15
Done when	16
Chapter 2. Cursor and Cline: the tool you type into, and the free bench beside it	16
What it is (plain words and an analogy)	16
Why this tool now (for this project)	17
How it is used in this project	17
Learn it	20
10-minute exercise	21
Done when	22
Chapter 3. GitHub Actions: the robot that tests your code and runs it at night	22
What it is (plain words and an analogy)	22
Why this tool now (for this project)	22
How it is used in this project	23
Learn it	26
10-minute exercise	26
Done when	27
Chapter 4. TypeScript for a Python brain	27
What it is (plain words and an analogy)	27
Why this tool now (for this project)	28
How it is used in this project	28
The eight things that bite	28
Learn it	31
10-minute exercise	32
Done when	32
Chapter 5. Postgres and Neon: the one place the numbers live	32

What it is (plain words and an analogy)	32
Why this tool now (for this project)	33
How it is used in this project	34
Learn it	37
10-minute exercise	38
Done when	39
Chapter 6. Pinecone, Voyage and RAG in plain words	39
What it is (plain words and an analogy)	39
Why this tool now (for this project)	40
How it is used in this project	41
Learn it	44
10-minute exercise	44
Done when	45
Chapter 7. OpenRouter and benchmark literacy: one door to every model, and how to read the scoreboard	45
What it is (plain words and an analogy)	45
Why this tool now (for this project)	46
How it is used in this project	46
Learn it	50
10-minute exercise	50
Done when	51
Chapter 8. “Fine-tuning” at three levels: the harness, the context, and the model	51
What it is (plain words and an analogy)	51
Why this tool now (for this project)	52
How it is used in this project	52
Learn it	56
10-minute exercise	56
Done when	57
Chapter 9. Data sources: where every number comes from, and what you may do with it	57
What it is (plain words and an analogy)	57
Why this tool now (for this project)	57
How it is used in this project	58
Learn it	63
10-minute exercise	64
Done when	65
Chapter 10. OCR, web crawling and sub-agents: three helpers, and when to leave them in the drawer	65
What it is (plain words and an analogy)	65
Why this tool now (for this project)	66
How it is used in this project	67
Learn it	69
10-minute exercise	70

Done when	71
Chapter 11. Evals and Langfuse: how you find out whether the model is right	71
What it is (plain words and an analogy)	71
Why this tool now (for this project)	71
How it is used in this project	72
Learn it	74
10-minute exercise	75
Done when	76
Chapter 12. Hex: a private dashboard over Neon	76
What it is (plain words and an analogy)	76
Why this tool now (for this project)	76
How it is used in this project	77
Learn it	79
10-minute exercise	79
Done when	80
Chapter 13. Figma and FigJam: the whiteboard where the system is drawn	80
What it is (plain words and an analogy)	80
Why this tool now (for this project)	81
How it is used in this project	81
Learn it (a two-hour starter path)	82
10-minute exercise	84
Done when	84
Chapter 14. Security hygiene for a solo builder	85
What it is (plain words and an analogy)	85
Why this tool now (for this project)	85
How it is used in this project	86
Learn it	87
10-minute exercise	88
Done when	88
Chapter 15. The Damodaran watch and read order for this project	88
What it is (plain words and an analogy)	89
Why this tool now (for this project)	89
How it is used in this project	89
Learn it	90
10-minute exercise	92
Done when	93
Chapter 16. The schedule: twelve evenings-only weeks, from “go” to a working system	93
What it is (plain words and an analogy)	93
Why this tool now (for this project)	93
How it is used in this project	93
Learn it	97
10-minute exercise	98

Done when	98
Chapter 17. Glossary: every term in the guide and Document A, in plain words	98
What it is (plain words and an analogy)	98
Why this tool now (for this project)	98
How it is used in this project	99
The glossary	99
Learn it	102
10-minute exercise	102
Done when	103

How to read this document

This is document B of four. Document A (“Read This First”) describes the system in plain words. This guide teaches the tools the system uses, one chapter per tool, so that when a contractor does its job you understand what it did and can check it.

Each chapter has the same shape: what the tool is (with an analogy), why this tool and not another, how it is used in this project, where to learn it for free with hours, a ten-minute exercise, and a “done when” line so you know when to stop.

You do not need to read it front to back. Chapter 0 (how to learn with an AI tool) and chapter 2 (Cursor) come first because you will touch them every day. After that, follow the schedule in chapter 16: it names the chapters to read each week, alongside the Damodaran sessions in chapter 15. Chapter 17 is the glossary; keep it open in a second window.

Nothing in this guide asks you to open an account or install anything today. Every account you will eventually need is listed in document A, section 10, and the schedule tells you the week to open each one. Nothing after M0 starts until you say go.

Chapter 0. How to learn with an AI coding tool without becoming dependent on it

READ FIRST

Read this before the tools manual. Focus on the attempt-then-ask habit, explaining changes in your own words, and the learning checkpoints.

What it is (plain words and an analogy)

An AI coding tool is a program that reads your files, writes code, and runs commands when you ask in plain English. In this project the tool is Cursor, with Cline as a free second tool (chapter 2). People also call such a tool a “harness” or an “agent”. All three words mean the same thing here.

Think of it as a very fast junior colleague who never sleeps and never says “I am not sure”. The speed is the gift. The false confidence is the trap. A junior colleague who is wrong once a day is fine if you check the work. The same colleague is dangerous if you stop checking.

Damodaran has a name for the trap. He calls it “the Google curse”, the growing habit of retrieving answers rather than reasoning toward them. He adds: “Reasoning is a muscle. If you stop using it, evolution takes it away.” He has drawn a related line since 2012. Using first principles and using the market as a check are healthy responses to uncertainty. “Outsourcing by off-loading the decision making to others” is not.

Source: Valuation/docs/sources/bkm2kob5x.txt, section “How to learn with an AI harness”, quoting <https://imaa-institute.org/blog/investing-age-of-ai-damodaran-imaa-webinar/> and <https://rpc.cfainstitute.org/blogs/enterprising-investor/2012/addressing-uncertainty-in-investment-valuations> (2026-09-14)

He also says where a machine should do the work. In Data Update 1 for 2026 he writes that AI bots “will not only match, but be better than I am, at mechanical and rule-based tasks”. He plans to hand

almost his entire data-compilation process to a bot. So the split is his own: the machine does the mechanical half, and the person keeps the reasoning half.

Source: [financeMD/damodaran/blog/2026/01/data-update-1-for-2026-push-and-pull-of.md](#) (2026-01-09)

Why this matters now (for this project)

This project's whole value is being right about numbers that other people may one day trust. The one thing a harness cannot supply is the instinct that a number is off by a factor you have not found yet. That instinct is built by doing the arithmetic yourself, many times. Nothing in the tooling will budget for that on your behalf, so you must.

The approved plan lists “dependence on the AI tool” as a named risk. Its mitigations are the rules below: **attempt-then-ask**, own the tests, one no-AI evening a week, and a decisions log.

Source: [Valuation/docs/plan.md](#), section 9 (2026-09-14)

How it is used in this project

- Week 1 you create accounts and read, with no code. From week 2 you drive Cursor, one small change at a time.
- Milestone M3 (“you take the wheel”) ends when five pull requests are merged by you, the watch-list has 20 names, and you can explain every rule in `AGENTS.md` out loud. `AGENTS.md` is a plain text file at the top of the repository that every AI tool reads first. A pull request (PR) is explained in chapter 1.
- M3 also asks you to track your ratio of lines written to lines accepted (rule 8 below).
- The day-one exercise in M1 is the pattern for everything after it: ask the tool, then find the answer yourself in the workbook, then compare.

Source: [Valuation/docs/plan.md](#), sections 4 and 5 (M1, M3) (2026-09-14)

The ten operational rules, each with a 5-minute exercise

These are the rules from the learning-path report, restated in plain words. Rule 9 was written for a terminal tool called pi and is translated here for Cursor.

Source for all ten: [Valuation/docs/sources/bkm2kob5x.txt](#), section “How to learn with an AI harness” (2026-09-14)

Rule 1. Attempt, then ask

Before any prompt, write two or three sentences in a scratch file. Say what you think the answer is and why. Then ask the tool. Compare the two. The difference is your learning signal. With no attempt there is no difference, and nothing is learned. This single rule does most of the work.

Five-minute exercise: create `~/Desktop/Valuation/scratch/attempt.md`. Write today's date and one guess: “I think a branch in git is ...”. Do not look anything up yet. You will check it in chapter 1.

Rule 2. You own the test; the tool owns the implementation

A test is a small program that checks a bigger program and reports pass or fail. Never let the tool write both the check and the code that satisfies it. The golden number for the Almarai example, 7.187840270062114 value per share, is the model. The expected values are extracted from the work-

book by a script, and the fixture file (the file of expected answers the test compares against) is protected in CI (continuous integration, the automatic check that runs on every pull request; chapter 1) so an agent can never “fix” the test instead of the code. If the tool proposes changing a test, that is a design conversation, not an edit.

Source: Valuation/docs/plan.md, sections 1 and 10.1 (2026-09-14)

Five-minute exercise: open `financeMD/damodaran/pc/fcffsimpleginzu.md` and search for “Estimated value /share”. Write the number you find in your attempts file, with the line number. That is the first invariant you own.

Source: `financeMD/damodaran/pc/fcffsimpleginzu.md`, line 111 shows 7.18784027 (workbook dated 2026-02-01 on the Almarai row)

Rule 3. Explain back before you merge

No PR merges until you can say out loud, without looking, what each changed file does and why. If you cannot, you have a PR you cannot maintain. Revert it and redo it in smaller pieces. This is the review step a solo developer otherwise skips.

Five-minute exercise: pick any file in `~/Desktop/Valuation/docs/`. Close it. Say in two sentences what it is for. Open it and check.

Rule 4. One no-AI evening a week

Pick the second evening of each week. Type it yourself. Look things up in the official docs. Be slow. You are training recall, and recall is what lets you notice when the tool is confidently wrong. Expect this to feel bad for about four weeks and then to stop feeling bad.

Five-minute exercise: put a repeating calendar block on the second evening of each week named “no-AI evening”. Keep it for twelve weeks.

Rule 5. Read the diff, not the summary

A diff is the list of lines added and removed. The summary of a change is written by the same tool that made the change. `git diff` and `gh pr view` show the ground truth. Make it a physical habit: diff first, summary second, never summary only.

Five-minute exercise: in Cursor, open the Source Control panel (the branch icon in the left bar). Note where a file’s diff appears. You will use this view in every PR.

Rule 6. Keep a decisions log, written by you

A decisions log is a folder of short notes, one per decision. The format is `docs/decisions/NNN-title.md`, about ten lines each, written by a human. Each note says what was decided, what was rejected, and why. It is the only artifact that proves you made the call. It is also the context you hand the tool next month, so it stops re-opening settled choices.

Five-minute exercise: create `~/Desktop/Valuation/docs/decisions/001-daily-tool-is-cursor.md`. Write: decided (Cursor as the daily tool), rejected (pi, a second paid tool), why (already paid, shows diffs, has a plan mode). The reasons are in plan section 2.

Source: Valuation/docs/plan.md, section 2 (2026-09-14)

Rule 7. Spend the tool on mechanical work, spend yourself on judgment

Mechanical work you may delegate freely: mapping XBRL tags (the labels inside SEC filings), parsers for CSV and spreadsheet files, boilerplate, migration scaffolding, test fixtures, document formatting, regular expressions. Judgment you never delegate: which vintage (data snapshot) applies, whether a seven-firm India industry falls back to the emerging-markets table or the global one, whether a cash flow is real, and what the app is allowed to say to a user. This is Damodaran's own split. He hands the bot the data compilation and keeps the narrative.

Five-minute exercise: draw a two-column table titled "mechanical" and "judgment". Add five items to each from your own week. Keep it in the attempts file.

Rule 8. Measure your ratio, honestly

Once a week, note two numbers. First: lines you wrote against lines you accepted from the tool. Second: how many times you accepted something you could not explain. The second number should trend to zero by week 6. If it does not, you are accumulating a codebase you rent rather than own.

Five-minute exercise: add a table to the attempts file with columns week, lines written, lines accepted, unexplained acceptances. Fill in week 1 with zeros. Ratios only mean something once the row is there.

Rule 9. Use branches as a thinking tool, not an answer machine

The original rule says to fork the session before a risky change and explore two approaches in parallel. In Cursor the same move is a git branch (chapter 1). Before a risky change, make a branch for approach A and another for approach B. Let the tool try both. Then you choose, and you write the choice in the decisions log. The tool generates options. You exercise the muscle that picks.

Five-minute exercise: in the attempts file, write one change you fear making to a valuation formula. Name the two branches you would create, for example `try-lag-on` and `try-lag-off`.

Rule 10. The corpus is the antidote to the tool

You have a world expert's own reasoning on disk, roughly 30 million tokens of it. A token is a piece of a word, about three quarters of a word on average. When you do not understand why the stable-growth cost of capital is the risk-free rate plus a mature-market premium, the right move is not "ask the model". The right move is to find the passage where he says it, then ask the model to check your restatement. Retrieval from a primary source is not the Google curse. Retrieval instead of reasoning is. The librarian in this system (the search tool over the corpus) is built to always show you the source passage, which pushes you toward the first behavior.

Five-minute exercise: run this in a terminal and read the two lines it prints.

```
grep -n "riskfree rate + 4.5%" financeMD/damodaran/pc/fcffsimpliginzu.md
grep -n "Mature Market ERP +" financeMD/damodaran/pc/fcffsimpliginzu.md
```

The first line is the workbook's own note that a mature company's cost of capital is about the risk-free rate plus 4.5%. The second is the mature-market equity risk premium cell, 0.0423, updated January 1, 2026. Notice that they are not the same number. Write one sentence about why that might be. Do not ask the tool yet.

Source: `financeMD/damodaran/pc/fcffsimpliginzu.md`, lines 48 and 608 (ERP cell dated January 1, 2026)

Learn it

Resource	Format	Hours	Why
IMAA Institute write-up of Damodaran's "Investing in the age of AI" webinar, https://imaa-institute.org/blog/investing-age-of-ai-damodaran-imaa-webinar/	article	about 0.5 (my estimate, not from a source)	The source of "the Google curse" and "reasoning is a muscle"
CFA Institute Enterprising Investor, "Addressing uncertainty in investment valuations" (2012), https://rpc.cfainstitute.org/blogs/enterprising-investor/2012/addressing-uncertainty-in-investment-valuations	article	about 0.5 (my estimate)	His healthy versus unhealthy responses to uncertainty; "outsourcing" is the unhealthy one
Data Update 1 for 2026, local copy at financeMD/damodaran/blog/2026/01/data-update-1-for-2026-push-and-pull-of.md , online at https://aswathdamodaran.blogspot.com/2026/01/data-update-1-for-2026-push-and-pull-of.html	blog post	about 0.5 (my estimate)	Where he says which half of the work goes to the bot
The corpus itself, financeMD/, searched with <code>grep -rn "phrase" path</code>	local markdown	ongoing	Rule 10. Every "why" question has a primary source on disk

All free. Hours marked "my estimate" are not from any report; they are reading-time guesses.

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

This is the day-one exercise from the plan, with rule 1 added.

1. In the attempts file, write two sentences: why you think the cost of capital after year 10 is the risk-free rate plus a mature-market premium.
2. Ask Cursor (chapter 2, plan mode, do not execute): "Explain why the terminal cost of capital in `fcffsimpleginzu` is the risk-free rate plus the mature-market ERP."
3. Open the workbook markdown yourself and find the cell label Mature Market ERP + and the note "riskfree rate + 4.5%".
4. Write three lines: what you guessed, what the tool said, what the workbook says. Note any place the tool was more confident than the workbook.

Source: Valuation/docs/plan.md, section 5, M1 step 6 (2026-09-14);
financeMD/damodaran/pc/fcffsimpleginzu.md, lines 48 and 608

Done when

KEY IDEA

- `~/Desktop/Valuation/scratch/attempts.md` exists with at least one attempt-then-ask entry and the week-1 ratio row.
- `~/Desktop/Valuation/docs/decisions/001-daily-tool-is-cursor.md` exists and is under fifteen lines.
- The no-AI evening is on your calendar for twelve weeks.
- You can list the ten rules from memory, in any order, in your own words.
- You have found one answer in the corpus with `grep` before asking the tool about it.

Chapter 1. Git, GitHub, branches, pull requests and CI checks

OPTIONAL - FIRST PASS

For the code-work stage. Start with the vocabulary and history model; return to the commands and exercises when working on code.

What it is (plain words and an analogy)

Git is a program that keeps a history of every change to the files in a folder. It runs on your Mac. Each saved snapshot is called a commit, and each commit has a short message. The folder plus its history is called a repository, or repo.

A branch is a separate line of commits that starts from a copy of the main line. The main line is a branch too, and it is called `main`. You do work on a branch so that `main` stays clean until the work is checked.

GitHub is a website that stores a copy of your repo and adds three things on top of git. A pull request (PR) is a request to merge a branch into `main`, shown as a diff with a comment thread. A diff is the list of lines added and removed. Issues are a to-do list attached to the repo. GitHub Actions is a robot that runs scripts when something happens, for example on every PR or every night at a set time.

CI stands for continuous integration. It means “run the checks automatically on every PR”. A check is a script that passes or fails, such as a test or a type check. Branch protection is a rule that `main` can change only through a PR whose checks pass. A merge conflict happens when two branches changed the same lines and git cannot pick one; you pick.

`gh` is GitHub’s command-line tool. It lets you make PRs and watch checks from the terminal instead of the website.

The analogy: a family recipe book. `main` is the printed book on the shelf. A branch is a photocopy of one page that you scribble on. A commit is each time you put the pencil down and date the margin. A PR is handing the photocopy to the family with your changes marked in red. CI is the taste test that runs by itself before anyone reads the page. A merge is pasting the approved page back into the book. A merge conflict is two cousins who both rewrote step 3, and someone has to decide.

Why this tool now (for this project)

Every week of the twelve-week plan ends with a merged PR that CI passed. That is the unit of progress. If you cannot make a branch, open a PR, and read a diff, you cannot do rule 5 from chapter 0.

Source: Valuation/docs/sources/bkm2kob5x.txt, “12-week evening schedule” (2026-09-14)

GitHub Actions is also where the correctness rules live. The golden test runs there. A PR that moves the Almarai value per share off 7.187840270062114 must go red, and red means it cannot merge. The same Actions robot runs the nightly jobs (pull new filings, re-value the watchlist, check for a new risk-premium file).

Source: Valuation/docs/sources/bkm2kob5x.txt, section 2 (2026-09-14); Valuation/docs/plan.md, section 10.6 (2026-09-14)

Cost: the repo is private, inside a free GitHub organization you own. A private repo on the free plan gets 2,000 Linux minutes of Actions per month, then \$0.006 per minute, with a \$25 spending cap set in the plan. Public repos get unlimited minutes, but this repo stays private.

Source: Valuation/docs/sources/bkm2kob5x.txt, section 0 (2026-09-14); Valuation/docs/plan.md, sections 2 and 5 M2 (2026-09-14)

How it is used in this project

- The repo is named `valuation`. It is created in M2 inside your organization, then handed to you. Your GitHub account is `jkv8fwytfb-byte`.
- Week 1 security, before any code: turn on two-factor authentication (2FA, a second code from an app at login) and save the recovery codes offline. Turn on secret scanning and push protection, so GitHub physically refuses a commit that contains an API key. Use fine-grained personal access tokens, one per purpose, scoped to one repo, with an expiry.
- `main` is protected. Every change goes: branch, commits, PR, CI green, explain-back (rule 3), merge with squash. Squash means the branch’s many commits become one commit on `main`, so the history reads like a diary.
- CI on every PR runs a type check, a linter (a style checker), and `vitest` (the test runner). The golden test is one of those tests. The fixture file holding the expected numbers is protected in CI, so an agent cannot edit the answer key.
- Nightly jobs are Actions too: `ingest-edgar`, `value-watchlist`, `watch-erp-crp`, `rag-eval`, `news-headlines`, `realised-prices`. Each writes a heartbeat row, and a staleness alarm fires after 36 hours of silence. Schedules run off the hour (for example 03:17, not 03:00), because GitHub drops many jobs scheduled exactly on the hour. A cron schedule is the line in the Actions file that says when a job runs.
- Nothing licensed ever enters `git`. `.gitignore` (a list of files git must ignore) covers `.env*` files, caches, and raw data dumps. Keys live in three places only: the local `.env` file, GitHub Actions secrets, and the hosting provider’s environment settings.

Source: Valuation/docs/plan.md, sections 5 (M1, M2) and 10.6 (2026-09-14); Valuation/docs/sources/bkm2kob5x.txt, sections 1, 2 and 15 (2026-09-14)

The six gh commands

Learn exactly these six and stop. Cursor’s agent runs `gh` from the terminal, so knowing them lets you read what the agent did to your repo.

Command	What it does, in plain words
<code>gh repo create</code>	Makes a new repo on GitHub from the folder you are in.
<code>gh pr create</code>	Opens a pull request for the branch you are on. It asks for a title and a description.
<code>gh pr view --web</code>	Opens the current PR in your browser so you can read the diff and comments.
<code>gh pr checks</code>	Lists the CI checks on the current PR and whether each passed, failed, or is still running.
<code>gh pr merge --squash --delete-branch</code>	Merges the PR as one commit and deletes the branch. Only works when checks are green and protection allows it.
<code>gh run watch</code>	Watches an Actions run live in the terminal, line by line, until it finishes.

Source: Valuation/docs/sources/bkm2kob5x.txt, section 1 (2026-09-14). The one-line descriptions are mine, from using the tool; the manual at <https://cli.github.com/manual/> is the reference.

The daily loop, as commands

```
git switch -c add-costco-to-watchlist    # new branch, named for the change
# ... edit files, or let Cursor edit them, then read the diff ...
git diff                                # rule 5: read it
git add -A
git commit -m "Add Costco to the watchlist"
git push -u origin add-costco-to-watchlist
gh pr create                             # opens the PR
gh pr checks                             # wait for green
gh pr merge --squash --delete-branch     # after explain-back
git switch main && git pull               # back to the trunk
```

Learn it

All free. Hours are the ones stated in the learning-path report, which checked every link on 2026-09-14.

GitHub Skills (interactive courses that run inside a real repo, graded by Actions)

Course	URL	Hours
Introduction to GitHub	https://github.com/skills/introduction-to-github	under 1
Communicate using Markdown	https://github.com/skills/communicate-using-markdown	about 0.5
Review pull requests	https://github.com/skills/review-pull-requests	about 0.5
Resolve merge conflicts	https://github.com/skills/resolve-merge-conflicts	under 0.5
Release-based workflow	https://github.com/skills/release-based-workflow	about 1
Connect the dots (issues to PRs)	https://github.com/skills/connect-the-dots	about 0.5

Why: you do the work in a real repo and a robot grades it. Go to the repo URLs directly; the github.com/skills front page now leads with Copilot exercises, but these classic courses still work.

Pro Git, 2nd edition (Chacon and Straub), free online book

URL: <https://git-scm.com/book/en/v2>. Read only these, in this order: chapter 2 Git Basics (about 1.5 hours), chapter 3 Git Branching (about 2 hours, especially 3.1 to 3.3 and 3.6), chapter 6 GitHub (about 1 hour), and chapters 7.6 Reset Demystified and 7.7 Advanced Merging (about 1 hour) the first time you wreck a branch. Skip chapters 4, 9 and 10; they are server administration and internals. Why: it is the reference the tooling is written against, and chapter 3's diagrams are the best cure for "I do not know what a branch actually is".

gh CLI manual

URL: <https://cli.github.com/manual/>, about 1 hour to skim. Why: you only need the six commands above, and the manual is where you check a flag.

GitHub Actions

- Quickstart, <https://docs.github.com/en/actions/get-started/quickstart>, about 0.5 hours.
- Understand GitHub Actions, <https://docs.github.com/en/actions/get-started/understand-github-actions>, about 1 hour.
- Hands-on: <https://github.com/skills/hello-github-actions> (about 1 hour), then <https://github.com/skills/test-with-actions> (about 1 hour).

Why: this is where the golden test and the nightly robots live.

Security, week 1

- 2FA: <https://docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/about-two-factor-authentication>
- Fine-grained tokens: <https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>
- Secret scanning and push protection: <https://docs.github.com/en/code-security/secret-scanning/introduction/about-secret-scanning>

Total: about 8 hours for git, GitHub and gh, about 5 hours for Actions, and about 4 hours for the security set.

Source: Valuation/docs/sources/bkm2kob5x.txt, sections 1, 2 and 15 (2026-09-14)

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

Create and resolve a merge conflict on a throwaway repo. No AI tool. This is the first item on the M3 exercise list, done early on scratch files so the real repo is safe.

```
mkdir -p ~/Desktop/Valuation/scratch/conflict-drill && cd
  ↪ ~/Desktop/Valuation/scratch/conflict-drill
git init
echo "growth = 0.05" > inputs.txt
git add inputs.txt && git commit -m "Start"
git switch -c cousin-a
echo "growth = 0.06" > inputs.txt
```

```
git commit -am "Cousin A raises growth"
git switch main
echo "growth = 0.04" > inputs.txt
git commit -am "Main lowers growth"
git merge cousin-a
```

Git will stop and say there is a conflict in `inputs.txt`. Open the file. You will see both versions between markers that look like `<<<<<<`, `=====` and `>>>>>>`. Delete the markers and keep the line you decide is right. Then:

```
git add inputs.txt
git commit -m "Resolve: keep 0.05, neither cousin cited a source"
git log --oneline --graph
```

Read the graph. That picture, two lines joining, is what a merge is. Write in the attempts file (chapter 0) what you guessed a branch was and what you now think it is.

Source: Valuation/docs/plan.md, section 5 M3 (2026-09-14). The commands are standard git; Pro Git chapter 3.2 covers the same drill.

Done when

KEY IDEA

- You can draw main, a branch, a PR and a merge on paper and explain each word without notes.
- You have resolved one merge conflict by hand in the drill above.
- 2FA is on, the recovery codes are saved offline, and push protection is on for the valuation repo.
- You have run each of the six gh commands at least once (the merge one on a real PR).
- Your first PR is merged by you, with CI green, and you explained every changed file out loud first.
- The Skills courses “Introduction to GitHub” and “Resolve merge conflicts” show as completed in your GitHub account.

Chapter 2. Cursor and Cline: the tool you type into, and the free bench beside it

READ FIRST

Understand the everyday workflow. Learn how the editor, assistant, modes, and review of changes fit together; use the setup details when needed.

What it is (plain words and an analogy)

Cursor is a code editor with an AI agent built in. A code editor is a program for writing code files, the way Word is a program for writing letters. An agent is a model that can read your files, edit them, and run commands when you ask in plain English. Cursor is a modified copy (a “fork”) of VS Code, the most common free editor, so it keeps the built-in terminal, the Source Control panel, and side-by-side diffs.

Source: btzaixzyh.txt, CHECK 1, “Terminal, git panel, diff review” (2026-09-14)

Think of Cursor as a workshop with a fast apprentice. The apprentice drafts every change. You are the owner, and nothing is built until you look at it and say yes. Chapter 0 gave you the habits. This chapter gives you the controls.

Cline is a free, open-source extension. An extension is a plug-in that adds a feature to an editor. Cline runs inside the same Cursor window but brings its own connection to a model provider, in our case OpenRouter. Think of Cline as a test bench next to the workshop: a place to try a tool before it goes onto the production line.

Source: btzaixzyh.txt, CHECK 1, section 2, Cline (2026-09-14)

Why this tool now (for this project)

- You already pay for Cursor Pro at \$20 a month. The plan says: do not add a second paid harness. A harness is the program that wraps a model with tools and a screen.
- You are a beginner building a system you will not fully understand on day one. Reviewing each change as a diff, and reading a plan in English before code is written, is the difference between learning and copying.
- Privacy is a stated requirement. With Privacy Mode on, Cursor holds zero-data-retention agreements with all its model providers.
- The budget ceiling is \$200 a month, and the Cursor line is fixed at \$20. Every other dollar is better spent on the system's own model calls.

Source: btzaixzyh.txt, CHECK 1, "RECOMMENDATION for you specifically" (2026-09-14); read-this-first.md sections 11 and 12 (2026-09-14)

There are two separate places where an AI model is used, and this chapter is only about the first. Place 1 is while you write code: that is Cursor, on the models Cursor includes. Place 2 is inside the product when it runs at night: that is our own TypeScript code calling GLM-5.3 and friends through OpenRouter. Document A section 4 explains the split. Keep it in your head while reading the rest.

Source: read-this-first.md section 4 (2026-09-14); plan section 2b (2026-09-14)

How it is used in this project

The plan you are on, and what the two token pools mean

Cursor stopped counting "requests" in June 2025. It now bills tokens, which are word pieces of roughly four characters, against two monthly pools. A pool is an allowance that refills each month.

Pool	Which models	How it bills
Cursor Models pool	Composer 2.5, Grok 4.6, Grok 4.5	Very generous; in practice it does not run out
Other Models pool	Anthropic, OpenAI, Google, Meta, Moonshot, Z.ai	Charged at each model's raw API price per million tokens; this is the pool that empties

On Pro you get the Cursor Models pool plus roughly \$20 of Other Models usage. Third-party trackers report the \$20 figure consistently, but Cursor's own pages no longer print it, so treat it as approximate. Heavy agent work on frontier models can use it up in days, after which you pay overage. Auto mode has three settings, Cost, Balance and Intelligence, and bills at the price of whichever model it picks.

Source: btzaixzyh.txt, CHECK 1, "Plans and prices" and "Requests no longer exist" (2026-09-14)

The ladder is Hobby (free), Start (India only, ₹649 a month), Pro (\$20, about \$20 of usage), Pro+ (\$60, about \$70) and Ultra (\$200, about \$400). You stay on Pro. Your bottleneck is your learning rate and

the pipeline's token spend, not editor capacity. Use Auto or Composer for routine edits, and a frontier model only when a plan needs deep reasoning.

Source: btzaixzyh.txt, CHECK 1, plans table and recommendation point 4 (2026-09-14)

Privacy Mode, and why no API key ever goes into Cursor

Privacy Mode is a setting under Settings, then General. With it on, your code is never used for training, and Cursor's zero-data-retention agreements apply. Zero data retention (ZDR) means the model provider deletes your prompt and files after answering instead of storing them. Privacy Mode is on by default only for Enterprise accounts. Individuals must switch it on themselves. Do this before you open the repo for the first time.

Source: btzaixzyh.txt, CHECK 1, "Privacy / ZDR" (2026-09-14); read-this-first.md section 11, step 1 (2026-09-14)

Cursor lets you paste your own provider key, called BYOK, "bring your own key". Do not. The BYOK help page says, in Cursor's own words, that its Zero Data Retention policy does not apply when you use your own keys. Three more things break: Tab completion never uses your key, Auto and Composer may not route through it, and OpenRouter is not officially supported at all. Routing open-weight models into Cursor would also waste the pools you already pay for. So the rule is simple: your OpenRouter key goes into the repo's `.env` file, into GitHub secrets, and into Cline's own settings. It never goes into Cursor's settings.

Source: btzaixzyh.txt, CHECK 1, "BYOK and OpenRouter" and "Is open-weight via OpenRouter still meaningful inside Cursor" (2026-09-14); read-this-first.md section 10 (2026-09-14)

Plan mode and Agent mode (Shift+Tab)

The agent panel has two modes, and Shift+Tab switches between them. In Plan mode the agent asks clarifying questions, reads the code, and writes an implementation plan in English. You edit that plan in the chat or as a markdown file, and nothing on disk changes until you hand it to Agent mode. In Agent mode the agent edits files and runs commands. Use Plan mode for anything larger than a one-line edit. This is the workflow that lets you approve the idea before you see the code.

Source: btzaixzyh.txt, CHECK 1, "Agent mode / Plan mode" (2026-09-14); read-this-first.md section 11, step 4 (2026-09-14)

Reading diffs hunk by hunk

Every agent edit lands as a diff: old lines in red, new lines in green. A diff is split into hunks, and a hunk is one contiguous block of changed lines. Cursor lets you accept or reject each hunk before anything is committed. Rejecting is free and leaves the file as it was. Chapter 0, rule 5, says read the diff, not the summary. This is where you do it. When a hunk touches a calculator formula, read it twice, then check the golden test still passes (chapter 3).

Source: btzaixzyh.txt, CHECK 1, "Terminal, git panel, diff review" (2026-09-14); read-this-first.md section 11, step 5 (2026-09-14)

The rules file

The folder `.cursor/rules/` will hold the hard rules of the project on one page: golden test, vintage, source and license class, India fallback, ranges not points, no advice. Cursor reads these rules on every request, so the agent cannot "forget" them between chats. The repo will also carry `AGENTS.md`,

a plain file with the same rules that Cursor, Cline and Claude all read. Both files arrive with the repo in M2.

Source: read-this-first.md sections 3 and 11, step 3 (2026-09-14); plan, M2 step 1 (2026-09-14)

MCP and .cursor/mcp.json

MCP stands for Model Context Protocol. It is a standard plug that lets an AI tool call other programs, the way a USB port lets any laptop use any keyboard. A program that offers tools this way is an MCP server. Cursor reads the list of servers from .cursor/mcp.json in the repo (or ~/.cursor/mcp.json for all repos) and supports three ways of connecting to a server (called transports): over a local process, or over the network by SSE or Streamable HTTP. The repo’s file will use the local one. The repo will ship this file, and the agent panel will then list the system’s own tools: value_company, write_memo, review_memo, get_industry_stats, search_corpus and report. The same server will serve Cline and Claude, so there is one calculator with several callers, not several drifting copies.

Source: btzaixzyh.txt, CHECK 1, “MCP” (2026-09-14); read-this-first.md section 11, step 2 (2026-09-14); plan, M2 step 7 (2026-09-14)

Cloud agents, BugBot and Automations

Cursor offers three services that run away from your laptop. Each draws on your usage pool; the table shows how each one is charged.

Service	What it does	How it bills	Our use
Cloud agents	Comment @cursor on a GitHub pull request or issue; it clones the repo in a cloud machine, works on a branch, and pushes back for review	API price of the model chosen, from your pool, with a spend limit you set	Occasional, for a chore you describe in a PR comment
BugBot	Reviews PR diffs on GitHub, leaves inline comments with fixes, can spawn a cloud agent to apply them	Usage-based since about 8 June 2026; a third-party estimate is \$1.00 to \$1.50 per run, and a run is every push, not every PR	Turn on manually for big PRs, not on every push
Automations	Scheduled agents, shipped 5 March 2026: cron expressions (a compact way of writing a schedule such as ‘every night at 02:15’), GitHub events, Slack, webhooks (a URL another service calls to trigger a run), Linear, Sentry, PagerDuty; each run gets a cloud sandbox, your MCP servers, and a memory across runs	Billed as cloud-agent usage from your pool	Repo chores only: summarize what changed, triage a PR

Source: btzaixzyh.txt, CHECK 1, “Cloud / Background Agents”, “BugBot”, “Scheduled jobs” (2026-09-14)

Why the nightly pipeline still lives on GitHub Actions and not in Automations: Automations bill through Cursor, tie the data pipeline to your editor vendor, and cost more than a plain GitHub Actions cron job calling OpenRouter directly. GitHub Actions keeps the keys in repo secrets and stays independent of whichever editor you use in 2028. Chapter 3 covers that side.

Source: btzaixzyh.txt, CHECK 1, “How it pairs with GitHub Actions” (2026-09-14); plan section 2b (2026-09-14)

Cline, the free bench

Cline is licensed Apache 2.0, had about 1.5 million VS Code Marketplace installs by April 2026, and costs \$0 as a tool. You install it from the Extensions panel inside Cursor, because Cursor runs VS Code extensions. In Cline's provider settings you pick OpenRouter and paste your OpenRouter key. That key lives in Cline, and Cline talks to OpenRouter under the settings you set there in M1: monthly limit \$75, prompt logging off, zero-data-retention routing on.

Source: btzaixzyh.txt, CHECK 1, section 2, Cline (2026-09-14); read-this-first.md section 10, row 3 (2026-09-14)

Cline has two modes, Plan and Act. Plan can read the code, search and discuss, but cannot write files or run commands. Act executes. History carries across the switch. The useful trick is under Settings, "Use different models for Plan and Act": one model to think, a cheaper one to type. The plan pairs GLM-5.3 for Plan with GLM-5.3-Flash for Act. This is your bench for feeling what an open-weight model does on your code before its id goes into a nightly robot. Typical BYOK spend cited for individuals is \$5 to \$50 a month; used sparingly, yours sits inside the \$75 OpenRouter cap.

Source: btzaixzyh.txt, CHECK 1, section 2 and recommendation (2026-09-14); plan, M1 step 5 (2026-09-14)

Kilo and pi, in one paragraph each

Kilo is a fork of Roo Code, which is itself a fork of Cline. Anaconda acquired it in July 2026. It is free for individuals, offers 500 or more models at zero markup through its gateway, and works in VS Code, JetBrains and a CLI. It is a fair substitute for Cline if you prefer its modes, but it is the youngest and fastest-moving of the three, so Cline is the safer bench.

Source: btzaixzyh.txt, CHECK 1, section 3, Kilo (2026-09-14)

pi (pi.dev) is a minimal terminal-only agent, MIT-licensed, that ships four tools, read, write, edit and bash, and treats everything else as an extension you write in TypeScript. It has no permission pop-ups by design. It is deferred, not rejected: the runtime here is a scheduled TypeScript program, not a chat harness, so you do not need to build a coding tool. pi is worth a look later for scripted agent runs inside a nightly job, since it can also run non-interactively, printing its output or streaming it as machine-readable JSON for another program to read.

Source: btzaixzyh.txt, CHECK 1, section 4, pi (2026-09-14); plan section 2b (2026-09-14); bkm2kob5x.txt, learning path section 9 (2026-09-14)

Learn it

Hours are my estimates unless a source is named. All free.

Resource	Format	Hours	Why
Cursor docs, Agent modes: https://cursor.com/docs/agent/modes	Web page	0.5	Plan mode versus Agent mode, the one control you use daily
Cursor docs, Models and pricing: https://cursor.com/docs/models-and-pricing	Web page	0.5	Read the two pools yourself, and find the Auto setting

Resource	Format	Hours	Why
Cursor privacy page: https://cursor.com/docs/enterprise/privacy-and-data-governance and the BYOK page: https://cursor.com/help/models-and-usage/api-keys	Web pages	0.5	Confirm the ZDR promise and the sentence that BYOK voids it
Cursor docs, MCP: https://cursor.com/docs/context/mcp , plus the MCP intro: https://modelcontextprotocol.io/docs/getting-started/intro	Web pages	1 (MCP intro about 0.5 per the learning path)	Understand <code>.cursor/mcp.json</code> before the repo ships it
Cursor docs, Rules page (cursor.com/docs , search “Rules”; verify the exact URL)	Web page	0.5	How <code>.cursor/rules/</code> is read on every request
Cline docs, Plan and Act: https://docs.cline.bot/features/plan-and-act	Web page	0.5	The per-mode model split, and what Plan cannot do
Optional: BugBot https://cursor.com/docs/bugbot and Automations https://cursor.com/docs/cloud-agent/automations	Web pages	0.5	Skim so you know what you are choosing not to use for the pipeline

Source: URLs from `btzaixzyh.txt`, CHECK 1, Sources list (verified 2026-09-14); MCP hours from `bkm2kob5x.txt`, learning path section 10 (2026-09-14)

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter’s tool. The following “Done when” checklist tells you how to judge completion.

You can do this today, before the repo exists.

1. Open Cursor. Settings, then General, then Privacy Mode: on. Two minutes.
2. Open the folder `~/Desktop/Valuation/docs`. Open the agent panel. Press Shift+Tab until the mode reads Plan. Ask: “Plan a change that adds a one-line glossary entry for the word hunk to `read-this-first.md`, section 15. Do not make the change.” Read the plan. Note which file it says it would touch. Four minutes.
3. Press Shift+Tab to Agent mode. Ask for the same one-line change. When the diff appears, read the red and green lines, then reject the hunk. Open the Source Control panel and confirm no file is modified. Three minutes.
4. Add one line to your decisions log from chapter 0 saying what Plan mode showed you. One minute.

In M1, when your OpenRouter key exists, add the second half: install Cline from the Extensions panel, choose OpenRouter as provider, paste the key, and set GLM-5.3 for Plan and GLM-5.3-Flash for Act.

Source: plan, M1 steps 4 and 5 (2026-09-14)

Done when

KEY IDEA

- Privacy Mode shows as on, and you checked it yourself.
- You can switch modes with Shift+Tab and explain in one sentence what Plan mode cannot do.
- You have rejected a hunk and seen the file unchanged in Source Control.
- You can say where an API key goes (.env, GitHub secrets, Cline's provider setting) and where it never goes (Cursor's settings).
- You can say why the nightly pipeline runs on GitHub Actions and not on Cursor Automations.
- After M2: the agent panel lists the six tools from .cursor/mcp.json, and Cline runs with different models for Plan and Act.

Chapter 3. GitHub Actions: the robot that tests your code and runs it at night

OPTIONAL - FIRST PASS

For the foundation stage. Learn why scheduled jobs and checks exist, then return to the YAML examples when inspecting those jobs.

What it is (plain words and an analogy)

GitHub Actions is a service built into GitHub. It runs commands on a rented computer whenever something happens to your repository. "Something happens" can be a push, a pull request, a timer, or a button you click. The rented computer is called a runner. GitHub creates it, runs your steps, and throws it away.

It does two jobs for you. The first is continuous integration, usually shortened to CI. CI means that every time code changes, the tests run automatically before a human merges. The second job is scheduling. A scheduled workflow runs at a set time with nobody at the keyboard.

Analogy: a night-shift lab assistant. You hand the assistant a checklist, which is the workflow file. The assistant reruns the checklist on every change to your work, and also at 6:17 every morning. If any step fails, the assistant puts a red mark on the change, and you see it before it lands.

Source: <https://docs.github.com/en/actions/get-started/understand-github-actions> (listed in bkm2kob5x.txt, learning path section 2, verified 2026-09-14). The analogy is mine.

Why this tool now (for this project)

There are two reasons, and both come from the plan.

First, the golden test lives here. Damodaran's spreadsheet ships with a worked example, Almarai, whose value per share is 7.187840270062114. Our calculator must reproduce that number to 1e-6, which means to six decimal places. CI turns that rule from a promise into a gate. A pull request that moves the number goes red and cannot merge.

Source: Valuation/docs/plan.md, sections 1 and 10.1 (2026-09-14); bkm2kob5x.txt, learning path section 2.

Second, it is the free scheduler. Vercel, which hosts web pages, allows its Hobby plan only one cron run per day, fired at any minute inside the stated hour. A cron is a timer that starts a job on a fixed

calendar. GitHub Actions has a schedule trigger with no such daily cap. A private repo on the Free plan gets 2,000 Linux minutes per month. So every nightly robot in this system will run on Actions and will write its results to the Neon database. Vercel only serves pages.

Source: bkm2kob5x.txt, learning path sections 0 and 5 (2026-09-14); bo0hrclp7.txt, section 1.4 (Vercel cron docs, last updated 2026-07-15).

How it is used in this project

The plan lists nine jobs: load-damodaran, ingest-edgar, value-watchlist, watch-erp-crp, rag-index, rag-eval, news-headlines, realised-prices, and report. Four run nightly (ingest-edgar, value-watchlist, rag-eval, news-headlines). Three run on demand (load-damodaran, rag-index, report). realised-prices runs weekly. watch-erp-crp checks Damodaran's site monthly and quarterly for a new file. Every job will write a heartbeat row to the database when it finishes.

Source: plan section 10.6 (2026-09-14).

The workflow file, piece by piece

A workflow is one YAML file in the folder .github/workflows/. YAML is a plain-text format where indentation shows nesting, much like Python. Here is the shape of the nightly revaluation job, cut down to its bones:

```
name: value-watchlist

on:
  schedule:
    - cron: "17 6 * * *"          # 06:17 UTC every day, deliberately off the hour
  workflow_dispatch:            # adds a "Run workflow" button in the Actions
    ↪ tab

concurrency:
  group: value-watchlist
  cancel-in-progress: true      # a new run cancels a stale one

jobs:
  run:
    runs-on: ubuntu-latest      # a 2-core Linux runner, the cheapest kind
    timeout-minutes: 45         # never let a hung job sit for six hours
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - run: pnpm install --frozen-lockfile
      - run: pnpm -F engine value-watchlist
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }
      OPENROUTER_API_KEY: ${ secrets.OPENROUTER_API_KEY }
```

What each keyword means:

Keyword	Plain meaning
name	The label you see in the Actions tab.
on	The list of events that start the workflow.
schedule / cron	A timer. The five fields are minute, hour, day of month, month, day of week. Times are UTC.
workflow_ dispatch	Lets you start the job by hand from the website. Always add it, so you can test.
concurrency	If two runs of the same group overlap, cancel the older one. Saves minutes.
jobs	The units of work. Each job gets its own fresh runner.
runs-on	Which kind of rented computer to use.
timeout-minutes	Kill the job after this long.
steps	The checklist, run top to bottom.
uses	Borrow a ready-made step published by someone else. The @v4 pins a version.
run	Run a shell command.
env / secrets	Pass a stored secret into the command as an environment variable (a named value the shell hands to a program when it starts).

Source: the keyword meanings follow the GitHub “Write workflows” documentation at <https://docs.github.com/en/actions/how-tos/write-workflows/> (bkm2kob5x.txt, section 2). The action version numbers in the example are illustrative; check the current major version before copying.

Secrets

A secret is a value like an API key that must never appear in code. You paste it once into the repository settings, under Secrets and variables. Actions then hands it to a job only through `${{ secrets.NAME }}`. The logs mask it. For this project the names are fixed in the plan: `DATABASE_URL`, `DATABASE_URL_UNPOOLED`, `OPENROUTER_API_KEY`, `PINECONE_API_KEY`, `VOYAGE_API_KEY`, the four `R2_*` values, the three `LANGFUSE_*` values, `MISTRAL_API_KEY`, and later `EODHD_API_TOKEN`. The same names go into the local `.env` file, which is never committed.

Source: plan section 7, accounts table (2026-09-14); <https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets> (bkm2kob5x.txt, section 2).

Schedule gotchas

These are the things that make a nightly job silently skip a day. Read them twice.

- **Off-hour minutes.** Runs queued at the top of the hour are delayed, and under load they are dropped. GitHub’s own docs advise picking a different minute. Use `17 6 * * *`, never `0 6 * * *`.
- **The 5-minute floor.** The shortest allowed interval is 5 minutes. You will never need that. Nightly is the cadence here.
- **The 6-hour cap.** A job’s default `timeout-minutes` is 360, which is six hours. A hung API call sits there for six hours at \$0.006 per minute, which is about \$2.16 in silence. Set a real timeout on every job.
- **Minute rounding.** GitHub rounds each job up to the whole minute. Fifty jobs of 20 seconds cost 50 minutes, not 17. For fan-out work, loop inside one job instead of spreading it across a matrix of many jobs (a matrix is an Actions feature that clones one job once per input value, each clone billed separately).

- **The 60-day sleep.** In a public repo, a scheduled workflow is switched off after 60 days with no new commits. The docs say this is public-only. Community reports disagree. Treat a private repo as probably safe, and use the heartbeat below rather than trusting either side.
- **The spending cap.** Set a hard Actions spending limit of \$25 in the organization's billing settings before the first scheduled workflow exists. A retry loop in an ingest job is the realistic way to burn money.

Source: [bj0fbejiq.txt](#), CHECK 1 part 2, sections 2.3 and 2.7 (fetched 2026-09-14), citing <https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows>; plan sections 5 (M2) and 9.

Private-repo minutes and prices

Public repos get standard runners free. Private repos get an included pool, then pay per minute.

Plan	Price	Included minutes per month	Artifact storage	Cache storage
Free (personal or organization)	\$0	2,000	500 MB	10 GB
Pro	\$4 per month	3,000	1 GB	10 GB
Team	\$4 per user per month	3,000	2 GB	10 GB

Per-minute rates on standard runners, after the price cut of January 1, 2026: Linux 2-core x64 (ubuntu-latest) is \$0.006, Linux 2-core arm64 is \$0.005, Windows 2-core is \$0.010, and macOS is \$0.062. A Linux 1-core rate of \$0.002 is listed, but the report could not confirm a usable label for it. Stay on Linux x64 and none of the Windows or macOS questions matter. Larger runners (4 cores and up) never draw on your included minutes; you pay from the first minute.

Source: [bj0fbejiq.txt](#), CHECK 1 part 2, section 2.1 and “Open items” (fetched 2026-09-14), citing <https://docs.github.com/en/billing/concepts/product-billing/github-actions>.

Cost math for this project, all on Linux x64: at 3,000 minutes a month, GitHub Free costs \$6 and GitHub Pro costs \$4. At 6,000 minutes, Free costs \$24 and Pro costs \$22. Pro pays for itself above about 2,667 minutes a month. The plan budgets \$0 to \$25 a month for CI.

Source: [bj0fbejiq.txt](#), section 2.6; plan section 6 (2026-09-14).

The organization trick and Blacksmith

An organization on GitHub is a shared account that owns repositories. A free organization gets the same 2,000 private minutes as a personal account. The plan puts the private repo inside an organization you own from day one. The reason is a vendor called Blacksmith.

Blacksmith sells drop-in replacement runners on faster bare-metal machines. You change one line, `runs-on: ubuntu-latest` becomes `runs-on: blacksmith-2vcpu-ubuntu-2404`, and nothing else. They claim about twice the speed. They give 3,000 free minutes a month per organization, then charge \$0.004 a minute for Ubuntu x64 and \$0.0025 for ARM. But their docs say, verbatim, “Blacksmith is limited to GitHub organizations and not available for personal repositories.” Moving a repo into an organization after you have wired up secrets is painful. Creating the organization first is free and takes two minutes.

When to switch: only once usage is consistently over about 3,000 minutes a month. Start on GitHub's own runners on the Free plan. Watch actual usage for three or four weeks in the billing page. If you

would rather have zero extra vendors, GitHub Pro at \$4 a month is a completely defensible choice, with a worst case of \$22 a month at 6,000 minutes.

Source: bj0fbeiqli.txt, sections 2.4, 2.6 and 2.7 (Blacksmith pricing and quickstart fetched 2026-09-14); plan sections 2 and 5 (M2).

One risk to keep in view: GitHub announced a \$0.002 per minute charge for self-hosted runners on 2025-12-16, then postponed it within two days. It is postponed, not cancelled. If revived, it would apply on top of every third-party runner.

Source: bj0fbeiqli.txt, section 2.2 (2026-09-14).

The dead-man's-switch heartbeat

A dead man's switch is a control on a train that stops the train if the driver lets go. The idea here is the same. A scheduled job can be dropped, disabled, or hung, and GitHub will not tell you. So each job will write a row to the `job_heartbeat` table in Neon when it succeeds, with the job name and the time. A separate, cheap check will read that table and raise an alarm if any nightly job's row is older than 36 hours. The rule in the report: never trust a cron you cannot observe.

Source: bj0fbeiqli.txt, sections 2.3 and 2.7; plan sections 10.3 and 10.6 (2026-09-14).

Learn it

All free. Total about 5 hours.

#	Resource	Format	Hours	Why
1	https://docs.github.com/en/actions/get-started/quickstart	Official docs	0.5	Your first workflow runs in ten minutes.
2	https://docs.github.com/en/actions/get-started/understand-github-actions	Official docs	1	The vocabulary: workflow, event, job, step, runner.
3	https://docs.github.com/en/actions/how-tos/write-workflows/	Official docs	2	Every keyword in the YAML above, with examples.
4	https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows	Official reference	0.5	The <code>schedule</code> section carries the off-hour and 60-day warnings in GitHub's own words.
5	https://github.com/skills/hello-github-actions	Interactive course inside a real repo	1	You write and run a workflow, graded by Actions itself.
6	https://github.com/skills/test-with-actions	Interactive course	1	Adds a test step and a status check, which is exactly the golden-test setup.

Source: bkm2kob5x.txt, learning path section 2 (all URLs verified 2026-09-14).

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

This assumes the repo from milestone M2 exists, with the golden test green in CI.

1. Create a branch: `git switch -c break-golden`.
2. Open the engine's reinvestment code and change the lag switch by one year. The plan names this exact change as the M2 acceptance check.
3. Commit and push, then open a pull request with `gh pr create`.
4. Watch the check run with `gh pr checks` or the Actions tab. It should go red within a few minutes, with the failing assertion showing the expected value 7.187840270062114.
5. Close the pull request without merging and delete the branch.

If the repo does not exist yet, do the smaller version: add `.github/workflows/hello.yml` with only a `workflow_dispatch` trigger and one step, `run: echo "hello from a runner"`. Push it, open the Actions tab, click Run workflow, and read the log.

Source: plan sections 5 (M2 “Done when” and M3 exercises) and 10.1 (2026-09-14); the `gh` commands are two of the six listed in `bkm2kob5x.txt`, section 1.

Done when

KEY IDEA

- You can read the YAML above and say what every keyword does without looking at the table.
- You have watched one pull request go red and one go green.
- A \$25 spending limit is set on the organization, and every job in the repo has `timeout=minutes`.
- Every schedule in the repo uses an off-hour minute.
- You can point at the `job_heartbeat` table and explain why it exists.

Chapter 4. TypeScript for a Python brain

OPTIONAL - FIRST PASS

For reading and editing the code. Use the Python comparisons and the eight traps as a reference when TypeScript appears in your work.

What it is (plain words and an analogy)

JavaScript is the language that web browsers run. Node.js is a program that runs JavaScript outside a browser, on a server or on your Mac. TypeScript is JavaScript plus a type system. You write small annotations such as `growth: number`. A compiler called `tsc` checks them, then strips them out and produces plain JavaScript. The types never reach the running program.

Analogy: TypeScript is Python with `mypy` switched on for every file and no way to turn it off. The type hints are removed before the program runs, exactly as Python ignores its own hints at runtime.

One more framing helps a Python brain. TypeScript types are structural. Two things with the same shape are the same type, whatever they are called. That is duck typing, the same idea as Python's Protocol. There is no official “TypeScript for Python programmers” page, so the closest useful starting point is the page written for JavaScript programmers.

Source: `bkm2kob5x.txt`, learning path section 3 (verified 2026-09-14).

Why this tool now (for this project)

The plan builds the whole system in one language: the calculator, the database layer, the librarian, the memo writer, the MCP server (the small program that lets an AI tool call the engine; chapter 10 explains it), and the nightly jobs. Python in the production path is an explicit non-goal. One language means one test runner, one build, one set of rules for Cursor to follow, and no glue between two worlds.

Source: plan sections 9 and 10.2 (2026-09-14).

There is also an arithmetic reason, and it matters more than the convenience. Excel stores every number as an IEEE-754 float64. So does JavaScript. IEEE-754 is the international standard for how computers store decimal fractions in binary, and float64 is its 64-bit size. Because both sides use the same format, a port that performs the spreadsheet's operations in the same order agrees with it far more tightly than the golden test requires. The golden test only asks for 1e-6. That is why a program written in a language with "no Decimal type" can still match Damodaran's workbook to six decimal places.

Source: plan sections 10.1 and 10.2 (2026-09-14); bkm2kob5x.txt, section 3.

How it is used in this project

The repo is a pnpm workspace, which is one folder holding several packages that share tools. A package is a folder with its own `package.json`, the file that names a package and lists what it depends on. The plan's packages are `engine`, `db`, `rag`, `memo`, `mcp-server`, and `models`, all under `packages/`.

The engine is pure functions. A pure function takes inputs, returns an output, and touches nothing else: no network, no database, no clock. Modules mirror the spreadsheet's sheets: `rating.ts`, `costOfCapital.ts`, `fcff.ts`, `terminal.ts`, `failure.ts`, `india.ts`, `reverseDcf.ts`, `monteCarlo.ts`, `impliedErp.ts`. Everything else, such as reading a filing or writing a row, sits outside the engine so the arithmetic stays testable.

Source: plan section 10.2 and "Critical files" (2026-09-14).

The eight things that bite

Each item names the Python habit, the TypeScript reality, and what it does to a valuation if you get it wrong.

1. There are two kinds of nothing: `null` and `undefined`

Python has one `None`. JavaScript has `undefined` (never set) and `null` (set to nothing on purpose). Turn on `strictNullChecks`, which `strict` includes, and the compiler forces you to handle both.

The trap is the default operator. Python's `x or default` has a JavaScript twin, `x || default`, and it fires on `0` and on the empty string as well as on the two nothings. Use `??` instead, which fires only on `null` and `undefined`.

```
const g = input.growth || 0.05;    // a real 0% growth silently becomes 5%
const g = input.growth ?? 0.05;    // 0 stays 0; only a missing value gets the
↪ default
```

Source: bj0fbeiqr.txt, CHECK 3 section 8, item 1 (2026-09-14).

2. Always ===, never ==

== converts types before comparing. '' == 0 is true and '1' == 1 is true. === compares without conversion, the way Python's == behaves. Use === everywhere. The one accepted exception is x == null, which catches both nothings in one test.

Source: bj0fbejiq.txt, CHECK 3 section 8, item 2.

3. number is float64, and there is no int and no Decimal

Python has int, float, and Decimal. JavaScript has number, which is float64, and that is it for ordinary arithmetic. 0.1 + 0.2 !== 0.3 holds, exactly as it does for Python floats. For the engine this is fine and even desirable, because the spreadsheet is float64 too, as explained above.

It is not fine for money totals or share counts that you add up over many rows. For those, keep integers in minor units, or use the decimal.js library. And note a related surprise: the node-postgres driver returns Postgres NUMERIC and BIGINT columns as strings, on purpose, so that precision is not lost. Do not “fix” that with Number() until you have decided where rounding is allowed.

Source: bj0fbejiq.txt, CHECK 3 section 8, item 3; plan section 10.2 (2026-09-14).

4. async and await are everywhere, and a missing await is silent

Almost every input or output call returns a Promise. A Promise is an object that says “the value will arrive later.” Python has asyncio, but you can avoid it. In Node you cannot. Forgetting await does not raise an error. It hands you the Promise object, which looks fine and turns into NaN three functions later. Enable the lint rule @typescript-eslint/no-floating-promises so ESLint catches it (ESLint is the linter, a second checker that runs alongside tsc). Use Promise.all to run fetches in parallel. Node has no GIL, but it also has one thread; it is an event loop, not threads.

Source: bj0fbejiq.txt, CHECK 3 section 8, item 4.

5. Modules: ESM and the .js extension rule

JavaScript has two module systems. CommonJS is the old one with require. ESM, for ECMAScript Modules, is the standard one with import and export. Set "type": "module" in package.json and you are in ESM. Then three things change: no require, no __dirname (use import.meta.url), and under moduleResolution: "NodeNext" every relative import must end in .js even though the file on disk is .ts.

```
import { terminalCostOfCapital } from "./terminal.js"; // the file is
↳ terminal.ts; this is correct
import { terminalCostOfCapital } from "./terminal";    // fails at build or
↳ run time
```

The report calls this the cause of most first-week build failures. Tape it to the monitor.

Source: bj0fbejiq.txt, CHECK 3 section 8, item 5.

6. strict mode, and the fact that types vanish at runtime

Because tsc erases types, a typed API response is a promise, not a check. Nothing stops a JSON file from putting a string where the type says number. Two settings and one library close the gap. In tsconfig.json, which is the compiler's settings file, keep strict: true (on by default in TypeScript 6.0) and add noUncheckedIndexedAccess. Then validate every external input at the boundary with Zod. Zod is a library that checks a value against a schema at runtime and throws if it does not

match. It is the TypeScript twin of pydantic. External inputs here are filing JSON from EDGAR (the US regulator's public filing database), text produced by OCR (software that reads scanned pages), vendor data, and every model output. The engine only ever sees values that passed Zod.

Source: bj0fbejiq.txt, CHECK 3 section 8, item 6.

7. pnpm workspaces instead of virtual environments

Python gives each project a virtual environment with its own interpreter. Node gives each project a node_modules folder. pnpm is a package manager that is strict about it. A workspace is a pnpm-workspace.yaml file listing package folders. One package depends on another by writing "@valuation/engine": "workspace:*". pnpm links packages with symlinks and refuses an import you did not declare. That is good, but it surprises anyone used to Python's flat site-packages. Run one package's tests with pnpm -F engine test.

Source: bj0fbejiq.txt, CHECK 3 section 8, item 7; <https://pnpm.io/workspaces>.

8. Vitest is pytest

Vitest is the test runner. describe groups tests, it is one test, expect is the assertion. vi.mock() is monkeypatching. --coverage is built in. It runs TypeScript directly, so there is no build step before tests.

```
import { describe, it, expect } from "vitest";
import { terminalCostOfCapital } from "../terminal.js";

describe("terminal cost of capital", () => {
  it("is the risk-free rate plus the mature-market ERP", () => {
    expect(terminalCostOfCapital(0.0458, 0.0423)).toBeCloseTo(0.0881, 6);
  });
});
```

Use toBeCloseTo, not toBe, for any float. The two inputs are the workbook's own January 2026 values, and 8.81% is the workbook's own answer.

Source: bj0fbejiq.txt, CHECK 3 section 8, item 8; bkdrkpfnq.txt (model schema report), the terminal cost of capital row: 4.58% + 4.23% = 8.81%.

The translation table

Python	TypeScript	Watch for
None	null and undefined	Use ?? for defaults, not \ \ .
==	===	Never == except x == null.
int, float, Decimal	number (float64), bigint, decimal.js	Fine for the engine; not for money totals.
asyncio	Built in; every I/O call returns a Promise	A missing await is silent.
import x from pkg. mod	import { x } from "../mod.js"	The .js is required, even for a .ts file.
mypy	tsc with strict: true	Types are erased at runtime.
pydantic	Zod	Validate at the boundary.
venv and pip	pnpm and node_modules	Undeclared imports fail loudly.

Python	TypeScript	Watch for
pytest	Vitest	describe, it, expect, vi.mock.

Source: the mapping is a summary of bj0fbeiqr.txt, CHECK 3 section 8 (2026-09-14).

Learn it

Skip everything aimed at people who have never programmed. You have the concepts; you need the type system and the module story. All free. Total about 18 hours if you read the Handbook and the Essentials chapters listed, plus about 5 hours for the tools.

#	Resource	Format	Hours	Why
1	https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html	Official page	0.5	Written for JavaScript programmers; the closest thing to a Python track.
2	https://www.typescriptlang.org/docs/handbook/2/basic-types.html , then in order: Everyday Types, Narrowing, More on Functions, Object Types, Generics	Official Handbook	6 to 8	Narrowing is the chapter that makes TypeScript click. Read it twice.
3	https://www.totallytypescript.com/books/total-typescript-essentials	Free book with exercises, 16 chapters	10 to 12 for the whole book	Do chapters 1 (tsconfig), 5 (unions, literals, narrowing), 10 (deriving types), 13 (modules and declaration files), 14 (configuring TypeScript) first. Chapters 13 and 14 alone repay the time because of the <code>.js</code> rule.
4	https://www.typescriptlang.org/tsconfig/ and https://github.com/tsconfig/bases	Official reference	1	Every flag explained; bases gives a correct Node config to copy instead of inventing one.
5	https://vitest.dev/guide/	Official docs	2	Getting started, mocking, coverage, with direct pytest analogies.
6	https://zod.dev/	Official docs	1.5	Runtime validation; the boundary layer that makes model output safe to store.

Not recommended for now: Execute Program's TypeScript course. It is live in 2026 and good, but it is paid and its spaced-repetition shape is wrong for someone shipping an engine in twelve weeks.

Source: bkm2kob5x.txt, learning path section 3 (URLs verified 2026-09-14); bj0fbeiqr.txt, CHECK 3 section 8 resource table.

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

Inside the repo, or in a scratch folder with `pnpm init` and `pnpm add -D typescript vitest`:

1. Create `terminal.ts` containing one exported pure function, `terminalCostOfCapital(rf: number, matureErp: number): number`, that returns `rf + matureErp`. Terminal cost of capital is the discount rate the model uses for the years after the explicit forecast, and the workbook sets it to the risk-free rate plus the mature-market equity risk premium.
2. Create `terminal.test.ts` with the Vitest test shown in item 8 above.
3. Run `pnpm vitest run`. It should pass.
4. Change `toBeCloseTo(0.0881, 6)` to `toBe(0.0881)`. Run again. If it fails, you have just seen float64 in person. If it passes, change the inputs to `0.1` and `0.2` with an expected `0.3` and try again.
5. Remove the `.js` from the import in the test file. Run again. Read the error message slowly. That is the message you will see in week one.
6. Put the `.js` back and leave both files green.

Source: `bkdrrpfng.txt`, the stable cost of capital row (`rf 4.58% + mature ERP 4.23% = 8.81%`); plan section 10.2; `bj0fbeiqr.txt`, CHECK 3 section 8 items 3, 5 and 8.

Done when

KEY IDEA

- You can explain, without notes, why `||` is dangerous for a growth rate and `??` is not.
- You can say in one sentence why a float64 language can still match the spreadsheet to `1e-6`.
- You have seen the missing `.js` error once and fixed it.
- You can name the Zod schema that guards each external input the engine consumes.
- You have run `pnpm -F engine test` and read the output.

Chapter 5. Postgres and Neon: the one place the numbers live

OPTIONAL - FIRST PASS

For the database stage. The central idea is that the numbers and their labels live in the database. Save the SQL and connection details for that work.

What it is (plain words and an analogy)

A database is a program whose only job is to store facts and answer questions about them. Postgres (its full name is PostgreSQL) is a free, open-source database that has existed for decades. You talk to it in SQL, a small language of questions such as "give me every row where the country is India". Postgres is not a spreadsheet, but the mental picture of a spreadsheet is a good place to start.

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 3 section 7 (2026-09)

Here are the five words you will meet on the first day.

- A **table** is one spreadsheet tab with a fixed shape. Every row has the same columns, and the database enforces that.
- A **row** is one record. In this project, a row is one fact: one number, for one company, from one source, on one date.
- A **column** is one named slot with one data type. A numeric column holds numbers and refuses the word “unknown”.
- **NOT NULL** is a rule on a column that says “this slot can never be empty”. The database refuses the write, not the program.
- A **CHECK constraint** is a rule the database tests on every insert. `CHECK (tax_rate >= 0 AND tax_rate <= 1)` refuses a 340% tax rate at the door.

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 3 section 7 (2026-09)

The analogy the research report uses is worth keeping. Constraints are the guard rails on a mountain road. They are annoying while you drive, and they are the reason you are alive. A spreadsheet lets you type anything into any cell. A database with constraints is a spreadsheet that argues back.

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 3 section 7 (2026-09)

Three more phrases matter for this project.

- **System of record** means that for any fact there is exactly one place that holds the truth. Everything else is a copy that can be thrown away and rebuilt. Here, the Postgres rows are the truth. The search index, the model’s context window and every Markdown memo are copies.
- A **migration** is a versioned script that changes the shape of the database (the set of tables, columns and rules is called the schema): add a table, add a column, add a CHECK. Migrations are committed to git and applied in order, so every copy of the database has the same shape. Think of a migration as a commit for the database’s structure, not its contents.
- A **Neon branch** is a full copy of your database created in seconds. Neon does not duplicate the storage; it only records what changed, which is called copy-on-write. It behaves like a git branch: try a migration on a branch, and drop the branch if it goes wrong.

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 3 section 7 (2026-09)

Neon is a company that runs Postgres for you in the cloud. You install nothing. You get a connection string, a long URL with a host name, a user and a password inside it. Neon’s trick is that the database switches itself off after about five minutes of silence and costs nothing while asleep.

Source: Valuation/docs/sources/bo0hrclp7.txt, section 2.1 (2026-09)

Why this tool now (for this project)

The whole system rests on one rule from the research: retrieval finds the paragraph, the database holds the number. An AI model will eventually try to write a number that is wrong, badly scaled, or from the wrong year. If that number goes straight from a retrieved paragraph into a valuation, you find out weeks later. If it must first pass a NOT NULL and a CHECK, you find out at once, with an error message.

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 3 sections 6 and 7 (2026-09)

Why Neon rather than another host? The vendor comparison scored six options. Neon won because a nightly batch job that runs for forty minutes and then sleeps costs cents, and because branches make experiments safe. The same report is honest about the weak spot: Neon acknowledged outages on 2026-05-08, 2026-07-11 and 2026-08-10. That was accepted because a failed nightly run simply retries; nothing here serves live users.

Option	Why it lost
Supabase	Pro is \$25 a month flat with no true scale-to-zero; its branching is heavier, a branch is a whole preview environment
PlanetScale Postgres	No free tier; branches need a restore from backup, so they are not instant
Turso	It is SQLite, not Postgres; the wrong family for numeric work
Railway Postgres	A container you look after yourself, with no branching
Xata	Free tier retired; a 27-person company is a thin longevity bet

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 2 Role 1 (2026-09)

Two earlier chapters already lean on this one. Chapter 3 (GitHub Actions) showed the nightly robots writing the `job_heartbeat` row to Neon. Chapter 4 (TypeScript) warned that `node-postgres` returns NUMERIC columns as strings. Both make more sense once you can picture the tables.

How it is used in this project

The tables, in one glance

The plan lists about twenty tables. Do not memorize them. Notice the pattern: some hold Damodaran's data by vintage, some hold company facts, and some record what the system did.

Group	Tables	What a row is
Damodaran's data	vintage, country_risk, industry_stat, industry_dist, erp_monthly, erp_annual	One published number, stamped with the file it came from
Company facts	company, fact, ltm_financials, realised_price	One reported figure with its source URL, license class and vintage id
The system's own work	valuation_run, valuation_inputs, memo, claim, narrative_note, market_context_event, job_heartbeat	One run, one input, one memo, one claim, one note, one nightly success
The librarian's bookkeeping	chunk_manifest, rag_eval_case, rag_eval_run	One indexed chunk, one test question, one test result

In fact, the columns `source_url`, `license_class`, `filed`, `accession` and `vintage_id` are all NOT NULL. In `valuation_run`, the three vintage ids are NOT NULL. Raw files are not stored in Postgres; they sit in a private bucket, named by their hash.

Source: Valuation/docs/plan.md, section 10.3 (2026-09-14)

Vintage: why every run stores three ids

Damodaran refreshes most tables every January, the country risk premiums quarterly, and the implied equity risk premium monthly. So one calendar year holds several versions of the same number. The 2026 files alone carry three risk-free rates and three mature-market risk premiums. A risk-free rate is the yield on a government bond with no default risk. A mature-market equity risk premium (ERP) is the extra return investors demand for owning stocks in a stable market.

Where it sits	Risk-free rate	ERP
wacc.xls header block, January 2026	3.95%	4.46%
fcffsimploginzu, January 2026 vintage (the Almarai example)	4.58%	4.23%
ERPSept26.xlsx for the bond; ctrypremJuly26 for the premium	4.75%	4.20%

Source: Valuation/docs/sources/bafb6h9rb.txt, lines 34, 84 and 88; Valuation/docs/sources/bkdrkpfng.txt, line 23 (2026-09)

Mix a January industry table with a September risk premium and you get a confident answer with no warning. So every valuation_run row stores three ids: which market file, which country-risk file, which industry file. Each is NOT NULL, so the database refuses a run that is missing one. That is what “vintage” means in Document A, section 3.2. The workbook itself hard-codes 4.58% inside two formulas; the plan turns that literal into a stored vintage value.

Source: Valuation/docs/read-this-first.md, section 3.2 (2026-09-14); Valuation/docs/sources/bkdrkpfng.txt, line 446 (2026-09)

License class: why it sits on every row

Rights attach to facts, not to files. Document A, section 3.3, lists the seven classes. Note the spelling: Document A writes “licence”, and the column is named `license_class`; they are the same thing. The classes are `public_domain`, `damodaran_public`, `open_access`, `asr_youtube`, `licensed_personal`, `proprietary_personal` and `link_only`. A CHECK on the column allows only those seven strings. A second rule, on `chunk_manifest`, refuses the last three from the search index.

Source: Valuation/docs/read-this-first.md, section 3.3 (2026-09-14); Valuation/docs/plan.md, sections 10.1 and 10.3 (2026-09-14)

The Bloomberg example shows why. Your bloomberg.com subscription is a reading product. Its terms say the service “may not be used to construct a database of any kind” (Bloomberg.com Terms of Service, section 3). So when you read an article, the system stores only the link, the headline, the date and your own one-sentence note, in `narrative_note`, with `license_class = 'proprietary_personal'`. Books and paid newsletters get `link_only`, meaning the row holds a link and nothing else. Both classes are refused by the index rule, so they never reach the librarian. The research report’s analogy: allergen labels belong on every ingredient, not on the finished cake. One unlabeled Bloomberg number in fact would make the whole table unshareable, because you could no longer prove which rows were clean.

Source: Valuation/docs/sources/bo0hrclp7.txt, Bloomberg section (2026-09);
Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3 section 7 (2026-09)

The payoff arrives later. When you ask “can I show this to someone?”, the answer becomes a query: `WHERE license_class IN ('public_domain', 'damodaran_public', 'open_access')`. No audit, no guessing.

Source: [Valuation/docs/read-this-first.md](#), section 3.3 (2026-09-14)

Migrations and branches in the daily loop

The schema will be written in Drizzle, a TypeScript library where the table definitions are TypeScript and the migrations are plain SQL you can read. The learning path picked it over Prisma for exactly that reason: when a `vintage_id` join duplicates rows and the ERP comes out wrong, you want to see the SQL.

Source: [Valuation/docs/sources/bkm2kob5x.txt](#), learning path section 6 (2026-09)

The loop will work like this. You open a pull request. A GitHub Action creates a Neon branch, runs the migrations and tests against it, and posts the schema difference as a comment. When the pull request closes, the branch is deleted. Neon’s GitHub App sets a `NEON_API_KEY` secret and a `NEON_PROJECT_ID` variable in the repo to make that possible. The free plan allows 10 branches per project; on the paid plan each extra branch costs \$1.50 per branch-month, so branches are always deleted on close.

Source: [Valuation/docs/sources/bo0hrclp7.txt](#), sections 2.1 to 2.3 (2026-09)

One habit from the research to adopt on day one: create a branch before any schema change an AI tool proposes. Neon branching is your undo button.

Source: [Valuation/docs/sources/bj0fbeiqli.txt](#), CHECK 2 accounts checklist (2026-09)

The two connection URLs, and the SNI gotcha

Neon gives you two connection strings for the same database, and Document A asks you to save both.

Name in <code>.env</code>	Host looks like	Who uses it	Why
<code>DATABASE_URL</code> (pooled)	<code>ep-xxx-pooler.<region>.aws.neon.tech</code>	Web functions, Hex, any dashboard	A pooler (PgBouncer) shares a few real connections among many short visits
<code>DATABASE_URL_UNPOOLED</code> (direct)	<code>ep-xxx.<region>.aws.neon.tech</code>	Migrations and long scripts	Migrations need features the pooler does not pass through

A pooler is a doorman. A web function opens and closes a connection on every request, and bare Postgres would run out of doors. The pooler keeps a few doors open and lets visitors share them. The pooled host needs `sslmode=require`, not `verify=full`, and accepts only a short list of startup settings. Migrations use the direct endpoint.

Source: [Valuation/docs/read-this-first.md](#), section 10, rows 4 and 10 (2026-09-14);

[Valuation/docs/sources/bo0hrclp7.txt](#), section 2.4; [btzaixzyh.txt](#), “Neon connection gotchas” (2026-09)

Now the gotcha. SNI stands for Server Name Indication. It is a field at the start of an encrypted connection where the client names the host it wants. The Postgres protocol carries no host name, so Neon relies on SNI to route your connection to the right database. Modern drivers send it; the reference library `libpq` has since version 14, released in September 2021. An old or unusual tool

may not, and the connection fails with a confusing error. The fix is to append `?options=endpoint%3D<endpoint-id>` to the URL, or to prefix the password with `endpoint=<endpoint-id>;`. If a dashboard tool refuses to connect, check this first.

Source: Valuation/docs/sources/btzaixzyh.txt, “Neon connection gotchas”, citing <https://neon.com/docs/connect/connection-errors> (2026-09)

What it costs

Plan	Included	Price
Free	0.5 GB storage per project, 100 compute-hours a month, 10 branches per project, sleeps after 5 minutes	\$0
Launch	Pay as you go, no monthly minimum since December 2025	\$0.106 per compute-hour, \$0.35 per GB-month, \$1.50 per extra branch-month

A compute-hour (CU-hour) is one hour of Neon’s smallest unit of processor and memory, so 100 CU-hours is roughly 400 hours at the quarter-unit size. The binding limit for this project is the 0.5 GB of storage, not compute. Document A budgets Neon at \$0 now and \$10 to \$20 later, inside the \$200 ceiling.

Source: Valuation/docs/sources/bo0hrclp7.txt, sections 2.1 and 2.2 (2026-09); Valuation/docs/read-this-first.md, section 12 (2026-09-14)

Learn it

All free. Hours are the estimates in the research reports.

#	Resource	Format	Hours	Why
1	https://www.postgresql.org/docs/current/tutorial.html	Official tutorial	3	Tables, queries and joins from zero; short and authoritative
2	https://www.postgresql.org/docs/current/ddl-constraints.html	Official docs	1	NOT NULL, CHECK and foreign keys; the chapter that protects your valuations
3	https://neon.com/docs/introduction/branching	Official docs	1	What a branch is and how to make one per pull request
4	https://neon.com/docs/get-started/signing-up	Official guide	0.5	Account, first project, where the SQL editor lives (open in M1, not today)
5	https://orm.drizzle.team/docs/tutorials/drizzle-with-neon	Official tutorial	2	Define a table in TypeScript, generate SQL, run it against Neon
6	https://neon.com/docs/guides/drizzle-migrations	Official guide	1.5	The one that matters most: versioned schema changes on Neon

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 3 section 7 table; bkm2kob5x.txt, learning path section 6 (2026-09)

Read 1 and 2 before the exercise. Do 3 to 6 in the week Document A's schedule opens the Neon account.

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

This needs the Neon account from Document A, section 10, item 4, opened in M1. Until then, read the SQL and predict each result. With the account, open your project in the Neon Console, create a branch called `scratch` so nothing touches `main`, then open the SQL Editor (the SQL Editor is in the left sidebar of the Neon Console) and run the blocks one at a time.

Block 1 builds a tiny fact table with the two rules from this chapter.

```
CREATE TABLE practice_fact (
  id          serial PRIMARY KEY,
  company     text    NOT NULL,
  field       text    NOT NULL,
  value       numeric NOT NULL CHECK (value >= 0),
  vintage_id  text    NOT NULL,
  license_class text  NOT NULL
  CHECK (license_class IN ('public_domain', 'damodaran_public',
    ↪ 'open_access',
                                'asr_youtube', 'licensed_personal',
                                'proprietary_personal', 'link_only'))
);
```

Block 2 inserts a good row: the Almarai example's risk-free rate, stamped with its vintage.

```
INSERT INTO practice_fact (company, field, value, vintage_id, license_class)
VALUES ('Almarai', 'riskfree', 0.0458, 'ginzu-2026-01', 'damodaran_public');
```

Block 3 leaves out the vintage. Predict what happens, then run it.

```
INSERT INTO practice_fact (company, field, value, license_class)
VALUES ('Almarai', 'mature_erp', 0.0423, 'damodaran_public');
```

You should see an error like `null value in column "vintage_id" ... violates not-null constraint`. The wording may differ by version. Nothing was written. That refusal is the whole point of Document A, section 3.2.

Block 4 tries to smuggle in a number with a made-up license class.

```
INSERT INTO practice_fact (company, field, value, vintage_id, license_class)
VALUES ('Almarai', 'tax_rate', 0.2517, 'ginzu-2026-01', 'bloomberg');
```

You should see `violates check constraint "practice_fact_license_class_check"`. The database does not know or care that the string means Bloomberg; it only knows the string is not one of the seven.

Block 5 checks what survived, then cleans up.

```
SELECT * FROM practice_fact;
DROP TABLE practice_fact;
```

Exactly one row should come back. Delete the `scratch` branch afterward so it does not count toward the ten.

Source for the numbers: `Valuation/docs/sources/bkdrkpfng.txt`, lines 23 and 90, and section 6 for the 25.17% India new-regime tax rate (2026-09)

Done when

KEY IDEA

- You can explain table, row, column, NOT NULL and CHECK to a friend in one minute, and say why the Postgres rows are the **system of record** and the search index is not.
- You ran the exercise and saw two refusals and one surviving row, or you predicted all three outcomes correctly while reading.
- You can name the three 2026 risk-free rates, say which file each comes from, and explain why a run must store three vintage ids.
- You can state which two connection strings go in `.env`, which one the migrations use, and what SNI has to do with a failed connection.
- You know that a Bloomberg reading note is `proprietary_personal`, that a book is `link_only`, and that neither ever reaches the index.

Chapter 6. Pinecone, Voyage and RAG in plain words

OPTIONAL - FIRST PASS

For the librarian stage. Understand the retrieval analogy first; return to embeddings, reranking, and configuration when checking the librarian.

What it is (plain words and an analogy)

Picture a librarian who has read every paragraph Damodaran ever wrote or said. She does not remember page numbers. She remembers what each paragraph is about. You ask, “why does he subtract a default spread from the Indian bond yield?” She walks to the eight paragraphs that answer it, even though none contains your exact words. That librarian is what we are building. The technical name is RAG, retrieval-augmented generation. “Retrieval” is the librarian finding the paragraphs. “Generation” is a language model writing an answer from them, with a footnote to each one.

Source: `Valuation/docs/sources/bj0fbejiq.txt`, CHECK 3 section 6 (2026-09-14)

Here are the words you will meet, each with the librarian’s version.

Term	Plain meaning	The librarian’s version
Token	A word piece, roughly four characters.	The unit she counts in.
Chunk (passage)	One paragraph-sized slice of a document, 800 to 1,200 tokens here.	One card in her memory.
Embedding (vector)	A list of numbers that places a chunk where similar meanings sit close together.	Where the card sits on a shelf sorted by meaning, not alphabet.

Term	Plain meaning	The librarian's version
Dimensions	How many numbers are in that list. Ours have 1,024.	How many directions the shelf runs in.
Cosine similarity	How closely two vectors point the same way. 1.0 is same meaning, 0 is unrelated.	How near two cards sit.
Vector database (index)	A service that stores the vectors and returns the nearest ones to a question, fast.	Her shelf, plus her speed.
Dense search	Search by meaning, using the vectors.	"Cards about this idea."
Sparse search (BM25)	Search by exact words: does the phrase appear?	"Cards containing the word Almarai."
Hybrid search	Dense and sparse run together and the lists are merged.	Memory and card catalog, both.
Reranker	A slower model that reads the question and each candidate together and re-scores them.	Fifty cards on the desk, the best eight kept.
Citation	Document id, page and position stored with every chunk.	The stamp on the back of every card.
Context window	The most tokens a model can read in one call.	How much fits on the desk.

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 3 section 6; Valuation/docs/plan.md, section 10.4 (2026-09-14)

Why a second pass? Nearest in meaning is not always most useful. So the system over-fetches, say the top 50, and lets the reranker read each one properly. Keeping 5 to 8 is the 2026 default. Why hybrid? Embeddings are fuzzy about exact names. In our corpus, 101 files say "sales to capital", 97 say "sales/capital" and 4 say "sales-to-capital". Meaning-based search treats all three as one idea, which is good. But a ticker, a rating like "Baa3" or a file name like "fcffsimpleginzu" must match letter for letter. Keyword search does that.

Source: grep over financeMD/ (2026-09-14); Valuation/docs/sources/bcy86uopa.txt, section 5d

Why this tool now (for this project)

The corpus is too big to read in one go. The markdown mirror of Damodaran's site, blog and packets is about 21.0 million tokens across 3,916 files. His 1,477 YouTube transcripts add about 18.0 million. After the recommended drops the total is about 29.7 million tokens. The largest context window for sale today is 2,000,000 tokens (Grok 4.20 on OpenRouter). The corpus is fifteen times larger than that window. No model can read it in a single call, at any price.

Source: Valuation/docs/sources/bfxevcj1s.txt, section 1; bcy86uopa.txt, section 6 (2026-09-14)

Even where a window is big enough, the price is not. One question, answered three ways:

Strategy	Input tokens	On GLM-5.3-Flash	On GLM-5.3
RAG: 8 chunks of 800 tokens, a 1,500-token system prompt, a 100-token question	8,000	\$0.00155	\$0.0143
Stuff one 200,000-token lecture packet	200,000	\$0.0304	\$0.283
Stuff the whole corpus, 15 calls plus a synthesis	30,000,000	\$4.50	not quoted

Stuffing a packet costs 19.6 times a RAG answer on the same model. Stuffing the corpus costs about 2,900 times. The same question on Claude Fable 5.1 would cost \$300. Building the whole index costs \$0 on Voyage’s free allowance. So RAG pays for itself on the first question.

Source: Valuation/docs/sources/bcy86uopa.txt, section 6 (2026-09-14)

Three more reasons are about correctness. RAG returns chunk ids you can print as footnotes, and every claim here must be cited. Chunks carry a vintage label, so the system filters to one dated snapshot before the model sees anything. That is how you avoid mixing the 3.95%, 4.58% and 4.75% risk-free rates from the three 2026 files. And a question costs the same whether the index holds 30 million or 300 million tokens, because it only fetches eight chunks. That is the honest answer to “does it scale”.

Source: Valuation/docs/sources/bcy86uopa.txt, section 6; Valuation/docs/read-this-first.md, sections 3.2 and 5 (2026-09-14)

One practical fact decides the vendor list. OpenRouter, which serves every language model in this system, offers zero embedding models and zero rerankers, verified across all 445 models in its catalog. So embeddings and reranking need a second vendor. That vendor is Voyage, and the vectors live in Pinecone.

Source: Valuation/docs/sources/bcy86uopa.txt, section 4 and 5 (2026-09-14)

How it is used in this project

Nothing below is built yet. Milestone 2 will build it, and this section describes what it will do. The librarian is one box on the Architecture board (<https://www.figma.com/board/DtFTvDZeSQpuKLM5tavDUT>) and the “retrieve passages” step on the Memo pipeline board (<https://www.figma.com/board/LGcrXp6yT3fh4O2IV3mEY8>).

The corpus, in tiers

The corpus goes in by tiers, best answers per token first. Tier 1 is the blog posts. Tier 2 is the book, the lecture packets with speaker notes, and the exams. Tier 3 is the dataset rows rewritten as sentences. Tier 4 is the transcripts, after a punctuation pass. Milestone 2 will index tiers 1 and 3 first and prove the pipeline on those. Transcripts come last because YouTube’s machine-made captions have no punctuation, are all lowercase, and carry 10 to 20 percent classroom chatter. Cleaning 18 million transcript tokens with GLM-5.3-Flash in batch mode, a slower queue at half price, is priced at about \$11.25.

Source: Valuation/docs/plan.md, sections 5 (M2) and 10.4; bfxevcj1s.txt, section 2(f); bcy86uopa.txt, section 7 row (b) (2026-09-14)

Cleaning before indexing

The corpus survey found rubbish that would poison retrieval if indexed as is. The cleaning steps are:

- Drop the 194 blog index.md archive pages. They reprint full posts and are exactly half of the blog bucket by bytes.
- Drop damodaran/pc/archives/, 1,647 files of historical numeric tables. They belong in the database, not the index.
- Strip in place: Blogger <div> wrappers, dead image links, the “An error occurred” YouTube embed blocks, the navigation header, and **Damodaran<N>Aswath** header artifacts in packets.

- Repair the book’s broken ligatures before indexing it. A ligature is a joined letter pair such as “fi”, which the converter turned into the replacement symbol U+FFFD. Also skip its first 682 lines of contents and lists.
- In transcripts, remove [Music] markers and bare “yes / right / okay” filler lines.

Source: Valuation/docs/sources/bfxevcj1s.txt, sections 4 and 5 (2026-09-14)

Chunks and their labels

Chunking follows the shape of each source. A blog post is split on its sub-headings, never across posts, with the title and date prepended to every chunk. A slide deck keeps one slide and its speaker notes together, because the `### Notes:` block under a slide is where he explains why. Exams join a question with its solution. Transcripts become 60 to 90 seconds of speech, about 800 tokens, with 20 percent overlap. Every chunk carries metadata: `source_url`, `title`, `date`, `doc_type`, `license_class`, `vintage`, `asr`, `region` and `company`. Two labels do the most work. `author` is Damodaran only for his own words, so a filing or press clipping can never be quoted as his view. `year` lets the reranker prefer a 2026 packet over a 2025 one.

Source: Valuation/docs/sources/bfxevcj1s.txt, section 6; Valuation/docs/plan.md, section 10.4; financeMD/damodaran/pptfiles/val3E/valpacket2spr25.md (2026-09-14)

The license fence

Every chunk has a license class, and the index admits four of the seven. `public_domain` (SEC filings), `damodaran_public` (his posts and datasets), `open_access` (RBI yields, arXiv, SSRN author copies) and `asr_youtube` (auto-captions, marked as approximate wording) go in. `licensed_personal` (EODHD, a broker’s API), `proprietary_personal` (your Bloomberg notes) and `link_only` (books, paid newsletters) stay out. The fence is a database rule on the `chunk_manifest` table in Neon, a CHECK constraint that refuses a row with a forbidden class. It is not a habit you must remember. Think of an allergen label on every ingredient rather than on the finished cake. Because the label is on each chunk, “can I show this answer to someone else?” becomes a query, not an audit.

Source: Valuation/docs/read-this-first.md, section 3.3; Valuation/docs/plan.md, sections 10.1 and 10.3; bj0fbelijq.txt, CHECK 3 section 7 (2026-09-14)

Pinecone’s role, Voyage’s role, and what is free

Voyage turns text into vectors and re-scores candidates. The embedding model is `voyage-context-4`. It produces 1,024-number vectors, accepts up to 32,000 tokens of input, and embeds each chunk aware of its neighbors. That neighbor awareness is what rescues a transcript chunk that makes no sense on its own. The reranker is `rerank-3`. Voyage gives 200 million free tokens per account across its v4 models. Embedding our 30 million tokens would cost $30 \times \$0.12$, which is \$3.60 at list price, and \$0 inside the free allowance. Reranking 50 candidates of 800 tokens is 40,000 tokens per question, about \$0.002 at list price, and free for roughly the first 5,000 questions.

Source: Valuation/docs/sources/bcy86uopa.txt, sections 5b, 5c and 5d (2026-09-14)

Pinecone stores the vectors and answers “which are the nearest?” It is a serverless index, meaning you never run or size a machine. Document A specifies one index, 1,024 dimensions, cosine similarity, region `us-east-1`. The Starter plan costs \$0, needs no card, and includes 2 GB of storage, 2 million write units and 1 million read units a month. Our 60,000 chunks at 1,024 dimensions are about 245 MB of raw vectors, perhaps 400 to 600 MB with metadata, so they fit. Loading them uses about 3 percent of a month’s free writes. A sparse companion model, `pinecone-sparse-english-v0`,

provides the keyword half of hybrid search. If storage or reads ever overflow, the upgrade is Builder at \$20 a month.

Source: Valuation/docs/read-this-first.md, sections 10 and 12; bcy86uopa.txt, sections 5a, 5c and 5d (2026-09-14)

Answering a question, step by step

1. Classify the question. If it is about one document (“summarize this packet”), fetch the whole document and give it to the model in one call. GLM-5.3-Flash’s 1.31-million-token window holds a packet, and chopping one argument into chunks would lose it.
2. If it spans the corpus (“when did he change the India ERP?”), embed the question with Voyage.
3. Hybrid retrieve the top 50 chunks from Pinecone, filtered by license class, and by vintage or region when the question needs it.
4. Rerank the 50 down to 8 with rerank-3.
5. Send the 8 chunks, a system prompt and the question to GLM-5.3 through OpenRouter. The answer must cite chunk ids. An uncited claim is rejected.
6. Log the call in Langfuse, the tracing and evals tool covered in the evals chapter. About 40 of his exam questions, with solutions, are held out as the test paper, and Kimi K3 grades the answers as judge.

Source: Valuation/docs/sources/bcy86uopa.txt, section 6; bfxcvcj1s.txt, section 5 item 10; Valuation/docs/plan.md, sections 10.4 and 10.5 (2026-09-14)

The rule: retrieval finds the paragraph; the database holds the number

This is the one rule to carry out of the chapter. RAG is for narrative and reasoning: why he treats country risk the way he does, what “sales to capital” means, how a story becomes a driver. The database in Neon is the system of record for anything the calculator multiplies. System of record means the one place where a fact is the truth, and everything else is a copy. A figure never travels from a retrieved chunk into a valuation directly. It is first written to a Postgres row with its source, license class and vintage, and the memo writer’s numbers go to `valuation_inputs`, never parsed from prose. “What is the India default spread today?” is routed to the database tool, not the index. Tables retrieve badly as text, and the database will hold the July 2026 value of 1.75% once Milestone 2 seeds it.

Source: Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3 sections 6 and 7; bfxcvcj1s.txt, section 6 retrieval notes; Valuation/docs/plan.md, section 10.5; bafb6h9rb.txt, India rows (2026-09-14)

A concrete example. The July 2026 country-risk post says that “country risk exposure comes less from where the company is incorporated and more from where it operates”. That is a paragraph the librarian should find when you ask how to treat a Siemens factory in India. The 1.75% default spread that goes with it is a number, and it will live in the `country_risk` table with a vintage id.

Source: financeMD/damodaran/blog/2026/07/country-risk-drivers-measures-and.md, line 172 (2026-07-15); Valuation/docs/plan.md, section 10.3

Why Pinecone, and not vectors inside Neon

One research report argued for `pgvector`, a Postgres extension that keeps vectors in the same database as the fundamentals. At 60,000 vectors, it said, we are about 200 times below the point where a dedicated vector database pays for itself, and SQL joins against the fundamentals tables come free. The plan chose Pinecone anyway: it was your preference, it is free at this size, and hybrid

sparse-plus-dense search comes built in. The retriever sits behind one interface, so pgvector on Neon remains the fallback if two systems ever feel like a burden.

Source: Valuation/docs/sources/b31x0pyot.txt, section 6; Valuation/docs/plan.md, section 7 (2026-09-14)

Learn it

All free. Hours are rough reading-and-tinkering estimates.

#	Resource	Format	Hours	Why this one
1	https://www.pinecone.io/learn/series/rag/	Article series	3	The best plain-English sequence from embeddings through chunking, retrieval and generation. Read it first.
2	https://www.pinecone.io/learn/series/rag/rerankers/	Article with a notebook	1	Shows, with code, how much a reranking pass improves the chunks that reach the model.
3	https://docs.pinecone.io/guides/search/hybrid-search	Official docs	1	Sparse plus dense in one index, the pattern we need for tickers, ratings and file names.
4	https://docs.voyageai.com/docs/pricing	Official docs	0.5	Read the free allowance and prices yourself, then set the billing alert from Document A.
5	https://github.com/pgvector/pgvector	Repository README	1.5	The alternative we did not pick. Reading it explains what Pinecone does for us.
6	https://www.anthropic.com/news/contextual-retrieval	Article	1	Why a chunk embedded with knowledge of its neighbors retrieves better, the idea behind voyage-context-4. From my web knowledge, not the local reports; verify.

Source: Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3 section 6 resource table; bcy86uopa.txt, source table (2026-09-14)

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

The point is to feel the difference between exact words and meaning, on the real corpus.

1. Run the three keyword searches below and note the counts. A keyword engine treats these as three different things.

```
cd ~/Downloads/financeMD
grep -ril "sales to capital" . | wc -l
grep -ril "sales/capital" . | wc -l
grep -ril "sales-to-capital" . | wc -l
```

2. Open one blog post and read one paragraph:

```
sed -n '172p' damodaran/blog/2026/07/country-risk-drivers-measures-and.md
```

3. On paper, write the question that paragraph answers, in your own words, without reusing his. That rewrite is what an embedding captures: the idea, not the spelling.
4. Write the citation you would store: file path, line number, the date from the front matter (the header block between --- lines at the top of the file), and the license class (damodaran_public).
5. Decide: is anything in that paragraph a number the calculator would multiply? If not, it belongs to the librarian. If yes, it belongs in a database row first.

Done when

KEY IDEA

- You can explain chunk, embedding, cosine similarity, reranker and hybrid search using the librarian, without notes.
- You can say why the corpus cannot be read in one call, and what a RAG answer costs on GLM-5.3-Flash versus stuffing a packet.
- You can name the four license classes that enter the index and the three that never do, and you know the fence is a database rule.
- You can state which Voyage model embeds, which reranks, and what the free allowance is.
- You can repeat “retrieval finds the paragraph; the database holds the number” and give one example of each.
- You have done the exercise and kept the paper.

Chapter 7. OpenRouter and benchmark literacy: one door to every model, and how to read the scoreboard

OPTIONAL - FIRST PASS

For model configuration and comparison. Separate what a model does from how its price and benchmark score are measured.

What it is (plain words and an analogy)

OpenRouter

An **API** (application programming interface) is a door that one program uses to talk to another. Every AI company has its own door, its own key, and its own bill. **OpenRouter** is a single door in front of hundreds of those doors. You hold one key, you keep one credit balance, and you pick a model by changing one name in your code. If the company serving that model goes down, OpenRouter can quietly route the same request to another company serving the same model.

Source: Valuation/docs/sources/b31x0pyot.txt, section 3 (2026-09)

The analogy is a food-delivery app. You do not open an account with every restaurant. You open one account with the app, and the app takes your order to whichever kitchen you choose. The app adds a service fee on top-ups, but the menu prices are the kitchen's own prices. OpenRouter works the same way: it charges a fee when you buy credit, and charges nothing extra per token.

Source: Valuation/docs/sources/bcy86uopa.txt, section 4 (2026-09-14)

Artificial Analysis

A **benchmark** is a standardized exam for a model. **Artificial Analysis** (AA) is an independent site that runs many such exams on many models and publishes the marks, together with speed and price. Think of it as the consumer-testing magazine for AI models. The headline mark is the **Intelligence Index**, a weighted average of ten exams. Underneath it sit narrower scoreboards for coding agents, for making things up, and for web-search services.

Source: Valuation/docs/sources/b31x0pyot.txt, section 5

Why this tool now (for this project)

The two places a model is used

This confused you before, so slowly. **Place 1 is while you write code.** That is Cursor, on the models Cursor includes in the \$20 subscription you already pay. You never paste an API key into Cursor. Doing so is unofficial, switches off features like Tab, and voids Cursor's zero-data-retention promise. Privacy Mode stays on.

Source: Valuation/docs/read-this-first.md, sections 4 and 11 (2026-09-14);

Valuation/docs/sources/btzaixzyh.txt, CHECK 1

Place 2 is inside the product when it runs at night. Our own code, started by GitHub Actions (Chapter 3), calls a model to write memos, answer librarian questions, and clean transcripts. Those calls go through OpenRouter. That is where the volume and the cost live, so that is where the cheap open-weight models go. **Open-weight** means the model's weights are downloadable and hosted by many companies, so no single vendor can switch you off, and a pinned checkpoint keeps a 2026 valuation reproducible in 2029.

Source: Valuation/docs/plan.md, sections 2a and 2b (2026-09-14)

The price of that choice is capability. On the AA Intelligence Index v4.3 (released 2026-09-07) the frontier models score 53 and the best open-weight model, GLM-5.3, scores 45. The gap sits in long autonomous coding tasks, not in the "read a filing and extract" work that fills a valuation pipeline. Open-weight models cost 7 to 40 times less per token, so for Place 2 the trade is worth it.

Source: Valuation/docs/read-this-first.md, section 4; bcy86uopa.txt, section 1b

Cline, the free extension inside Cursor, is the one exception that touches both places. It takes your OpenRouter key and lets you feel what an open-weight model does before its name goes into a robot. It is a bench, not the daily driver.

Source: Valuation/docs/read-this-first.md, section 11

How it is used in this project

Three settings on the OpenRouter account

You will create the account in milestone 1 (Document A, section 10). Buy \$20 of credit. Then set three things before any code runs.

Setting	Value	Why
Monthly spend limit	\$75	The budget line for OpenRouter is \$30 to \$70 a month, inside the \$200 ceiling
Prompt logging	Off	Logging is opt-in, and turning it on hands OpenRouter broad rights over your prompts
Zero-data-retention routing	On	Only companies that promise not to store your prompts may serve your requests

Source: Valuation/docs/read-this-first.md, sections 10 and 12

Zero data retention (ZDR) means the company that ran your request deletes the prompt and the answer once the answer is sent. OpenRouter is the only router in the vendor scorecard that can enforce ZDR per request, which is the feature that makes “everything private” possible through a router. The research also notes that enabling prompt logging “grants OpenRouter an irrevocable right to commercial use” of the logged text. That is why logging stays off.

Source: Valuation/docs/sources/bj0fbejiq.txt, Role 4

Fees, and the credit arithmetic

OpenRouter adds no markup on tokens. It charges 5.5% when you buy credit by card, with a minimum of \$0.80 per purchase, or 5% by USDC (a dollar-pegged cryptocurrency; ignore this option). On small top-ups the minimum bites, so buy in \$20 chunks rather than \$5 chunks. Models with a : free suffix exist, but none of our four picks has one, and free models are capped at 20 requests a minute and at most 1,000 a day. Models with a : batch suffix cost half price for work that can wait, which is exactly our bulk job. One more fact that shaped the design: OpenRouter serves no embedding or reranking models at all, so the librarian’s embeddings come from Voyage.

Source: bcy86uopa.txt, section 4

Why not OpenRouter’s web-search plugin

OpenRouter offers a plugin that lets a model search the web mid-answer. We do not use it. The ZDR guarantee covers inference routing only. The plugin runs under a third party’s retention rules, so any prompt that touches it leaves the private zone. If the project ever needs web search for the news layer, it will use a separate vendor, chosen from the AA Search Index, described below.

Source: bj0fbejiq.txt, Role 4 and the accounts checklist

The model picks, and why each

The runtime will pin four model ids in one file, with the AA index version next to them. Prices will be re-checked monthly. A swap will be one line. All four are open-weight and all go through OpenRouter with ZDR on.

Job	OpenRouter id	Price per million tokens, in / out	Why this one
Write memos, answer questions	z-ai/glm-5.3	about \$0.92 to 1.40 / \$3.14 to 4.40	Best open-weight score on the index (45); lowest hallucination rate among strong open weights (30%)

Job	OpenRouter id	Price per million tokens, in / out	Why this one
Bulk cleaning: punctuate transcripts, tag passages	z-ai/glm-5.3-flash: batch	\$0.075 / \$0.25	Half price for work that can wait; the whole transcript job costs about \$11 once; reasoning must be switched off
Judge: grade the librarian's answers in evaluations	moonshotai/kimi-k3	\$2.65 / \$13.28	Best-calibrated open weight on AA-Omniscience (+18 to 20); it abstains rather than invents; the judge must never be the model being judged
Fallback	deepseek/deepseek-v4.1-flash	\$0.15 / \$0.60	Fastest in class at 214 tokens a second; plain MIT license
Never	DeepSeek V4 Pro		94% hallucination rate on AA-Omniscience, fatal for a chat that must cite

Source: Valuation/docs/read-this-first.md, section 4.1; bcy86uopa.txt, sections 1a, 3 and 9

To make the prices concrete: a typical librarian answer sends about 8,000 tokens and receives about 700. That costs about \$0.0143 on GLM-5.3 and about \$0.00155 on GLM-5.3-Flash. One judge call on Kimi K3 costs about \$0.0298, so a 200-case evaluation run costs about \$5.96. A **token** is a word piece, roughly four characters.

Source: Valuation/docs/read-this-first.md, section 4.1; bcy86uopa.txt, section 9

One caution on licenses. The model-selection report lists GLM-5.3 under a bespoke permissive license that is explicitly not MIT, and DeepSeek V4.1 Flash under plain MIT. The plan describes GLM-5.3-Flash as plain MIT, while the report lists it as bespoke permissive. For a private project neither wording matters. If the system ever becomes public, read each model card yourself before deciding.

Source: bcy86uopa.txt, section 1a; Valuation/docs/plan.md, section 2a

Reading the Artificial Analysis pages

Intelligence Index v4.3. Ten exams in four groups, released 2026-09-07. The weights matter because they tell you what the headline rewards.

Group	Weight	Exams inside it
Agents	30%	AA-Briefcase 15%, GDPval-AA v2 10%, AutomationBench-AA 5%
Coding	20%	Terminal-Bench v4.0 10%, SciCode 10%
General	30%	AA-Omniscience accuracy 10%, AA-Omniscience non-hallucination 5%, GDP.pdf 10%, AA-LCR v1.1 (long context) 5%
Scientific reasoning	20%	Humanity's Last Exam 10%, CritPt 10%

Half the headline is agent and coding work, which our pipeline barely does. Only 15% is “does it make things up”, which is the part we care about most. That is why the picks were made from the sub-scores, not the headline. Current marks: frontier 53, GLM-5.3 45, Kimi K3 44, GLM-5.3-Flash 42, DeepSeek V4.1 Flash 40, DeepSeek V4 Pro 36.

Source: bcy86uopa.txt, section 1

Coding Agent Index v1.5 (snapshot 2026-09-11) is different in kind. It scores a **harness plus model pair**, not a model alone. A harness is the tool wrapped around the model: the prompts, the file access, the permission gates. Claude Code with Fable 5.1 scores 62.2%. The best open-weight pair, Opencode with GLM-5.3, scores 53.6%. This index describes Place 1 work, so it matters for Cursor and Cline, not for the nightly robots.

Source: bcy86uopa.txt, section 2

AA-Omniscience is the hallucination exam. Six thousand closed-book questions across 42 topics. The index runs from minus 100 to plus 100. A right answer earns points, a confident wrong answer loses points, and “I do not know” costs nothing. **Calibration** is the habit of knowing when you do not know. As of 2026-08-27 only 46 of 167 models scored above zero. Kimi K3 scores +18 to 20 with a 51% hallucination rate on the questions it attempts. GLM-5.3 scores 14 with a 30% hallucination rate. DeepSeek V4 Pro hallucinates 94% of the time, which is why it is banned from the answer path despite a decent headline score. In our system the facts arrive from retrieved passages, so raw recall matters less. Willingness to abstain transfers directly.

Source: bcy86uopa.txt, section 3

Search Index (launched 2026-08-18) grades web-search services, not models. AA holds the model and the agent scaffold fixed and swaps only the search provider. Three exams are averaged: DeepSearchQA, BrowseComp, and an AA-Omniscience subset. Parallel scores 75, Exa 74, Firecrawl 73; the no-search baseline is 33. Search cost alone ranges from \$13.64 per 1,000 tasks (Parallel turbo) to \$126 (Tavily basic), which is why Tavily is the worst value on the board; a second report (bj0fbejiq.txt) quotes higher figures for the same index, \$60.21 and \$192.58, so re-check the AA page before choosing a vendor.

Source: b31x0pyot.txt, section 5; bj0fbejiq.txt, section 5

Speed and money columns. Time to first token (TTFT) is the seconds until the first token arrives; for reasoning models that is the first thinking token, so a fast TTFT can still mean a long wait for a useful word. Output tokens per second is the speed after that: GLM-5.3 runs at 72, GLM-5.3-Flash at 107, Kimi K3 at 37, DeepSeek V4.1 Flash at 214. Cost per task is the average dollars to finish one index task, computed from real token counts and cache-hit rates; this is the column that maps to a bill. Blended price is a single dollars-per-million figure that assumes a fixed mix of cached, input and output tokens. AA’s blended price for GLM-5.3 shows \$0.90; OpenRouter lists \$1.40 in and \$4.40 out. Always budget from OpenRouter’s numbers.

Source: bj0fbejiq.txt, section 5; bcy86uopa.txt, sections 0 and 1a

The five traps

1. **Index inflation.** Labs tune for the composite. Read the sub-exam that matches your job: AA-Omniscience and AA-LCR for valuation, not Humanity’s Last Exam.
2. **Provider versus model pricing.** The same open-weight model is served by many companies at different prices, speeds, and **quantizations** (compressed copies that are cheaper and slightly dumber). The leaderboard row may not be the endpoint you get routed to. Pin the provider.
3. **Reasoning tokens bill as output.** A \$0.30 model that thinks for 8,000 tokens costs more than a \$3 model that answers in 300. Compare cost per task, never dollars per million. This is why reasoning is off for the bulk job.

4. **Blended price assumes a cache-hit rate you may not reach.** A **cache** is the provider re-using an unchanged prompt prefix at a discount; GLM-5.3 cache reads cost \$0.26 instead of \$1.40. If every prompt changes, your real bill can be several times the blended figure. Keep a stable prefix.
5. **Harness dependence.** A model at the top of the Coding Agent Index under one scaffold may do worse under another.

Source: bj0fbeiqr.txt, section 5; bcy86uopa.txt, sections 7 and 9

Learn it

All free. About five hours in total.

#	Resource	Format	Hours	Why
1	https://artificialanalysis.ai/methodology/intelligence-benchmarking	Official methodology page	1	The exact v4.3 weights and how token counts and cache rates feed cost per task
2	https://artificialanalysis.ai/evaluations/omniscience and the paper at https://arxiv.org/pdf/2511.13029	Leaderboard plus paper	1.5	Hallucination and abstention, the axis that matters most for a system that must cite
3	https://artificialanalysis.ai/methodology/coding-agents-benchmarking	Official methodology page	0.5	Why that index scores harness plus model pairs, and why it belongs to Place 1
4	https://openrouter.ai/docs/features/provider-routing	Official docs	1	Pin providers, exclude quantizations, set price and latency ceilings
5	https://artificialanalysis.ai/methodology/search-api	Official methodology page	0.5	How to choose a search vendor on evidence if the news layer ever needs one
6	OpenRouter docs, the page on zero data retention and data policies (search the docs site for “zero data retention”; verify the exact URL)	Official docs	0.5	What ZDR routing does and does not cover

Source: bj0fbeiqr.txt, section 5 resource table; bcy86uopa.txt, section 0

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter’s tool. The following “Done when” checklist tells you how to judge completion.

Pick a model for one job and defend the pick in writing.

1. The job: tag 60,000 corpus passages with a topic label each. Not urgent. Finance jargon such as “sales-to-capital” and “Baa3” must survive. Every passage sends about 600 tokens and gets 150 back.

2. Open <https://artificialanalysis.ai/models/open-source>. Sort by price. Write down the three cheapest models with an Intelligence Index of 40 or more.
3. For each of the three, open its AA-Omniscience row and note the hallucination rate.
4. Open <https://openrouter.ai/models>, search each id, and copy the real in and out price and whether a :batch variant exists.
5. Work out cost per task for each: 600 times the input price plus 150 times the output price, divided by one million. Multiply by 60,000.
6. Choose. Write five lines in your decisions log (Chapter 0, rule 6): the job, the pick, the cost per task, the trap you checked, the number you would recheck next month.

Then compare with the project's answer: `z-ai/glm-5.3-flash:batch`, about \$4.95 for the tagging half of the bulk job, reasoning off. If you chose differently with a written reason, that is a good outcome, not a wrong one.

Source: `bcy86uopa.txt`, section 7

Done when

KEY IDEA

- You can say, in one sentence each, what OpenRouter is and what Artificial Analysis is.
- You can explain why Cursor never sees an OpenRouter key, and where the open-weight models actually run.
- Your OpenRouter account has a \$75 monthly limit, logging off, and ZDR routing on, and you can say why each.
- You can name the four pinned model ids and give one number that justifies each.
- You can list the five traps without looking, and say which one the bulk job guards against.
- Your decisions log holds the five-line note from the exercise, with a cost per task in it.

Chapter 8. “Fine-tuning” at three levels: the harness, the context, and the model

OPTIONAL - FIRST PASS

For improving the assistant later. Keep the three meanings of tuning distinct before choosing a change or reading the deeper examples.

What it is (plain words and an analogy)

You asked how to “fine-tune this thing”. The word has three meanings in practice, and people mix them up. Only the third one changes the model itself. The first two change what the model sees and what it is allowed to do. In this project you will spend nearly all your time on the first two.

Think of the model as a brilliant new employee who has amnesia every morning. You cannot retrain their brain each day. You can leave them a desk. On the desk sits a job description, a filing cabinet of procedures, a set of tools, and some pre-written memos. That desk is level 1, the harness. A harness is the program that wraps the model, feeds it files, and runs its commands. In this project the harness is Cursor while you code, and our own TypeScript while the product runs.

Level 2 is what you put on the desk for today's task. The model reads a fixed amount of text per request, called the context window. Choosing what goes into that window is called context engineering. Writing the instruction itself is called prompt engineering.

Level 3 is sending the employee back to school. Training changes the model's weights, the millions of numbers inside it that decide how it answers. This is what most people mean by fine-tuning. It is real, it is cheap to run, and it is not what this project needs yet.

Source: Valuation/docs/sources/bj0fbejiq.txt, CHECK 3, section 1 (2026-09)

Level	What changes	Cost	Reversible?	Who does it here
1. Harness	Rules file, tools, prompt templates, model routing	\$0 and your time	In seconds, by a pull request	You, from M3
2. Context	Which facts and examples enter the window per request	\$0 and your time	In seconds	You and the code
3. Model weights	How the model behaves by default	About \$2 per training run, but see the serving trap	Only by retraining	Nobody, until stated conditions are met

Source: Valuation/docs/plan.md, section 8, "Fine-tuning, three levels" (2026-09-14)

Why this tool now (for this project)

The plan's own summary is blunt: the harness is "\$0 and 95% of the quality; this is what you will do." The tool landscape report reaches the same conclusion from a different angle. Damodaran's method is facts, arithmetic, and a book of rules. Cost-of-capital tables, equity risk premiums, and industry betas change every year. Facts that change yearly belong in a database, where they can be updated and cited. They do not belong baked into a model's weights, where they go stale and cannot be cited.

Source: Valuation/docs/plan.md, section 8 (2026-09-14); Valuation/docs/sources/b31x0pyot.txt, section 4 (2026-09)

The same report gives the accepted 2026 order of work: prompt first, then retrieval, then fine-tuning, then distillation (training a small model to copy a big one's answers). Its rule of thumb is that fine-tuning is for form, not facts. Form means output shape, tone, and refusal habits. This project's form problems, such as always emitting a valid table of assumptions, are solved for \$0 by a prompt plus a schema plus a validator function. So the chapter teaches level 3 so you understand it, and tells you honestly why it stays on the shelf.

Source: Valuation/docs/sources/b31x0pyot.txt, section 4 (2026-09)

How it is used in this project

Level 1. The harness: the files that will exist

Milestone M2 will create these files in the private repo valuation. Nothing after M0 exists yet, so read "will" throughout. Tuning the harness means editing one of these files, opening a pull request, and reading the diff (chapter 1).

File or folder	What it is	What “tuning” it means
AGENTS.md	The eight hard rules, read by Cursor, Cline, and Claude alike	Add or sharpen a rule when the model repeats a mistake
.cursor/ rules/	The same rules on one page; Cursor reads them on every request	Keep them short so they always fit in the window
.cursor/mcp. json	Registers the system’s tools inside Cursor	Add a tool, or take one away
packages/ mcp-server	The tools themselves: value_company, write_memo, review_memo, get_industry_stats, search_corpus, report	Change what a tool returns, or what it refuses
packages/ models	One file with the pinned model ids and prices	Model routing: swap a model in one line
packages/ memo and packages/rag	The memo template and the grounded prompt	Rewrite a prompt, add an example, change the citation format

Source: Valuation/docs/plan.md, milestone M2 and section 10 (2026-09-14);
Valuation/docs/read-this-first.md, sections 4.1 and 11 (2026-09-14)

MCP stands for Model Context Protocol. It is a standard plug that lets an AI tool call other programs. It matters for tuning because a tool is a fence. When the model must call get_industry_stats to get a beta, it cannot invent one. The rules file says what to do; the tool set says what is even possible.

Source: Valuation/docs/read-this-first.md, section 11 (2026-09-14)

Model routing is the choice of which model does which job. In the runtime that is z-ai/glm-5.3 for memos and answers, z-ai/glm-5.3-flash:batch for bulk cleaning, moonshotai/kimi-k3 as the judge, and deepseek/deepseek-v4.1-flash as the fallback. DeepSeek V4 Pro is never used, because it scored a 94% hallucination rate on the AA-Omniscience test. All of this lives in packages/models, pinned to the Artificial Analysis index version it was chosen under. Changing a model is a one-line edit and a pull request, not a retraining job.

Source: Valuation/docs/read-this-first.md, section 4.1 (2026-09-14); Valuation/docs/plan.md, section 10.4 (2026-09-14)

Sub-agents are helper sessions with their own context that return a summary. The plan defers them to v2. The rule for later is that helpers write database rows, never prose you must audit. The .pi/ folder layout you may see in the model-selection report (skills, prompts, extensions, agents) belongs to pi, which is also deferred.

Source: Valuation/docs/plan.md, section 8, “Sub-agents” (2026-09-14);
Valuation/docs/sources/bcy86uopa.txt, “PI SETUP FOR THE VALUATION REPO”, part B (2026-09)

Level 2. Prompt and context engineering

A prompt is the question on the exam paper. Context is which textbooks, notes, and past papers are on the desk. Choosing the desk is the harder and more valuable skill. Anthropic’s article of 29 September 2025 names the enemy: context rot. As the number of tokens in the window climbs, the model’s recall of any single fact degrades. A token is a word piece, roughly four characters. The discipline is the smallest set of high-signal tokens, not everything you have.

Source: Valuation/docs/sources/bj0fbeiqli.txt, CHECK 3, section 1b (2026-09)

For a valuation this has a concrete meaning. Never paste a 200-page annual filing into the window. Extract the eight line items you need into a database row, then put that row in the window with its citation. The plan's librarian follows a hybrid rule: a question about one document gets that document stuffed in whole; a question across the corpus retrieves 50 passages, then a second model scores them again (reranking) and keeps the best 8.

Source: Valuation/docs/sources/bj0fbeiqli.txt, CHECK 3, section 1b (2026-09); Valuation/docs/plan.md, section 10.4 (2026-09-14)

The money says the same thing. A retrieval question that sends 8,000 tokens and gets 700 back costs about \$0.0143 on GLM-5.3. Stuffing 200,000 tokens into the same model costs about \$0.283, which is about 20 times more. Prompt caching helps too: a cached read on GLM-5.3 costs \$0.26 per million tokens against \$1.40 uncached, a 5.4 times discount.

Source: Valuation/docs/sources/bcy86uopa.txt, RESEARCH 1, "Three operational rules" and KEY FACTS (2026-09-14)

Few-shot examples are worked answers you show the model before asking your question. They teach form better than instructions do. This project already has its example: the "Stories to Numbers" sheet in Damodaran's own workbook. Each assumption row ends with a "Link to story" cell, and the sheet's own instruction reads "Tie each assumption to the part of your story that relates to it." The memo writer will use that sheet as its template, one row per driver, with the link mandatory.

Source: financeMD/damodaran/pc/fcfsimpleloginzu.md, "Stories to Numbers" sheet (2026-02-01); Valuation/docs/plan.md, section 10.5 (2026-09-14)

Evals are the steering wheel. An eval is a fixed set of questions with known good answers, run again after every prompt change. Without one, prompt editing is guesswork. With one, it is measurement. The database will contain rag_eval_case and rag_eval_run tables. Langfuse Hobby is free at 50,000 units a month, and its datasets and evaluation runner are included on every tier. Every model call will carry vintage, source_ids, license_class, model_id, and prompt_version in its meta-data, so a score can be sliced by which prompt produced it. Kimi K3 grades the answers, and the judge is never the model being judged. Chapter 11 covers the mechanics.

Source: Valuation/docs/plan.md, section 10.3 (2026-09-14); Valuation/docs/sources/bo0hrclp7.txt, section 6, "LLM observability / evals" (2026-09); Valuation/docs/read-this-first.md, section 4.1 (2026-09-14)

Level 3. The model itself, as an outsourced service

Fine-tuning proper changes the weights so the model behaves differently by default. The analogy in the explainer report is exact: you are not teaching the analyst new facts, you are drilling your firm's memo template into them until they cannot write it wrong. LoRA, short for Low-Rank Adaptation, trains a small adapter on top of a frozen base model. That is why it is cheap. A form task needs roughly 500 to 1,000 high-quality examples, and every example must be exactly the output you want.

Source: Valuation/docs/sources/bj0fbeiqli.txt, CHECK 3, section 1c (2026-09)

The best evidence for finance is the FinLoRA paper (arXiv 2505.19819, May 2025). It tested five LoRA methods on five base models across 19 financial datasets, including four new XBRL sets built from 150 SEC filings. XBRL is the tagged format companies use to file numbers with the SEC.

FinLoRA result	Figure
Average lift of LoRA over the base models	+36%

FinLoRA result	Figure
Vanilla LoRA (8-bit, rank 8) on Llama 3.1 8B	74.74 versus 37.05 for the base model
Training time	14.1 to 15.9 hours on four A5000 cards, 56 to 64 GPU-hours
Compute cost at \$0.26 per GPU-hour	\$14.66 to \$16.54

Source: Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3, section 1c; Valuation/docs/sources/b31x0pyot.txt, section 4, citing <https://arxiv.org/abs/2505.19819> (2025-05-26)

The 2026 prices per training run are small. Prices are per million training tokens. SFT means supervised fine-tuning, the plain kind. An epoch is one pass over your examples.

Vendor	Price for LoRA SFT	Note
Together AI	about \$0.48 (16B parameters or less), about \$1.50 (17 to 69B), about \$2.90 (70 to 100B)	Weights can be downloaded
Fireworks	\$0.50 (under 16B), \$3.00 (16 to 80B), \$6.00 (80 to 300B), \$10.00 (over 300B)	DPO (direct preference optimisation, a second training style that learns from pairs of better and worse answers) costs about twice SFT
Google Vertex	Training tokens = dataset tokens times epochs	Tuned endpoint priced the same as the base model
Unsloth on Colab	Free on a T4	A 7 to 8B model fits in 6 to 8 GB; 1,000 to 10,000 examples take 15 to 60 minutes

Worked arithmetic: 800 examples times about 1,500 tokens times 3 epochs is about 3.6 million training tokens, or about \$1.80 on a 16B-or-smaller LoRA. Ten experiments cost under \$20.

Source: Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3, section 1c (2026-09)

The training run is never the expense. Two things are. First, data curation: 500 to 1,000 hand-corrected pairs is weeks of your evenings, and the tool landscape report budgets three to five times the training cost for a year of retraining and evaluation. Second, serving. A dedicated GPU endpoint left running at \$2 to \$10 an hour will eat the whole \$200 monthly ceiling in a weekend. The rule, if you ever do this, is serverless LoRA inference billed at the base-model rate, or shut the endpoint down. OpenRouter can route a Together or Fireworks key for a 5% fee, waived for the first million requests a month, but routing a truly private endpoint is an Enterprise-plan feature. Two of the friendlier solo options are gone: Predibase was acquired by Rubrik in June 2025 and OpenPipe by CoreWeave in September 2025.

Source: Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3, section 1c (2026-09);
Valuation/docs/sources/b31x0pyot.txt, section 4 (2026-09)

Why not yet for this project. Level 3 is revisited only when two conditions are both true: the running system has logged at least 500 hand-corrected outputs, and there is a measurable, repeated format failure that a prompt plus a schema plus a validator cannot fix. Examples would be a model that keeps mislabeling operating leases or will not emit the assumptions table in the required shape. Until then it is a \$2 experiment with a \$200 trap attached, solving a problem you do not have.

Source: Valuation/docs/plan.md, section 8 (2026-09-14); Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3, section 1c (2026-09)

Learn it

All free. Read them in this order; the last two are for understanding, not for doing.

#	Resource	Format	Hours	Why
1	https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents	Article	1	The reference on context rot and just-in-time retrieval; explains why the librarian retrieves instead of stuffing
2	https://github.com/anthropics/prompt-eng-interactive-tutorial	Notebooks, 9 chapters	4 to 6	Hands-on few-shot and structured-output practice you can port to open-weight models
3	https://www.anthropic.com/engineering/building-effective-agents	Article	0.75	When not to use an agent; most of this pipeline is a plain workflow
4	https://www.promptingguide.ai/	Site	3	Model-agnostic; good on chain-of-thought and self-consistency
5	https://arxiv.org/abs/2505.19819 and https://github.com/Open-Finance-Lab/FinLoRA	Paper and code	1 (abstract and results table only)	The only rigorous LoRA benchmark on SEC and XBRL tasks
6	Unsloth docs, fine-tuning guide (unsloth.ai, verify)	Docs and free Colab	3 to 4	Only when the two revisit conditions are met; cheapest end-to-end run

Source: Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3, sections 1b and 1c resource tables (2026-09)

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

Rewrite one prompt and compare two answers. Use Cursor's chat with Privacy Mode on, or any chat model you already pay for. The material is public Damodaran text, so nothing private leaves your machine.

1. Prompt A, no context: "Why is the terminal cost of capital equal to the risk-free rate plus the mature-market ERP?" Save the answer.
2. Open `financeMD/damodaran/pc/fcfsimpleleginzu.md` and copy the lines around the label Mature Market ERP +.
3. Prompt B, with context: paste those lines, then ask the same question, and add "Answer in three sentences. Quote the label you relied on. Say if the label and the formula disagree."
4. Compare. Does answer A mention the misleading "+4.5%" label at all? Does answer B notice that the label says one thing and the formula another? Which answer could you cite in a memo?

5. Write two lines in your decisions log (chapter 0, rule 6): what you added to the window, and what changed in the answer.

Source: Valuation/docs/sources/bkdrkpfnfng.txt, terminal cost of capital note (2026-09);
Valuation/docs/plan.md, milestone M1, day-one exercise (2026-09-14)

Done when

KEY IDEA

- You can state the three levels, each in one sentence, with its cost.
- You can name the six harness files from the table above and say what tuning each one means.
- Your decisions log holds the A and B answers from the exercise, and you can say which one you would cite.
- You can recite the two conditions under which level 3 gets revisited, and the serving trap that goes with it.

Chapter 9. Data sources: where every number comes from, and what you may do with it

OPTIONAL - FIRST PASS

For connecting data sources. Focus on where numbers come from and their usage labels; look up individual endpoints when implementing them.

What it is (plain words and an analogy)

A data source is any place the system fetches a number or a sentence from. Think of the system as a restaurant kitchen. The engine (Chapter “The DCF chain” in Document C) is the recipe. The data sources are the suppliers who deliver ingredients to the back door.

Every supplier differs in three ways: a delivery schedule (daily, monthly, yearly), a container (a JSON file, an old Excel file, a CSV), and a contract saying what you may do with the goods. In this project that contract is written onto every stored row as a licence class (section 3.3 of read-this-first.md).

Source: Valuation/docs/read-this-first.md, section 3.3 (2026-09-14)

Two terms you will meet on every page below:

- An API (application programming interface) is a web address that returns data for a program instead of a web page for a human.
- A rate limit is the maximum number of requests a supplier allows per second or per day. Exceeding it gets you blocked.

Why this tool now (for this project)

Damodaran himself tells you where the numbers should come from. His data page says that for a live valuation you should “get real time data from the company’s filings or annual report.” His own tables are industry averages, refreshed mostly once a year. So the project needs two kinds of suppliers: company filings for the base year, and his tables for the averages that fill in the rest.

Source: financeMD/damodaran/New_Home_Page/datahistory.md, rule 3 (corpus copy, 2026)

The second reason is money and law. The system must run under \$200 a month, and everything stored must be legal to keep. So: free public sources first (SEC, Damodaran, FRED, NSE files), one paid licence for India statements (EODHD), and nothing scraped from apps or newspapers.

Source: Valuation/docs/read-this-first.md, section 12 (2026-09-14)

How it is used in this project

The nightly robot (Chapter 3, GitHub Actions) wakes at about 6:17 UTC, checks each watchlist company for a new filing, downloads the numbers, builds trailing-twelve-month figures, and writes them to Neon with source, licence class and vintage. A monthly and quarterly job watches Damodaran's files for a changed fingerprint. A third job pulls licensed headlines and stores them as annotations only.

Source: Valuation/docs/read-this-first.md, section 7 (2026-09-14)

The source table

Source	What it supplies	Cadence	Format	Cost	Licence class
SEC EDGAR	US company statements, filing lists	nightly	JSON	free	public_domain
Damodaran datasets	industry averages, country risk, implied ERP	yearly, quarterly, monthly	.xls and .xlsx	free	damodaran_public
EODHD	India (and global) statements	on request	JSON	\$60 a month, from M4	licensed_personal
NSE bhavcopy	daily NSE closing prices	daily file	CSV in a zip	free	read NSE terms before use
INDstocks API	your own holdings, quotes, price history	on request	JSON	free with account	licensed_personal
FRED	US Treasury yields, credit spreads	daily	CSV or JSON	free	open_access
Bloomberg	reading only	n/a	web, email	your own subscription	proprietary_personal

Source: Valuation/docs/read-this-first.md, sections 3.3, 10 and 12; Valuation/docs/plan.md, section 7 (2026-09-14)

SEC EDGAR

EDGAR is the US regulator's public filing system. Every listed US company files its statements there in XBRL, a tagging standard that gives every number a name such as `OperatingIncomeLoss`. There is no API key and the data is public domain.

Three endpoints matter. Each takes the company's CIK, a ten-digit identifier padded with zeros.

Endpoint	Address pattern	What you get
submissions	<code>https://data.sec.gov/submissions/CIK#####.json</code>	the list of filings, each with an accession number
companyfacts	<code>https://data.sec.gov/api/xbrl/companyfacts/CIK#####.json</code>	every tagged number the company ever filed
frames	<code>https://data.sec.gov/api/xbrl/frames/us-gaap/<Tag>/USD/CY2019Q1I.json</code>	one tag, one period, across all filers

The nightly job reads submissions first. A new accession number means a new filing, and only then does it fetch companyfacts. The frames endpoint is your free peer-group builder: one number for every filer in one period, which is how you check a US industry median for free. Bulk zip files of both are published nightly at about 03:00 US Eastern time.

Source: Valuation/docs/sources/bkm2kob5x.txt, section 12 (2026-09-14)

The User-Agent rule. The SEC requires every request to carry a descriptive User-Agent header with your name and email. A request without it is refused. Read the SEC's fair-access policy before you write any loop, because it sets a request-rate limit and the SEC blocks addresses that exceed it.

Source: same report, section 12

The LTM construction. LTM means "last twelve months." Company filings do not give it to you directly. A 10-K (the annual report) covers a fiscal year, and a 10-Q (the quarterly report) covers a year-to-date period. The rule is: LTM = last 10-K, minus the prior year's year-to-date interim, plus the current year-to-date interim. Damodaran's own workbook has a "Trailing 12 month" worksheet that computes exactly $E = B - C + D$, but it is a standalone helper that you paste into by hand. Our ingest code does the subtraction and records which three filings it used.

Source: Valuation/docs/sources/bkdrkpfng.txt, sections 1.2 and 3.8 (2026-09-14)

Missing interest expense. The workbook turns interest expense into a synthetic credit rating through the coverage ratio, operating income divided by interest expense. If interest expense is zero the sheet sets coverage to 1,000,000, which reads as a perfect rating. Apple's `InterestExpense` tag exists in companyfacts, but its last value covers the year ending 2023-09-30 (3,933 million dollars) and nothing after. So a naive loader sees "missing" and writes zero, and the model rewards Apple with a top rating by accident. The plan's fix is a visible `interest_missing` flag on the `ltm_financials` row, never a silent zero.

Source: `https://data.sec.gov/api/xbrl/companyfacts/CIK0000320193.json` (fetched 2026-09-14); `bkdrkpfng.txt`, section 3.4; Valuation/docs/plan.md, sections M2 and 10.3 (2026-09-14)

Damodaran's datasets

Cadence. His data page says: "I update most of the data only once a year, in the first two weeks of January." The current full update is dated January 9, 2026. Three things move faster: country risk premiums (January, April, July), the implied equity risk premium (monthly), and a daily market tape during crises.

Source: `financeMD/damodaran/New_Home_Page/datahistory.md`; `Valuation/docs/sources/bafb6h9rb.txt`, section 5 (2026-09-14)

File names. Files live under `https://pages.stern.nyu.edu/~adamodar/pc/datasets/`. The name is a family plus a region suffix: `wacc.xls` for the US, `waccIndia.xls`, `waccemrg.xls`, `wacc-Global.xls`, and so on across eight regions. Five names break the pattern and must be hard-coded: `betas.xls` (not beta), `pedata`, `pbdvdata`, `psdata`, and `DollarUS.xls`. The R&D files contain an ampersand that must be URL-encoded. Dated files such as `ctrypremJuly26.xlsx` and `ERPSept26.xlsx` cannot be guessed; the job must read the link list and discover them.

Source: `bafb6h9rb.txt`, sections 1 and 5 (2026-09-14)

The 244-URL manifest. His `datacurrent` page contains 244 distinct dataset addresses. That page is a ready-made download list, and the corpus copy at `financeMD/damodaran/New_Home_Page/datacurrent.md` yields the same 244 with one `grep`. The `load-damodaran` job will use it once per vintage. Two more gotchas: the site misbehaves for Chrome, so use a plain HTTP client with a User-Agent, and `indname` (the company-to-industry map) is not linked from that page at all.

Source: `bafb6h9rb.txt`, section 5; verified by `grep` on `financeMD/damodaran/New_Home_Page/datacurrent.md` (2026-09-14)

The `.xls` gotcha. Most files are `.xls`, the pre-2007 Excel binary format called BIFF8. ExcelJS cannot read it at all. The only mainstream JavaScript reader is SheetJS Community Edition, version 0.20.3, which is installed from `cdn.sheetjs.com` by tarball address, not from npm. The `xlsx` package on npm is stuck at 0.18.5 with two unpatched high-severity security advisories. Never run `npm i xlsx`. The Python fallback is `pandas.read_excel(..., engine="xlrd")`.

Source: `Valuation/docs/sources/bo0hrclp7.txt`, section 7 (2026-09-14)

Inside each file the data sheet is named "Industry Averages" and the real header row is not row 1. Eight to twelve rows of notes come first, so the loader reads the sheet headerless and finds the row whose first cell is "Industry Name". Row 1 carries a "Date updated:" cell, which becomes the vintage stamp. His usage rules: acknowledgment is optional, per-company raw data is not to be redistributed, and he asks that the data stay out of court cases.

Source: `bafb6h9rb.txt`, section 5, points 5, 6 and 8 (2026-09-14)

EODHD

EODHD is a paid data vendor covering more than 70 exchanges, including NSE and BSE. It is the project's only paid data licence, chosen because it is the one vendor with documented, long-history India statements in a normalized schema. It arrives at milestone M4, not before.

Source: `Valuation/docs/sources/bj0fbeiql.txt`, Role 10 (2026-09-14)

- Price: the Fundamentals Data Feed is \$59.99 a month (`read-this-first.md` rounds it to \$60). The free plan allows 20 calls a day.
- Call units: one fundamentals request costs 10 API call units, not one. A 50-name daily refresh is 500 units a day. So the project caches every response in Neon and re-pulls only on a new filing.

- Licence: the standard plans are for personal use. Every row is stored as `licensed_personal`, which the database rule keeps out of the librarian's index. Read the redistribution clause before paying, and test 20 NSE tickers first. The ticker form is `TRENT.NSE`.

Source: `bkm2kob5x.txt`, section 12; `read-this-first.md`, sections 10 and 12; plan, section M4 (2026-09-14)

NSE bhavcopy

A bhavcopy is the free end-of-day price file that the National Stock Exchange publishes for every trading day, under “All reports” on its website. It is the licit free India price source. Screener.in and Tickertape forbid scraping, so they are not sources.

The format changed twice. In July 2024 the old layout was retired in favor of “CM-UDiFF Common Bhavcopy Final” (NSE circular 62424, dated 2024-06-12). In October 2025 the file names moved to four-digit years, `bhddmmyyyy.csv`, and gained session indicators (I1, I2, F1, F2). Any GitHub parser written before late 2025 will break silently. History is available from 2016-01-01.

Source: `bkm2kob5x.txt`, section 12 (2026-09-14)

INDmoney's INDstocks API

INDmoney's broking arm, INDstocks, publishes a real, free, official API at `api-docs.indstocks.com`. It offers REST and WebSocket endpoints for orders, holdings and positions, live quotes, more than 10 years of daily price history, and option chains. There is no subscription fee; you need an INDstocks account with SEBI KYC. Endpoints have per-endpoint rate limits, roughly 10 requests a second on several, and return HTTP 429 when you exceed them.

Two rules shape its use here. First, order-placement endpoints require a whitelisted static IP address under NSE rules. GitHub Actions runners have no fixed address, and this project never places orders, so that constraint never applies. Read-only market-data endpoints have no such restriction. Second, INDmoney's terms forbid bots on its app and forbid copying or republishing content. So the system reads your holdings and prices through the official API only, and never scrapes the app or the website.

Source: `Valuation/docs/sources/btzaixzyh.txt`, CHECK 2, INDmoney section (2026-09-14)

FRED

FRED (Federal Reserve Economic Data) is the free database run by the Federal Reserve Bank of St. Louis. Every series has a short code. The market-regime panel in M4 uses three:

Series	What it is	Value the report saw
DGS10	10-year US Treasury yield, daily	4.95% on 2026-09-10
BAMLH0A0HYM2	ICE BofA US high-yield option-adjusted spread, daily	2.70% on 2026-09-10
BAMLC0A4CBBB	the BBB-rated version of the same spread	not stated in the report

Damodaran's own September 2026 post cites “Federal Reserve Data (FRED)” for these spreads, and DGS10 is the risk-free input for replicating his implied-ERP calculator. FRED offers a free API key and plain CSV downloads; that detail is web knowledge, so verify it on the site.

Source: `btzaixzyh.txt`, CHECK 3, indicator 7 and “How to compute it” (2026-09-14); plan, section 10.7

Bloomberg

There is no consumer API. The “API Library” page is the download index for the BLPAPI software kit, which connects to nothing without a Terminal (about \$31,980 per user per year) or an enterprise data contract. A bloomberg.com subscription is a reading product with no data entitlement.

Source: bj0fbejiq.txt, CHECK 1, parts 1.1 to 1.7 (2026-09-14)

Headline RSS is a grey area. RSS is a machine-readable list of headlines with links. On 2026-09-14 feeds.bloomberg.com/markets/news.rss redirected to www.bloomberg.com/feeds/markets/news.rss and returned same-day items with no licence text. Bloomberg does not document or support these feeds, and its terms forbid any “scraper, robot, bot, spider” and forbid building a database from the service. If the project polls them at all it will do so a few times a day with caching, store title, link and time only, keep the output private, and never make them load-bearing.

Source: btzaixzyh.txt, CHECK 2, Bloomberg RSS section; bo0hrclp7.txt, RESEARCH 6, part 1 (2026-09-14)

Newsletters are the cleanest path. Bloomberg sends them to your Gmail, so reading your own mailbox does not touch its website. For India the two that matter are “India Edition” and “Markets Daily India”. Your reading becomes a narrative_note row: link, date, your own one-sentence paraphrase, and the valuation input it moves. Never article text, and never a row in the index.

Source: bo0hrclp7.txt, RESEARCH 6, part 2 (2026-09-14)

Licence classes, applied

The seven classes in read-this-first.md section 3.3 are the contract label on every box. A database rule refuses licensed_personal, proprietary_personal and link_only rows from the librarian’s index. For any new source, write down its class first, then its cadence, then its format. Legal, then fresh, then correct.

Source: Valuation/docs/read-this-first.md, section 3.3 (2026-09-14)

Learn it

About four hours of documentation reading in total, all free.

Resource	URL	Format	Hours	Why
SEC EDGAR APIs page, then the fair-access policy linked from it	https://www.sec.gov/search-filings/edgar-application-programming-interfaces	web docs	1	The three endpoints, the User-Agent rule and the rate limit, from the source itself
Damodaran’s data pages, current and history	https://pages.stern.nyu.edu/~adamodar/New_Home_Page/datacurrent.html and datahistory.html (local copies in financeMD/damodaran/New_Home_Page/)	web, markdown	1	The 244 links, the cadence, the usage rules, in his words

Resource	URL	Format	Hours	Why
EODHD fundamentals docs and quick start	https://eodhd.com/financial-apis/stock-etfs-fundamental-data-feeds and https://eodhd.com/financial-apis/quick-start-with-our-financial-data-apis	web docs	0.5	Call units, ticker format and the personal-use licence, before M4
NSE “All reports”	https://www.nseindia.com/all-reports	web	0.25	See the bhavcopy file names and session codes with your own eyes
INDstocks API FAQ	https://api-docs.indstocks.com/faq/	web docs	0.5	Rate limits, the static-IP rule for orders, and what read-only endpoints exist
FRED series pages and API docs	https://fred.stlouisfed.org/series/DGS10 , https://fred.stlouisfed.org/series/BAMLH0A0HYM2 , API docs at fred.stlouisfed.org (verify)	web	0.5	Learn to read a series page, then fetch the same series by code

Source: bkm2kob5x.txt, section 12; btzaizy.txt, CHECK 2 and CHECK 3 (2026-09-14)

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter’s tool. The following “Done when” checklist tells you how to judge completion.

Pull Apple’s companyfacts file and find its revenue tag. Apple’s CIK is 320193, padded to CIK0000320193. Replace the name and email in the User-Agent with your own.

```
curl -s -H "User-Agent: Your Name your.email@example.com" \
  "https://data.sec.gov/api/xbrl/companyfacts/CIK0000320193.json" -o
  ↪ apple.json
```

```
python3 -c "
import json
d = json.load(open('apple.json'))
tags = d['facts']['us-gaap']
print(d['entityName'], 'tags:', len(tags))
for t in sorted(tags):
    if 'Revenue' in t:
        vals = tags[t]['units']['USD']
        print(t, 'last period ends', max(v['end'] for v in vals))
"
```

What you should notice:

1. The file is about 3.8 MB and holds 503 us-gaap tags for Apple.

2. The tag literally named Revenues stops at a period ending 2018-09-29. Searching for “Revenues” alone would give you stale numbers.
3. The live tag is RevenueFromContractWithCustomerExcludingAssessedTax. Its FY2025 value, for the year ending 2025-09-27, is 416,161,000,000 dollars, filed 2025-10-31 with frame CY2025.
4. Now repeat the loop with 'Interest' in t and check the last end date for InterestExpense. You should see 2023-09-30. That is the missing-interest-expense problem, seen with your own eyes.

Source: <https://data.sec.gov/api/xbrl/companyfacts/CIK0000320193.json> (fetched 2026-09-14)

Done when

KEY IDEA

- You can say, without looking, which three EDGAR endpoints exist and what each returns.
- You can write the LTM formula in one line and name the three filings it needs.
- You can explain why a zero interest expense produces a perfect synthetic rating, and what flag replaces the zero.
- You can list the Damodaran cadence (yearly, quarterly, monthly) and the five file-name exceptions.
- You know why `npm i xlsx` is banned and what to install instead.
- You can state EODHD's price, its call-unit rule, and its licence class.
- You can say which INDstocks endpoints need a static IP and why this project never touches them.
- You can name the two FRED series the regime panel uses.
- You can explain, in two sentences, why Bloomberg is reading only and where your reading notes go.
- The exercise printed Apple's FY2025 revenue and the 2023-09-30 interest-expense cutoff.

Chapter 10. OCR, web crawling and sub-agents: three helpers, and when to leave them in the drawer

OPTIONAL - FIRST PASS

For corpus preparation. Read the problem each helper solves before using the OCR, crawling, or delegation details.

This chapter covers three tools that sound advanced and are mostly optional. Each solves a narrow problem here. Knowing the problem matters more than knowing the tool. Read it after Chapter 3 and before you touch the corpus ingest code in milestone M2.

What it is (plain words and an analogy)

OCR

OCR stands for optical character recognition. It is software that looks at a picture of a page and writes out the letters it sees. A PDF can contain real text, or it can contain a photograph of text. On screen the two look the same. In the first you can select and copy words. In the second you cannot, because there are no words, only pixels.

Analogy: a typed letter and a photograph of it look identical on your desk. Only the typed one can be searched by a computer. OCR is the typist who reads the photograph and types it back in. Modern

OCR services are vision language models, AI models that read images. They also rebuild headings, reading order and, most important for us, tables.

Source: bj0fbejiq.txt, CHECK 3 section 2 (2026-09-14). The photograph analogy is mine.

Search APIs, crawling and scraping

These are three different jobs, and beginners mix them up.

Job	What you send	What you get back	Library analogy
Search API	A question	Ranked web pages, often with their text	Asking the librarian
Crawling	One starting web address	Every page linked from it, then every page linked from those	Walking every aisle and noting every book
Scraping	One known page	Specific fields pulled out of that page	Opening a known book to page 42 and copying the table

An API, an application programming interface, is a door a service opens so a program can ask it questions. A search API is a search engine with such a door. Search first. Crawl only when you must list a whole site. Scrape only when the numbers are not available through an API.

Source: bj0fbejiq.txt, CHECK 3 section 3 (2026-09-14); the plan, section 8.

Sub-agents

A sub-agent is a second AI session that your main AI session starts. It has its own memory window, its own instructions and a limited set of tools. It does one job and hands back a short summary. The parent never sees the thousands of words the helper read.

Analogy: a research assistant who reads a whole annual report in another room and returns a one-page note. Three things make this worth doing: helpers work at once, each starts with a fresh memory, and a cheap model can read while an expensive model thinks.

Source: bj0fbejiq.txt, CHECK 3 section 4 (2026-09-14); read-this-first.md, glossary.

Why this tool now (for this project)

OCR: three decks that our converter could not read

The local corpus, the folder `financeMD`, was made by converting Damodaran's site to text. Three free PDF decks did not convert cleanly: the "Dark Side of Valuation" core deck, the 2012 full-day seminar deck and the 2013 extended "Jedi Guide" deck. They are image-heavy. A plain PDF-to-text pass returns headings and loses the numbers in the tables.

Source: bkm2kob5x.txt, learning path, Dark Side table and ingestion note (2026-09-14); the plan, section 8; other-side/00-read-me-first.md, "The gaps in the local corpus".

The plan says the librarian's tier 1 is not complete until these decks are backfilled. Document D, "The Other Side", leans on them for the uncertainty types the memo writer names. So OCR is a one-time chore, not a feature.

Source: the plan, sections 1 and 9 ("Corpus gaps"); read-this-first.md, section 2.

Search APIs: only for news, and not yet

Our numbers come from structured sources: SEC EDGAR, Damodaran's tables and later EODHD. None needs a search engine. A search API earns its place only for the qualitative layer, "what changed this quarter for Trent", which feeds the memo writer's evidence list. That is deferred beyond v1.

Source: b31x0pyot.txt, section 5 ("Not relevant" and "Relevant" notes) and section 8; the plan, section 9 ("Deferred").

Sub-agents: deferred to v2

The plan defers sub-agents, for auditability. A prose summary is lossy, and every number in a memo must trace to a row with a source. When the time comes, Cursor's cloud agents and Cursor Projects supply the mechanism, so there is nothing to build.

Source: the plan, sections 8 and 9.

How it is used in this project

OCR, step by step

1. Check before you pay. Run `pdffonts file.pdf`. If no fonts are listed, the file is image-only. Then run `pdftotext -layout file.pdf` – on a few pages. If the text is there, skip OCR.
2. Send the image-only decks to Mistral Document AI through its batch endpoint. Batch means you queue all the pages at once and collect results later, for a lower price. The account is number 9 in Document A's list, key `MISTRAL_API_KEY`.
3. Store the output in the R2 bucket (our cloud file store), named by its hash (a fingerprint computed from the file's bytes), next to the other raw files. It is Markdown, so it enters the librarian's index like any other page.
4. Check quality with three cheap tests. Hand-type three to five representative pages and compare. Add up the columns of each table; if they miss the printed total, a digit was misread. Confirm the page count so no page was silently dropped.

`pdffonts` and `pdftotext` come from a free tool set called Poppler. That is general knowledge, not from the project reports, so verify it.

Source: bj0fbejiq.txt, CHECK 3 section 2 and CHECK 2 Role 8 (2026-09-14); the plan, section 8; read-this-first.md, sections 2 and 10.

Why Mistral. On the only independent 2026 OCR comparison the project found, an olmOCR-Bench run dated 2026-06-28, Mistral scored 74.9% overall and 94.3% on tables. Open-source tools scored far lower: Marker 55.2%, Docling 37.7%. Mistral also returns bounding boxes, the pixel coordinates of each block, which will let a memo cite a slide number.

Source: bj0fbejiq.txt, CHECK 2 Role 8 (2026-09-14).

What it costs. Mistral OCR is \$4 per 1,000 pages, about \$2 per 1,000 through the batch API. A 100-page deck is therefore about \$0.40 at list price, or about \$0.20 through the batch endpoint we use. Document A budgets about \$10 once for all OCR and expects the three decks to cost about \$2. Table quality, not price, decides.

Source: bj0fbejiq.txt, CHECK 3 section 2 and CHECK 2 Role 8; read-this-first.md, sections 10 and 12 (2026-09-14).

Crawling, search and the law, in plain words

Robots.txt is a small text file at a website's root, for example `example.com/robots.txt`. It tells automated visitors which paths they may fetch. The format is a published standard, RFC 9309. Obeying it is voluntary, and no law makes ignoring it a crime. Courts have treated ignoring it as bad faith. We honor it, always.

Source: `bj0fbejiq.txt`, CHECK 3 section 3 (2026-09-14).

The legal picture, as the report summarizes it, not as legal advice: in the United States, fetching a public page is generally not a computer-crime offense. The *hiQ v. LinkedIn* case held that an open page has no gate to break. But *hiQ* still lost on LinkedIn's terms of service, and *Meta v. Bright Data* confirmed that contract claims survive. The real risks are a site's terms, copyright and privacy law. That is why every database row carries a `license_class`, and why the index refuses the classes that may not be shown.

Source: `bj0fbejiq.txt`, CHECK 3 section 3 (2026-09-14); `read-this-first.md`, section 3.3; the plan, section 10.3.

The good news is that we rarely scrape. SEC EDGAR publishes free official JSON APIs (structured data a program can read directly) with no key. You send a descriptive `User-Agent` header (a short self-identification line sent with each request) with a contact email and stay under 10 requests per second. Damodaran's pages are static files we fetch directly. Bloomberg and INDmoney forbid scraping in their terms, so we never crawl them; Chapter 9 covers the allowed feeds.

Source: `b31x0pyot.txt`, section 8B (2026-09-14); the plan, section 8.

Which search API, if we add one. Artificial Analysis published a Search Index on 2026-08-18. It ran the same AI agent over three question sets and changed only the search provider. Higher score is better.

Provider	Index score	Cost per 1,000 tasks	Seconds per task
Parallel (advanced)	75	\$83.51	35.9
Exa (auto)	74	\$127.15	26.2
Firecrawl	73	\$75.42	55.0
Parallel (turbo)	67	\$60.21	18.8
Tavily (basic)	66	\$192.58	36.3

Parallel turbo is about six cents per task, my division of the table's figures. The project's pick is Parallel: turbo for routine lookups, advanced for the few that matter. Tavily is the worst value on the board, lowest quality of the five shown here (second-lowest on the full board) at the highest price. Two cautions. This is one benchmark from one publisher, so treat three points as noise. And two project reports quote different dollar figures for the same index; the ranking agrees, the dollars do not, so re-check the live page before spending.

Source: `bj0fbejiq.txt`, CHECK 2 Role 9 and caveat 2 (2026-09-14); `b31x0pyot.txt`, section 5; `read-this-first.md`, section 13.

Outside the benchmark, all three leaders have free tiers: Exa \$20 of credit plus \$10 a month, Parallel \$5 of credit monthly, Firecrawl 1,000 credits a month at one credit per page. Document A lists Parallel as an optional later account at about \$18 a month.

Source: `bj0fbejiq.txt`, CHECK 3 section 3 (2026-09-14); `read-this-first.md`, section 10, row 12.

Sub-agents: what Cursor already gives you

Cursor cloud agents run in isolated cloud machines. You start one by writing @cursor on a GitHub pull request or issue, or from cursor.com/agents. Any number can run at once, under a spend limit you set. They bill at the standard price of the chosen model, from your Cursor usage pool. The MCP tools (the project's own tool server that Cursor plugs into, see Chapter 2) in `.cursor/mcp.json` work inside them, so a cloud agent can call `search_corpus` or `value_company`.

Cursor Projects, released 2026-09-10, adds a coordinator agent that plans and hands pieces to parallel sub-agents without writing code itself. That is the sub-agent pattern, packaged.

Source: `btzaixzyh.txt`, CHECK 1, "Cloud / Background Agents" (2026-09-14); `read-this-first.md`, section 11.

The cost, plainly. Each helper pays for its own instructions and tool list. Five helpers cost roughly five times one call, plus the parent's cost to split and merge. Anthropic's account of its multi-agent research system puts the token bill near fifteen times a single chat. On a \$200 a month ceiling, remember that number.

Source: `bj0fbеjiq.txt`, CHECK 3 section 4 (2026-09-14).

When not to use them:

- Step two needs the full output of step one, and a summary would lose it.
- Two helpers would write the same database row and silently conflict.
- The task is small, and the handoff costs more than the work.
- The numbers must be audited. Make helpers write structured rows with citations and return only row ids. The parent reads facts from the database, never from a paraphrase.
- You are debugging. One straight transcript is easier to read than ten interleaved ones.

Source: `bj0fbеjiq.txt`, CHECK 3 section 4 (2026-09-14); the plan, section 8.

Speed and cost per task, side by side

Task	Tool	Rough cost	Rough speed
OCR one 100-page deck	Mistral batch	about \$0.20 to \$0.40	Queued; results come back later, not in seconds
One search query	Parallel turbo	about \$0.06	About 19 seconds inside an agent loop
One EDGAR fact lookup	SEC API	\$0	Under a second; stay under 10 per second
Five parallel helpers	Cursor cloud agents	about 5 times one call	All five run at once

Source: `bj0fbеjiq.txt`, CHECK 2 Roles 8 and 9, CHECK 3 sections 2 to 4; `b31x0pyot.txt`, section 8B (2026-09-14).

Learn it

All free. Total about 4 hours. Stop after row 3 if you are not adding news search.

#	Resource	Format	Hours	Why
1	https://mistral.ai/pricing/api/ and https://mistral.ai/news/mistral-ocr	Official pricing and announcement	0.5	The exact price per 1,000 pages and what the Markdown output looks like.
2	https://www.sec.gov/search-filings/edgar-application-programming-interfaces	Official docs	1	Free, legal, structured US data; removes most of the reason to scrape.
3	https://www.rfc-editor.org/rfc/rfc9309.html	Standard, about 30 pages	0.5	The actual robots exclusion protocol, so you know what you are honoring.
4	https://artificialanalysis.ai/methodology/search-api	Benchmark write-up (2026-08-18)	0.5	How the Search Index was built, and the cost table you would budget from.
5	https://www.anthropic.com/engineering/building-effective-agents	Article	0.75	The decision tree between one chain of steps and an orchestrator with workers.
6	https://cursor.com/docs/cloud-agent	Official docs	0.5	How to start, watch and cap a cloud agent from a pull request.

Source: bj0fbeiqr.txt, CHECK 3 sections 2 to 4 resource tables; b31x0pyot.txt, section 5; btzaixyh.txt, CHECK 1 links (all as listed 2026-09-14).

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

The goal is to tell a text PDF from an image PDF with your own hands.

1. Download the short core deck, `darkside.pdf`, from <https://pages.stern.nyu.edu/~adamodar/pdfiles/country/darkside.pdf> (the link Document D also lists). Save it in your Downloads folder.
2. In a terminal, run `pdffonts ~/Downloads/darkside.pdf`. If no fonts are listed under the header, the file is image-only.
3. Run `pdftotext -layout ~/Downloads/darkside.pdf - | head -40`. Read what comes out. Are the words there, or only a few headings and blank lines?
4. Note which class the deck falls in, in your decisions log.
5. In a browser, open <https://www.sec.gov/robots.txt>. Read the first twenty lines. Write one sentence on what it allows and what it delays.

If `pdffonts` is missing, install Poppler first; on a Mac that is a Homebrew command, which you should verify on the Homebrew site.

Source: bkm2kob5x.txt, learning path, Dark Side table (2026-09-14); bj0fbeiqr.txt, CHECK 3 sections 2 and 3.

Done when

KEY IDEA

- You can say in one sentence why a PDF can look normal yet contain no text.
- You have run `pdffonts` and `pdftotext` on one deck and written down which class it is.
- You can name the three quality checks for OCR output without looking: hand-typed pages, column sums, page count.
- You can explain the difference between search, crawl and scrape with the library analogy.
- You can state why terms of service, not computer-crime law, are the real scraping risk, and why every row carries a `license_class`.
- You can name two cases where a sub-agent is the wrong tool, and say where the sub-agent machinery already lives in Cursor.

Chapter 11. Evals and Langfuse: how you find out whether the model is right

OPTIONAL - FIRST PASS

For checking model quality. Start with what an evaluation measures, then use the examples when building the project's checks.

What it is (plain words and an analogy)

An “eval” is short for evaluation. It is a test for a program that has a language model inside it. Chapter 4 showed you normal tests in Vitest. A normal test gives one input and expects the exact same output every time. A model does not behave like that. Ask it the same question twice and the wording changes. So you need a different kind of test.

Think of a driving license. The written test has one correct answer per question, and a machine can mark it. The road test has an examiner who grades you against a checklist. This project has both kinds, and they must never be mixed up.

Three more terms, defined once:

- **Error analysis.** You read a pile of real outputs yourself and write down what went wrong in each. You do this before writing any automatic check. A teacher reads twenty essays before writing the rubric, not after.
- **LLM-as-judge.** A second model grades the first model's answer against a rubric you wrote. The judge is never the same model as the writer. A student does not mark their own exam.
- **Tracing** (also called observability). A recorder that stores every model call: prompt, answer, time, cost, and labels you attach. It is the system's flight recorder. Langfuse is the recorder this project uses.

Source: Valuation/docs/sources/bkm2kob5x.txt, section 11 (2026-09)

Why this tool now (for this project)

The calculator is protected by the golden test. It must reproduce Almarai's value per share, 7.187840270062114, within 0.000001, before any code merges. That proves the arithmetic and

nothing else. A memo can cite the wrong page and still read beautifully. A librarian answer can quote a 2025 risk premium while calling it 2026. Nobody would notice by eye. Evals are how you notice.

Source: Valuation/docs/read-this-first.md, section 3.1 (2026-09-14)

The timing is set by the plan. In M2 the librarian ships with an “exam-based eval set”. In M4 the plan adds Langfuse tracing of every model call, error analysis on 100 real memos and answers, and a nightly eval issue. Nothing beyond M0 exists today, so everything below is “will”.

Source: Valuation/docs/plan.md, section 5, M2 item 5 and M4 item 3 (2026-09-14)

The cost is zero at this size. Langfuse’s Hobby tier is free: 50,000 units a month, 30-day data access, 2 users, unlimited projects, no card. Datasets and evaluations are included on every tier, including the free one. That is why the research picked it.

Source: Valuation/docs/sources/bo0hrc1p7.txt, section 6 “LLM observability / evals” (2026-09)

How it is used in this project

Two suites, two runners

The research is blunt about this: the two suites “must not share a runner”. Here is the split.

	Deterministic golden tests	Generative evals
What is checked	Arithmetic	Retrieval, faithfulness, citations, vintages, refusals
Examples	Almarai within 1e-6; terminal cost of capital equals risk-free rate plus mature ERP; reinvestment lag; synthetic rating from interest coverage	Is the cited passage the source of the claim? Is the vintage right? Are advice words refused?
Who marks it	Code, exact comparison	Kimi K3 as judge, against a rubric
Where it runs	Vitest in CI (continuous integration: the robot that runs the tests on every pull request)	A nightly job over a hand-built set of about 50 questions
When it fails	The merge is refused	A GitHub issue is opened; it never blocks a merge

Source: Valuation/docs/sources/bkm2kob5x.txt, section 11 and summary line “The two eval suites must not share a runner” (2026-09)

Why report-only? Because the judge is itself a model and can be wrong. A red generative eval means you read the failing case. It does not refuse code.

One refusal check is deliberately not a judge. The rule that no report, memo or chat answer may contain the advice words (listed in read-this-first, rule 3.6) is a plain text scan. It is deterministic, so it lives with the golden tests and does block merges.

Source: Valuation/docs/plan.md, section 10.1 “No advice” (2026-09-14)

The judge never equals the generator

GLM-5.3 writes the memos and answers. Kimi K3 grades them. The research states the rule bluntly: “Never self-judge.” A model grading its own work shares its own blind spots. If it misread a passage when answering, it will misread it again when checking.

Kimi K3 is the judge because it is the best-calibrated open-weight model on the AA-Omniscience test (chapter 7 explains that test). “Calibrated” means it says “I do not know” rather than inventing an answer. That is the habit you want in an examiner. One judged case costs about \$0.0298, so a 200-case suite is about \$5.96 per run.

Source: Valuation/docs/sources/bcy86uopa.txt, row (d) “judge / eval” (2026-09);
Valuation/docs/read-this-first.md, section 4.1

Error analysis first, scorers second

This is the part most people skip, and the part that matters most. The method, from Hamel Husain’s writing:

1. Collect 100 real outputs. Here that will be memos and librarian answers from actual runs.
2. Read every one yourself. Write one line per failure, in plain words, in a spreadsheet.
3. Group the lines into failure modes. Expect five to ten groups.
4. Only now write a scorer per group. Some scorers are code. Some are a judge prompt with a rubric.

Husain’s field guide says teams who skip step 2 “build dashboards that measure nothing”.

Source: Valuation/docs/sources/bkm2kob5x.txt, section 11, “Recommendation for you” (2026-09)

Failure modes to expect here, based on the system’s rules: a claim with no citation; a cited passage that does not contain the claim; two vintages mixed in one answer; an India industry row with fewer than 10 firms and no fallback note; an invented number where the corpus has none.

Source: Valuation/docs/plan.md, section 10.1 (2026-09-14)

What Langfuse logs

A **trace** is one request, such as “value Trent” or one librarian question. Inside it are **observations**, one per step: retrieval, rerank (a second pass that reorders the retrieved passages by relevance), the model call, the citation check. **Scores** are numbers or labels attached to a trace, by the judge or by you. Each of these is one “unit”. A trace with five steps is about six units, so the free tier holds roughly 8,000 traces a month.

Source: Valuation/docs/sources/bo0hrclp7.txt, section 6 (2026-09)

Every model call will carry the same labels, so that evals can be sliced by them later:

```
metadata: { vintage, source_ids, license_class, model_id, prompt_version }
```

Two budget rules follow. Log 100% of interactive chat. Sample the nightly batch at 10%. Logging 5,000 companies every night unsampled would be about 900,000 units a month, about \$93 a month on the Core plan.

Source: Valuation/docs/sources/bo0hrclp7.txt, section 6 and section 8.2 (2026-09)

Datasets and evals inside Langfuse. A dataset is a table of questions with expected answers and sources. Your 50 golden questions will live there. An eval run sends each question through the librarian, asks Kimi K3 to score the answer, and stores the scores. Two runs can be compared side by side, so a prompt change or a model swap shows up as better or worse.

Source: Valuation/docs/sources/bj0fbeiql.txt, Role 5 (2026-09); the mechanics of a dataset run come from Langfuse’s own docs at <https://langfuse.com/docs>, which the local reports do not describe

Practical details. The account is row 8 on the accounts list. The keys are LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY and LANGFUSE_BASEURL; they go into GitHub secrets, never the repo. Langfuse is open source and was acquired by ClickHouse on 2026-01-16. Self-hosting stays free, the escape hatch if pricing changes.

Source: Valuation/docs/read-this-first.md, section 10, row 8; Valuation/docs/sources/bj0fbeiqr.txt, Role 5 (2026-09)

Braintrust, optional

Braintrust is an eval-first product. Its core idea is `Eval()`, a dataset plus a task plus scorers, with per-row diffs and a regression view. The free Starter tier gives 10,000 scores, then \$2.50 per 1,000. Billing by score punishes a suite you re-run often, which is what the golden set is. So it is optional, row 12 on the accounts list. Open it only if you want its regression screen.

Source: Valuation/docs/sources/bo0hrclp7.txt, section 6; Valuation/docs/read-this-first.md, section 10, row 12 (2026-09)

The accuracy scoreboard: bias and variance

The two suites above grade the system against your rubric. The scoreboard grades it against the world. Every valuation will be stored with a timestamp, its inputs, its memo version and its three vintages. After 90, 180 and 365 days a job records the realized price. Two statistics follow, by sector, region, vintage and memo version:

- **Bias** is the average signed error of value versus the later price. Picture arrows at an archery bullseye. If they all land left of center, that is bias.
- **Variance** is the spread of those errors. If the arrows are scattered widely, that is variance, even when their average is dead center.

The page carries one banner: “The engine is tested (Almarai 1e-6). The forecast is not.” Chapter 12 shows how Hex draws it from Neon.

Source: Valuation/docs/read-this-first.md, section 6; Valuation/docs/plan.md, section 5, M4 item 4 (2026-09-14)

Why log now and score later? Because Damodaran’s own step five is “Keep the feedback loop open”, and he admits the three hardest words are “I was wrong”. A scoreboard makes those words a table instead of a feeling. His data also shows that no risk-premium method forecasts next year’s return well, so one month of results is noise. The first scored horizon appears 90 days after the first stored valuation. The scoreboard shows estimates and their errors, never a recommendation.

Source: financeMD/damodaran/blog/2014/06/numbers-and-narrative-modeling-story.md (2014-06-24); financeMD/damodaran/blog/2026/03/the-price-of-risk-equity-risk-premium.md (2026-03)

Learn it

All free. About 6 hours in total. Read them in this order.

Resource	Format	Hours	Why
Hamel Husain, “Your AI Product Needs Evals”, https://hamel.dev/blog/posts/evals/	Article	1	The case for error analysis before tooling

Resource	Format	Hours	Why
Hamel Husain, “Creating an LLM-as-a-Judge That Drives Business Results”, https://hamel.dev/blog/posts/llm-judge/	Article	1	How to write a judge rubric that agrees with a human
Hamel Husain, “A Field Guide to Rapidly Improving AI Products”, https://hamel.dev/blog/posts/field-guide/	Article	1	The loop: read outputs, label, fix, re-run
Hamel Husain, “AI Evals FAQ”, https://hamel.dev/blog/posts/evals-faq/	Long FAQ, about 40 questions	2	The densest single page; answers the “but what about” questions
Langfuse docs, https://langfuse.com/docs	Documentation	1	Read the tracing page, then datasets, then evaluations
Braintrust docs, https://www.braintrust.dev/docs/start	Documentation	0.5, optional	Only if you want to see the regression view

The paid Maven course by Husain and Shreya Shankar exists. The research says skip it: the free writing contains the method.

Source: Valuation/docs/sources/bkm2kob5x.txt, section 11 (2026-09)

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter’s tool. The following “Done when” checklist tells you how to judge completion.

You can start the golden question set today, with no code. It feeds M2 item 5 directly.

1. Open a blank file called `eval-questions.md` in your notes, not in the repo yet.
2. Write five questions you would ask the librarian, each answerable from the corpus.
3. Under each, write the expected answer in one line and the exact corpus path.
4. Under that, write one line saying how a judge should mark a wrong answer.

One row is done for you, so you can copy the shape:

Q: For a rupee valuation in 2025, how does Damodaran get the risk-free rate?
 Expected: India government bond 6.32%, minus the Baa3 default spread 2.16%,
 ↪ gives 4.16%.

Source: `financeMD/damodaran/pdfiles/country/val2dayIndia2025.md` (Vedant row)

Judge rubric: fail if the answer does not subtract the default spread, or cites
 ↪ a different file.

Source: `financeMD/damodaran/pdfiles/country/val2dayIndia2025.md` (2025)

Ten minutes gives you five rows. Fifty is the goal for M2. Nobody but you can write these, because they encode what a correct answer looks like to you.

Done when

KEY IDEA

- You can say, in one sentence each, what error analysis, LLM-as-judge and tracing are.
- You can explain why the two suites use different runners, and which one blocks a merge.
- You can name the generator and the judge here, and say why they must differ.
- You can say what bias and variance mean on the scoreboard, and why nothing is scored before 90 days.
- You have five golden questions written, each with an expected answer, a corpus path and a rubric line.

Chapter 12. Hex: a private dashboard over Neon

OPTIONAL - FIRST PASS

For the dashboard stage. Understand the connection between stored data and the chart, then return to the notebook exercises.

What it is (plain words and an analogy)

Hex is a website where you write small blocks of code, called cells, and see the result under each block. A page of cells is a notebook. One click turns the notebook into a dashboard, which is the same page with the code hidden and only the charts and tables showing. Hex runs in the cloud, so nothing is installed on your Mac.

Think of a kitchen with a serving hatch. The notebook is the kitchen, where you chop, mix and taste. The dashboard is the hatch, where the finished plate appears. Hex is the one tool on the shortlist where the kitchen and the hatch are the same room.

Three kinds of cell matter here. A SQL cell asks the database a question in SQL, the standard language for talking to databases. Its answer arrives as a dataframe, which is a table held in Python memory, like one tab of a spreadsheet. A Python cell can then do arithmetic on that table. A chart cell draws a table as a bar, line, scatter or histogram without any code. An input widget is a dropdown or slider on the page that feeds a value into the cells below it.

Source: Valuation/docs/sources/btzaixzyh.txt, CHECK 2 comparison table and “Why Hex” list (September 2026).

Why this tool now (for this project)

The plan lists seven parts of the system. Part 6 is “a private dashboard (you look, you do not print)”, a hosted notebook over the same database that shows diversification, value-versus-price distributions, Monte Carlo ranges, the accuracy scoreboard and the market panel. Hex was picked because it is the only tool within budget that combines five things in one surface: SQL straight against Neon, real Python for Monte Carlo and bias and variance, product-grade chart cells with input widgets, private single-user sharing, and a built-in AI agent.

Source: Valuation/docs/plan.md, sections 1 and 7 (2026-09-14).

Hex has two plans that concern you. The comparison below copies the report.

Plan	Price	What you get	What you do not get
Community	Free	Unlimited users, 5 projects, small compute, 7-day history, project-level database connections, AI agent trial	No published apps, so the dashboard is only visible inside the notebook editor
Professional	\$36 per editor per month	Published, code-hidden apps; one scheduled refresh per project, daily at most; workspace-level connections	Custom timed schedules (cron) and hourly runs, which need Team at \$75

The plan and Document A budget the same line: Hex costs \$0 now and \$36 later, when you want a published dashboard with a daily refresh. The honest costs of Hex are lock-in (the notebook lives in Hex, not in git), small compute on Community, and embedding into a web app being an Enterprise feature.

Source: btzaixzyh.txt CHECK 2 comparison table and “honest costs” paragraph; Valuation/docs/read-this-first.md section 12 (2026-09-14).

How it is used in this project

Connecting to Neon

Neon is the cloud Postgres database that holds every number the calculator uses. Hex talks to it through a native Postgres connector with SSL, which encrypts the link. The accounts table in the plan says exactly one thing about the Hex setup: configure the Neon connection inside Hex with the pooled host and SSL on. The pooled host is the address that ends in `-pooler` and goes through a connection pooler called PgBouncer, which shares a few real connections among many short queries. Use `sslmode=require`, not `verify=full`, on the pooled host. Keep the direct host for migrations only. Neon also recommends pointing dashboard tools at a read replica, because dashboards fire long queries at the main database. At this size a read-only database role is the cheaper first step; the replica can wait.

Source: btzaixzyh.txt CHECK 2, “Neon connection gotchas”; plan section 7, accounts row 10 (2026-09-14).

The pattern: robots compute, Neon stores, Hex draws

The nightly GitHub Actions jobs (chapter 3) do all the heavy work and write finished tables into Neon. Hex only runs `SELECT`. The report says this removes in-tool scheduling as a buying criterion, and it is why Community is enough for a long time. Treat the dashboard as a disposable read layer. If you ever swap Hex for something else, it is a weekend, not a rewrite.

Source: btzaixzyh.txt CHECK 2, last bullet of “Neon connection gotchas” and closing paragraph of “When you’d need a custom Next.js app” (September 2026).

The four dashboards

Dashboard	Tables it reads	Cells it uses	Milestone
Diversification and sector exposure	watchlist, company, valuation_run	SQL cell into a dataframe, treemap or stacked bar chart cell, a dropdown widget for country or date; a scatter of value against price with a 45-degree line; Monte Carlo range per run	M2 first version, M4 for 20 to 50 India and US names
Accuracy scoreboard	valuation_run, realised_price	Python cell with pandas for rolling error, table plus trend chart, grouped by sector, region, vintage and memo version	M4, first scored horizon after 90 days
Market regime panel	erp_monthly, erp_annual, credit spread series from FRED (the St. Louis Fed's free economic data service)	Line charts against Damodaran's 1960 to 2025 history, percentile bands, plain-sentence states, Damodaran's market-timing caveat printed on the panel	M4
Run health	job_heartbeat	One table: job name, last success time, hours since; a red flag when any nightly row is older than 36 hours	M2 onward

Source: plan sections 5 (M2 item 9, M4 items 2, 4 and 5), 10.3 and 10.7; btzaixzyh.txt CHECK 2 “Why Hex, concretely” (2026-09-14).

Python cells: Monte Carlo, bias, variance

Monte Carlo means drawing each uncertain input many times from a range, running the calculator each time, and looking at the spread of results. The ranges come from Damodaran's own industry quartiles. In Hex this is numpy code (numpy and pandas are the standard Python libraries for arrays and tables; scikit-learn adds statistics helpers) in a Python cell over the rows a SQL cell pulled above it. Bias is the average signed error of value against the later price. Variance is the spread of that error. Both are pandas or scikit-learn arithmetic in the same cell, then a table and a trend chart. The banner on that page must read: “The engine is tested (Almarai 1e-6). The forecast is not.”

Source: read-this-first.md sections 3.5 and 6; btzaixzyh.txt CHECK 2 items 3 and 4 (2026-09-14).

When a custom web app would be needed instead

Not now. Build one only when any of these becomes true: you want write-back (edit a growth assumption on the page and re-run); you need more than one class of viewer with per-user access; embedding is the product; you need bespoke visuals such as a DCF waterfall or a football-field range; you are stitching three or more tools together or the dashboard bill passes about \$75 per month; or you want a portfolio piece for an employer. Until then, Hex.

Source: btzaixzyh.txt CHECK 2, “When you'd need a custom Next.js app instead” (September 2026).

Deepnote, the runner-up

Deepnote is the close second. Its free tier gives 3 editors, 5 projects and 2 vCPU with 5 GB machines, with no scheduling. Team costs \$39 per editor per month on an annual plan, or \$49 monthly, and adds scheduled notebooks. Pick Deepnote if you value “it is just a notebook”, which will feel like coursework. Pick Hex if you value “it turns into a dashboard”.

Source: btzaixzyh.txt CHECK 2 comparison table and “Close second” paragraph (September 2026).

Learn it

All free. Total about 4 hours.

#	Resource	Format	Hours	Why
1	https://learn.hex.tech/docs	Official docs, start with the getting-started pages	1	Cells, projects, the app builder and the publish button, in Hex's own words.
2	Hex Learn docs, data connections page for PostgreSQL (verify exact URL)	Official docs	0.5	The form you fill to connect Neon: host, database, user, SSL.
3	https://neon.com/docs/connect/connection-pooling	Official docs	0.5	Why the <code>-pooler</code> host exists and what it does not allow.
4	https://neon.com/docs/connect/connection-errors	Official docs	0.5	The SNI rule (SNI is a field in the encrypted handshake that tells Neon which database you want; old drivers do not send it) and the fix if a connector refuses to talk to Neon.
5	https://pandas.pydata.org/docs/user_guide/10min.html	Official tutorial	1	Dataframes, grouping and rolling windows, which is all the scoreboard needs.
6	https://learn.hex.tech/docs/share-insights/scheduled-runs	Official docs	0.5	Read this before paying for Professional, so the daily cap is not a surprise.

Source: URLs 4 and 6 are from `btzaixzyh.txt` CHECK 2 (September 2026); URLs 1, 3 and 5 are official documentation homes that the local reports do not cover, so they come from general knowledge; the hours are my estimates.

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

This assumes the M2 dashboard project exists in your Hex workspace. The plan lists "add a chart in Hex" as one of the M3 exercises.

1. Open the project and add a SQL cell. Write a query that counts rows in `valuation_run` by the sector of the company, joined through `company`.
2. Name the result dataframe by `_sector`.
3. Add a chart cell on `by_sector`. Choose a bar chart, sector on one axis, count on the other.
4. Add an input widget above the SQL cell, a dropdown of countries. Reference its value in the query's WHERE clause.
5. Change the dropdown and watch the bar chart change.

If the repo does not exist yet, do the smaller version. Create a free Community workspace, make one project, add a Postgres connection with the pooled Neon host and SSL on, and run `select 1` in a SQL cell. Confirm the cell returns a one-row dataframe.

Source: plan section 5, M3 exercises; plan section 7, accounts row 10 (2026-09-14).

Done when

KEY IDEA

- You can say the difference between a notebook and a published app, and which Hex plan gives you each.
- Your Hex project connects to Neon through the pooled host with SSL on, and a `select 1` returns.
- You have made one SQL cell, one chart cell and one input widget that change together.
- You can name the four dashboards and the Neon table each one reads.
- You can list three of the six reasons to build a custom web app, and say why none applies yet.

Chapter 13. Figma and FigJam: the whiteboard where the system is drawn

OPTIONAL - FIRST PASS

For exploring the diagrams. The pictures in PDF 1 already explain the system; use this chapter when you want to move the boxes yourself.

What it is (plain words and an analogy)

Figma is a website for drawing what software will look like. A designer draws every screen and every button in Figma before a programmer builds it. Think of an architect’s blueprint. The blueprint is not the house, but the builder works from it.

FigJam is Figma’s whiteboard. It is made for boxes, arrows and sticky notes, not for pixel-perfect screens. Think of a meeting-room wall covered in sticky notes. The difference is that this wall lives online and never gets wiped.

Both run in a browser and both save to Figma’s cloud. One Figma drawing is called a file. One FigJam drawing is called a board. Files and boards belong to a team. A team sits on a plan, which is the pricing tier. A seat is your role on that plan, and it decides what you may edit.

	Figma	FigJam
Made for	Screens and buttons	Diagrams, flows, notes
Basic unit	A file made of frames (a frame is one screen)	A board
Typical objects	Rectangles, text, reusable components	Shapes, connector lines, sticky notes
You use it when	You are building a screen	You are thinking, planning or explaining
In this project	Not before a screen exists	Now, for all five system diagrams

Source: general product knowledge of Figma, which the local reports do not cover (2026-09-14)

Why this tool now (for this project)

The approved plan puts the FigJam diagrams inside milestone zero (M0), next to the four documents. M0 is the “explain everything” milestone, and nothing past it is built yet. The five boards are pictures of the system you will build later. If you can redraw a board in your own words, you understand that part of the system. That is why the plan makes editing the architecture board your first exercise.

Source: Valuation/docs/plan.md, M0 deliverables and the “Figma, concretely” note (2026-09-14)

How it is used in this project

The five boards

Each board below is live in your Figma account. A copy of each also sits as a PNG image in Valuation/docs/diagrams/png/. A PNG is a plain picture file. It opens anywhere, but you cannot move its boxes.

- System architecture, <https://www.figma.com/board/DtFTvDZeSQpuKLM5tavDUT>. The seven parts of the system and how they talk to each other. PNG: architecture.png.
- Nightly data flow, <https://www.figma.com/board/6LSfZZfWAnbQVY0twr7OMw>. What the GitHub Actions robot does each night, from checking for new filings to writing a heartbeat row. PNG: nightly-data-flow.png.
- The valuation chain, <https://www.figma.com/board/SNaMV0lrKHeDRQgelrbPsp>. Left to right, from revenue growth through free cash flow and cost of capital to value per share. PNG: valuation-chain.png.
- The memo pipeline, <https://www.figma.com/board/LGcrXp6yT3fh4O2IV3mEY8>. How a company story becomes numbers and then a written memo. PNG: memo-pipeline.png.
- The milestone map, <https://www.figma.com/board/LXcGUT7C2ArsRkwsUvRBWt>. The order of work from M0 onward. PNG: milestone-map.png.

Source: Valuation/docs/read-this-first.md, the board table and sections 1, 7, 8, 9 and 14 (2026-09-14); directory listing of docs/diagrams/png/ (2026-09-14)

The Figma connector in the Claude app

A connector is a plug that lets an AI app use another service on your behalf. Connectors speak a shared language called MCP, short for Model Context Protocol. MCP is a standard way for an AI tool to ask a service “what can you do?” and then call those functions.

The Figma connector in this app is signed in as you. Asked “who am I?” on 2026-09-14, it answered: handle Siddharth, team “Siddharth’s team”, plan tier starter, seat View. Web knowledge says the starter tier is Figma’s free plan. The local reports do not confirm that, so check figma.com/pricing before you rely on it.

What the connector can do, in plain words:

- Create a FigJam board from a text description of a diagram. That is how the five boards were made.
- Read a board back, so an AI session can see what you changed on it.
- Read a Figma design file and hand its layout to a coding tool as context. That is how a drawn screen becomes a real screen later.
- Take a screenshot of a file or a board.

What to keep in mind:

- It acts as you. Anything it creates lands in your account, under your name.
- Your seat says View. Creating the boards still worked, so they exist. If a board ever opens read-only, the plan mentions a one-click seat upgrade. Check the budget in read-this-first section 12 before paying for anything.
- Boards live in Figma's cloud, not on your Mac. Keep API keys, passwords and account numbers off every board. Chapter 14 covers security hygiene.

Source: Valuation/docs/plan.md, "Figma, concretely" and section 8 (2026-09-14); Figma connector whoami call and its tool list in this session (2026-09-14)

When to draw a screen

Only when you are about to build one. The plan places screens at v3, and M0 has no screens at all. Until then, every drawing you make is a FigJam board, not a Figma design. Drawing screens early is like choosing curtains before the walls are up.

Source: Valuation/docs/plan.md, section 8, Figma bullet (2026-09-14)

Learn it (a two-hour starter path)

All of these are free. The minutes are my estimates, not figures from a source. Where I am not sure of an exact web address, I give the site and mark it "verify".

Step	Resource	Where	Format	Time	Why
1	Figma Help Center, the FigJam guide	https://help.figma.com/ then search “Guide to FigJam” (exact article URL: verify)	Reading	20 min	Names every tool: sticky note, shape, connector line, section, stamp.
2	Figma’s official YouTube channel, beginner and FigJam tutorials	https://www.youtube.com/@Figma (playlist names: verify)	Video	40 min	Watch someone drag boxes before you drag your own.
3	Figma Learn, the short courses inside the Help Center	help.figma.com , “Figma Learn” section (exact address: verify)	Guided course	30 min	Structured path; take only the FigJam basics course.
4	Figma Community, FigJam templates	https://www.figma.com/community (filter to FigJam: verify)	Browsing	15 min	See how other people lay out flows and borrow a layout.
5	Figma developer docs, the MCP server page	developers.figma.com (verify)	Reading	10 min	Understand what the connector can and cannot touch.
6	The five boards above	Links in this chapter	Doing	15 min	Real diagrams of your own system, not a toy example.

Steps 1 to 6 total about two hours. Add the 10-minute exercise below and you are done for the evening.

Source: Valuation/docs/plan.md, M0 deliverable 4, “a two-hour starter path (Figma’s own free tutorials, then editing the architecture board)” (2026-09-14); web knowledge for the site addresses

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter’s tool. The following “Done when” checklist tells you how to judge completion.

Do this on the architecture board. Work on your own copy so the original stays as delivered.

1. Open <https://www.figma.com/board/DtFTvDZeSQpuKLM5tavDUT> in your browser while signed in to Figma.
2. Duplicate it. Click the board name at the top, or the main menu at the top left, and look for “Duplicate” or “Duplicate to your drafts”. Duplicating means making your own copy. Rename the copy “architecture, my notes”.
3. Add one sticky note per box. Press S for a sticky note, or pick it from the toolbar at the bottom (shortcut: verify). Drag each note next to its box.
4. Write one question on each sticky note. Not a summary, a question. For example, “what happens if this part is down for a night?” or “where does this read its inputs from?”
5. Pick one box and rename it in your own words. Double-click the box text to edit it. If the label says something technical, replace it with what it does for you.
6. Read your questions back. Bring the ones you cannot answer to your next session with the AI tool, and to chapter 3 of “Damodaran, the important parts” for the valuation boxes.

If the copy will not save, or the board opens read-only, stop. Note what the screen says and raise it in your next session. Do not upgrade the seat on the spot.

Source: Valuation/docs/plan.md, “the first exercise is on the architecture board” (2026-09-14)

Done when

KEY IDEA

- You can say in one sentence what Figma is for and what FigJam is for.
- All five board links open in your Figma account, or you have written down which ones do not.
- You have a duplicated architecture board with one question per box and one box renamed in your own words.
- You can name the four things the connector can do without looking.
- You have not drawn a single screen, and you know why that is correct for M0.

Chapter 14. Security hygiene for a solo builder

OPTIONAL - FIRST PASS

Read before opening accounts or handling keys. These are practical setup habits; keep the checklist beside you during that stage.

What it is (plain words and an analogy)

Security hygiene is the set of small, boring habits that stop one stolen key from ruining the project. Hygiene is the right word. It is brushing your teeth, not surgery. Each habit takes minutes. Skipping them costs an evening at best and a month at worst.

Think of the system as a building. Your GitHub account is the master key to the whole building. Each vendor key (OpenRouter, Neon, Pinecone) opens one room. This chapter puts two locks on the front door, a spare key in a safe, and room keys that open one door and expire.

Terms, defined once:

- **2FA** (two-factor authentication): a second proof of identity after your password, usually a six-digit code from a phone app. **TOTP** is the standard those codes follow.
- **Recovery codes**: one-time codes GitHub gives you when you turn on 2FA. They are the spare key for the day you lose the phone.
- **Token** (or PAT, personal access token): a long random string that lets a program act as you. **Fine-grained** means it opens one repository, with chosen permissions, and expires.
- **Secret**: any key, token, password, or database address. **Push protection**: a GitHub setting that refuses a push containing a secret.
- **gitleaks**: a free program that scans files and git history for strings that look like secrets. A **pre-commit hook** is a small script git runs before every commit.
- **.env**: a plain text file of secrets that git is told to ignore through `.gitignore`.
- **Rotation**: replacing a key with a new one and retiring the old.
- **BYOK** (bring your own key): pasting your own API key into a tool such as Cursor. **ZDR** (zero data retention): the provider deletes your prompts after answering.

Why this tool now (for this project)

M1 is the milestone where you will create every account. All keys exist at once and none is guarded yet. The accounts checklist calls GitHub “your root of trust” and says compromise there means everything. That is the account this chapter protects first.

Source: bj0fbejiq.txt, CHECK 2, accounts checklist row 1 (2026-09-14); plan section 5, M1.

Money is the second reason. The whole system has a \$200 per month ceiling. The checklist names four vendors that can run away with your budget: OpenRouter, Mistral, Parallel, and Cohere. A leaked OpenRouter key with no cap can spend the month in one night.

Source: bj0fbejiq.txt, CHECK 2, “Secrets hygiene for this build”; read-this-first.md section 12.

Privacy is the third. Open-weight models run through OpenRouter with ZDR so that no prompt lingers anywhere. A key pasted into a chat window, or into Cursor’s BYOK box, quietly breaks that promise.

Cursor's own help page says "Cursor's Zero Data Retention policy does not apply when you use your own API keys."

Source: btzaixzyh.txt, CHECK 1, "BYOK and OpenRouter", quoting <https://cursor.com/help/models-and-usage/api-keys> (2026-09-14).

How it is used in this project

Eleven habits, each with where it will live.

Habit	What you do	Where
2FA on every vendor	Authenticator app on your phone; GitHub requires it	Every account in Document A, section 10
Recovery codes offline	Download GitHub's codes; print them or store them on a drive that is not synced	Not in the repo, not in a chat
Fine-grained GitHub token	One token, one repo, 90-day expiry, named GITHUB_TOKEN	Repo secrets and local .env
Push protection	Secret scanning and push protection switched on	Repo settings, when the repo is handed over in M2
gitleaks	Runs before each commit and as a job in CI	Pre-commit hook and Actions workflow, from M2
.env discipline	Real values in .env, empty names in a committed .env.example, .env* in .gitignore	Repo root
One email alias per vendor	you+neon@gmail.com, you+voyage@gmail.com	Each signup
Spend caps	OpenRouter limit \$75 a month; Actions cap \$25; Voyage billing alert; a cap at Mistral	Vendor billing pages
Rotation drill	Rotate the OpenRouter key and the Neon connection string once, on purpose; 90-day reminders for the GitHub token and the OpenRouter key	Calendar
No keys in chats or BYOK	Never paste a key into Cursor's model settings, into Claude, or into any chat	Your fingers
Sensitive session logs	Treat AI tool transcript folders like .env	Your laptop

Source: read-this-first.md sections 10, 11 and 12; plan section 5, M1; bj0fbejiq.txt, CHECK 2; bkm2kob5x.txt, section 15 (all 2026-09-14).

A few rows need one more sentence.

Why an alias per vendor. Gmail delivers you+neon@gmail.com to your normal inbox. If Neon's user list leaks, the attacker learns an address that works nowhere else. For the same reason, sign up to Neon with email and a separate password, not only "Continue with GitHub". If GitHub falls, Neon should not fall with it.

Source: bj0fbejiq.txt, CHECK 2, accounts checklist row 2; read-this-first.md section 10.

Push protection on a private repo. The learning note says secret scanning is free on public repos. Whether the toggle appears on a private repo inside a free organization is a thing to verify in the repo's Code security settings when the repo is handed over in M2. If it is missing, gitleaks in a pre-commit hook does the same job on your machine. That is why the plan has both.

Source: bkm2kob5x.txt, section 15; plan section 5, M1.

Session logs. Every AI coding tool writes a transcript of your session to disk. The pi note says its transcripts live in `~/ .pi/agent/`, next to an `auth.json` that holds live tokens, including an OpenRouter key. pi is deferred, but the lesson transfers to Cursor and to any terminal tool. Rotate the OpenRouter key if a log ever leaves your machine.

Source: bkm2kob5x.txt, section 15.

Cline is the allowed exception. The plan lets you give Cline your OpenRouter key inside Cline's own settings. Cline sends prompts straight to OpenRouter under OpenRouter's terms, with logging off and ZDR routing on. Cursor's own model settings never see the key.

Source: plan section 5, M1 step 5; read-this-first.md section 11; btzaixzyh.txt, CHECK 1.

The agent's blast radius. Never give an AI agent a key with write access to anything you cannot restore. Neon branching is the undo button. Create a branch before any schema change the agent proposes. The private repo inside a free organization (Chapter 3) helps here too. It gives you one page to check who and what has access.

Source: bj0fbeiqr.txt, CHECK 2, "Secrets hygiene for this build"; read-this-first.md section 10.

Learn it

Chapter 1 lists the first three under "Security, week 1". They are repeated so this chapter stands alone. All are free.

Resource	Format	Hours	Why
About 2FA, https://docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/about-two-factor-authentication	Docs	0.5	The one page to read before the exercise below
Managing personal access tokens, https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens	Docs	0.5	How to make a fine-grained token scoped to one repo with an expiry; never a classic token with repo on everything
About secret scanning, https://docs.github.com/en/code-security/secret-scanning/introduction/about-secret-scanning	Docs	0.5	What push protection catches and what it does not
gitleaks, https://github.com/gitleaks/gitleaks	Tool readme	1	Install, scan the repo once, wire the pre-commit hook
github/gitignore, https://github.com/github/gitignore	Template file	0.5	Start from Node.gitignore; add <code>.env*</code> , <code>*.xlsx</code> caches, and any raw licensed data dumps
Cursor, bring your own API key, https://cursor.com/help/models-and-usage/api-keys	Help page	0.25	Read the one sentence that says ZDR does not apply with BYOK, then never do it

About 3.25 hours in total. The learning path budgets about 4 hours and \$0 for this set.

Source: bkm2kob5x.txt, section 15; btzaixzyh.txt, CHECK 1 (both 2026-09-14).

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

Turn on 2FA for GitHub and store the recovery codes offline. No AI tool.

1. Install an authenticator app on your phone if you do not have one.
2. Sign in to GitHub in a browser. Open Settings, then the Password and authentication page. Labels move, so verify them on the page.
3. Choose to enable two-factor authentication with an authenticator app. Scan the code the page shows. Type the six-digit code back.
4. GitHub then shows recovery codes. Download them. Print them, or copy them to a drive that is not synced to any cloud. Do not save them in the repo, in a notes app, or in a chat.
5. Sign out. Sign in again with your password and a fresh code. If that works, the lock is on.
6. Write one line in your decisions log: the date, "GitHub 2FA on, recovery codes stored at [place]." Do not write the codes themselves.

Source: bkm2kob5x.txt, section 15; bj0fbejiq.txt, CHECK 2, row 1.

Done when

KEY IDEA

- GitHub asks you for a code at sign-in, and you can name the offline place where the recovery codes sit.
- Every vendor account in Document A, section 10, has 2FA on and its own email alias.
- The repo has `.env*` in `.gitignore`, a committed `.env.example`, and push protection on, or gitleaks in a pre-commit hook if the toggle is not offered.
- OpenRouter shows a \$75 monthly limit, prompt logging off, and ZDR routing on. GitHub shows a \$25 Actions **spending cap**. Voyage has a billing alert.
- You have rotated the OpenRouter key once on purpose, and the nightly job still ran afterward.
- Two calendar reminders exist, 90 days out, for the GitHub token and the OpenRouter key.
- You can explain to someone else why a key never goes into a chat or into Cursor's BYOK box.

Source: read-this-first.md sections 10 and 12; bkm2kob5x.txt, section 15; bj0fbejiq.txt, CHECK 2.

Chapter 15. The Damodaran watch and read order for this project

READ FIRST

Use this to choose the next lesson. Read the watch-order structure and match a topic to the input or concept you are learning.

What it is (plain words and an analogy)

A watch order is a list of his free videos and posts, sorted by what your code needs next. His own syllabus goes intro, then discount rates, then cash flows, then relative valuation. Our order is different, because the engine already exists and you are learning to check it.

Two kinds of material appear below. A webcast is a short recorded lecture, 12 to 20 minutes, hosted on YouTube and listed on his NYU pages. A Data Update is a blog post series he writes every January and February, one post per topic, explaining where his yearly datasets come from.

Analogy: his syllabus is the owner's manual, read cover to cover. This chapter is the index at the back, opened at the page for the part you are holding.

Source: [financeMD/damodaran/New_Home_Page/webcastvalonline.md](#) (session length line); [bkm2kob5x.txt](#) section 13 (2026-09-14)

Why this tool now (for this project)

The calculator copies his spreadsheet, and the nightly jobs copy his datasets. So the only person who can explain why a number is what it is, is him. In Data Update 1 for 2026 he says AI bots “will not only match, but be better than I am, at mechanical and rule-based tasks”. He also says he plans to hand almost the whole data update process to his bot. You are building the mechanical part. The judgment part, reading a memo and spotting a story that does not add up, stays with you. Chapter 0, rule 10, calls the corpus the antidote to the tool. This chapter is the dosing schedule.

Source: [financeMD/damodaran/blog/2026/01/data-update-1-for-2026-push-and-pull-of.md](#) (2026-01-09); [Valuation/docs/guide/00-how-to-learn-with-an-ai-tool.md](#)

How it is used in this project

Each block below maps to one piece of the system. Terms: LTM means last twelve months of financials. EDGAR is the US filings database. A hurdle rate is the return a project must beat, which for us is the cost of capital. A synthetic rating is a credit grade guessed from interest coverage when no agency rating exists. A vintage is the dated copy of a dataset a run used. ERP is the equity risk premium, the extra return demanded for holding stocks over the risk-free rate. CRP is the country risk premium added on top of the mature-market ERP. DCF is discounted cash flow, the method the engine uses.

Block	What it teaches	Where it lands in the system
Accounting, sessions 1 to 5	The three statements, cash versus accrual	Building LTM from EDGAR tags
Valuation online, sessions 1 to 8	Risk-free rate, ERP, betas, cash flows, growth	The input sheet of <code>fcffsimple</code> in <code>zinzu</code> , row by row
Corporate Finance online, sessions 4 to 11	Hurdle Rates I to VIII, bottom-up betas, country ERP	The cost-of-capital worksheet and the country-risk vintage
Data Update 5 and 4 for 2026	Provenance of risk and country numbers	Why the India row says Baa3, 1.87%, 7.08%, 2.85% in the January 2026 vintage
Corporate Finance online, sessions 12, 13, 17, 18	Debt, its cost, the cost-of-capital approach	The synthetic rating table; why Apple's missing interest expense is a decision, not a bug

Block	What it teaches	Where it lands in the system
Valuation online, sessions 9 and 14 to 18	Terminal value, multiples	Terminal cost of capital = risk-free rate + mature ERP; the multiples tab
The Corporate Life Cycle, 20 sessions	Growth, margin and reinvestment by stage	Memo narratives that are not generic
Investment Philosophies	How different investors think	The watchlist framing, and Document D, The Other Side
Statistics 101	Regressions, distributions	Only when you regress across the 94 industry rows

Session numbers come from his NYU pages, which are in the local corpus. The hours in the next two sections are copied from the learning-path report.

Source: financeMD/damodaran/New_Home_Page/webcastacctg.md, webcastvalonline.md, webcastcfonline.md; bkm2kob5x.txt section 13 (2026-09-14); bafb6h9rb.txt for the India vintage values; bkdrkpfng.txt for the terminal cost of capital rule

Learn it

All free. Page URLs were verified on 2026-09-14 by the learning-path report. Playlist ids are taken from the session links on his pages.

Resource	URL	Format	Hours	Why
Self-paced classes index	https://pages.stern.nyu.edu/~adamodar/New_Home_Page/onlineclass.htm	Web page	0.2	One page that links every class below
Accounting (minimalist)	https://pages.stern.nyu.edu/~adamodar/New_Home_Page/webcastacctg.htm (YouTube playlist listed under the table)	Webcasts, slides, tests	3 to 4	Read a 10-K before you parse one
Valuation (online)	https://pages.stern.nyu.edu/~adamodar/New_Home_Page/webcastvalonline.htm (YouTube playlist listed under the table)	25 webcasts	5 to 6 for the DCF half, 3 for terminal value and multiples	The engine's input sheet, in prose
Corporate Finance (online)	https://pages.stern.nyu.edu/~adamodar/New_Home_Page/webcastcfonline.htm (YouTube playlist listed under the table)	36 webcasts	3 for hurdle rates, 2 for debt and taxes	Cost of capital and synthetic ratings
2026 Data Update posts	Listed below	Blog posts, 3,100 to 6,200 words each	2 for posts 4 and 5	The provenance of every vintage
The Corporate Life Cycle	https://pages.stern.nyu.edu/~adamodar/New_Home_Page/webcastCLC.htm	20 sessions of 25 to 35 minutes	10	Narratives by stage

Resource	URL	Format	Hours	Why
Investment Philosophies	https://pages.stern.nyu.edu/~adamodar/New_Home_Page/webcastinvphil2025.htm	Self-paced	8	The framing for The Other Side
Statistics 101	https://pages.stern.nyu.edu/~adamodar/New_Home_Page/webcaststatistics.htm (YouTube playlist listed under the table)	Webcasts	3 to 4	Only for the cross-sectional industry regressions

YouTube playlists for the rows above (moved out of the table so the links stay whole on the page):

- Accounting (minimalist):
<https://www.youtube.com/playlist?list=PLUkh9m2BorqmKaLrNBjKtFDhpdFdi8f7C>
- Valuation (online):
https://www.youtube.com/playlist?list=PLUkh9m2BorqnKWu0g5ZUps_CbQ-JGtbl9
- Corporate Finance (online):
https://www.youtube.com/playlist?list=PLUkh9m2BorqnDenjSLZ2DHIXrdxoN4Bn_
- Statistics 101:
<https://www.youtube.com/playlist?list=PLUkh9m2BorqmXcRzWFbzcjMd7fYErVexF>

The eight 2026 Data Update posts, all in the local corpus under blog/2026:

1. 2026-01-09, The Push and Pull of Data, <https://aswathdamodaran.blogspot.com/2026/01/data-update-1-for-2026-push-and-pull-of.html>
2. 2026-01-23, Equities get tested, and pass again, <https://aswathdamodaran.blogspot.com/2026/01/data-update-2-for-2026-equities-get.html>
3. 2026-01-28, The Trust Deficit: Bonds, Currencies, Gold and Bitcoin, <https://aswathdamodaran.blogspot.com/2026/01/data-update-3-for-2026-trust-deficit.html>
4. 2026-02-01, The Global Perspective, <https://aswathdamodaran.blogspot.com/2026/02/data-update-4-for-2026-global.html>
5. 2026-02-05, Risk and Hurdle Rates, <https://aswathdamodaran.blogspot.com/2026/02/data-update-5-for-2026-risk-and-hurdle.html>
6. 2026-02-16, In Search of Profitability, <https://aswathdamodaran.blogspot.com/2026/02/data-update-6-for-2026-in-search-of.html>
7. 2026-02-20, Debt and Taxes, <https://aswathdamodaran.blogspot.com/2026/02/data-update-7-for-2026-debt-and-taxes.html>
8. 2026-02-24, Time for Harvesting: Dividends and Buybacks, <https://aswathdamodaran.blogspot.com/2026/02/data-update-8-for-2026-time-for.html>

One more post belongs with them. The July 2026 country risk edition, 2026-07-15, is where the July vintage comes from: India default spread 1.75%, total ERP 6.92%, CRP 2.72%.

Source: bkm2kob5x.txt section 13 (2026-09-14); financeMD/damodaran/blog/2026/01 and /02 front matter and word counts; financeMD/damodaran/blog/2026/07/country-risk-drivers-measures-and.md (2026-07-15); bafb6h9rb.txt

The plan by week, at 45 minutes an evening

Five evenings of 45 minutes is 3.75 hours a week. The report's total is about 40 hours over 12 weeks, about 3.5 hours a week. So the plan fits with a little slack. Week numbers match the milestone map, where you take the wheel in weeks 2 to 4 and India follows in weeks 4 to 8.

Week	Watch or read	Hours
1	Accounting sessions 1, 2, 2A, 3, 3A, 4, 4A, 5	3 to 4
2	Valuation online 1 to 4: intro, foundations, risk-free rate, ERP	half of 5 to 6
3	Valuation online 5 to 8: betas, cost of debt and capital, cash flows, growth	other half
4	Corporate Finance online 4 to 11, Hurdle Rates I to VIII	3
5	Data Update 5, then Data Update 4; skim Data Update 1 for the bot section	2
6	Corporate Finance online 12, 13, 17, 18; then Data Update 7	2 plus reading
7	Valuation online 9, then 14 to 18; pair with Document C chapter 6	3
8 to 10	The Corporate Life Cycle, about seven sessions a week	10
11 to 12	Investment Philosophies; then start Document D chapter 5	8
Later	Statistics 101, only when you start the industry regressions	3 to 4

Data Updates 2, 3, 6 and 8 are optional. Read one whenever an evening finishes early.

Source: bkm2kob5x.txt section 13 (2026-09-14); Valuation/docs/plan.md section 4

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

1. In Cursor's terminal, run `grep -l "Data Update" financeMD/damodaran/blog/2026/*/*.md`. You should see eight posts plus two monthly index files.
2. Open the Data Update 1 file and search for "Damodaran Bot".
3. In your decisions log (chapter 0, rule 6), write two sentences. One says what he expects bots to do better. One says what he keeps for himself.
4. Add the week 1 accounting sessions to your calendar as five 45-minute blocks.

Source: financeMD/damodaran/blog/2026/01 and /02 directory listing (`grep -l "Data Update" run 2026-09-14`)

Done when

KEY IDEA

- The eight Data Update posts, with dates, are listed in your reading log.
- You can point to the webcast that explains each input row of the fcffsimpleginzu sheet.
- You can say in one sentence how a country risk premium is built: sovereign rating, then default spread, then scaled up.
- Week 1 is on the calendar, and the accounting playlist is bookmarked.

Chapter 16. The schedule: twelve evenings-only weeks, from “go” to a working system

READ FIRST

Use the order and checkpoints to orient yourself. Start with the weekly budget and milestone table; return to individual weeks when they become relevant.

What it is (plain words and an analogy)

A schedule is a promise about **order, not about speed**. It says what you read, watch and touch each week, and what must be true before the next week starts. Think of a school term. Each week has classes, and a few weeks end in an exam you must pass before the next term. Here the “exams” are called **checkpoints**. A checkpoint is one sentence that is either true or false about the system, such as “the golden test is green”.

The weeks are grouped into milestones. A milestone is a bundle of work with a named owner and a “done when” line. This project has five: M0 (documents, already delivered), M1 (your accounts and Cursor), M2 (the foundation, built for you), M3 (you take the wheel) and M4 (India, accuracy, ranges, the market panel). A sixth, M5, exists in the plan only if other people ever use the system, and is out of scope here. The milestone map board draws them as a road.

Source: Valuation/docs/plan.md section 5 (2026-09-14); <https://www.figma.com/board/LXcGUT7C2ArsRkwsUvRBWt>

Why this tool now (for this project)

The research report’s original 12-week plan assumed you would build everything yourself, with pi as the daily tool. The approved plan changed both facts. The foundation is built for you in days 3 to 7, and Cursor is the daily tool, so the same learning has to be re-timed. A written week also protects the 8 hours, which are easy to lose.

Source: bkm2kob5x.txt, “12-week evening schedule” (2026-09-14); plan sections 2 and 4

How it is used in this project

The rule that comes first

Nothing after M0 starts until you say go. No account is opened, no repository is created, no code is written before then. Week 1 begins on the day you say it, not on a calendar date.

Source: plan, “Scope of this approval: M0 only” (2026-09-14)

The weekly budget

About 8 hours a week: four weekday evenings of 1.5 hours plus one 2-hour weekend block. Roughly 4.5 hours go to guide chapters and 3.5 hours to Cursor work. The Damodaran track is separate: about 3.5 hours a week in the report, which chapter 15 schedules as five 45-minute evenings (3.75 hours, a little slack), and which can overlap with a commute, so the desk total stays near 8 hours. One evening a week is a no-AI evening (chapter 0, rule 4).

Source: bkm2kob5x.txt, “12-week evening schedule”; plan section 4; guide chapter 15

The twelve weeks

“PR” means pull request, a proposed change that someone reviews before it joins the main code (chapter 1). “CI” means continuous integration, the robot that runs the tests on every PR (chapter 3). A vintage is a dated snapshot of an input, so every number knows which day’s data it came from (chapter 5).

Week	Milestone	Guide chapters	Damodaran (chapter 15)	Accounts to open	Do in Cursor	You can stop here if
0 (now)	M0	Preface, 0, 2	Nothing yet	None	Nothing; read A and C	Always. This is the default until you say go.
1	M1 (you) and M2 (built for you)	1, 3, 14	Accounting sessions 1 to 5	Evening 1: GitHub 2FA and organization, Cursor Privacy Mode, OpenRouter (\$20 credit, \$75 limit, logging off, zero-data-retention routing on). Evening 2: Neon, Pinecone, Voyage, Cloudflare. Evening 3: Langfuse, Mistral, Hex, then <code>.cursor/mcp.json</code>	Day-one exercise: ask why terminal cost of capital = risk-free + mature ERP, then find the Mature Market ERP + label yourself	every key is in place and Cursor's agent lists the valuation tools
2	M3	4, then read docs/ <code>walkthrough.md</code>	Valuation online 1 to 4	None	Create and resolve a merge conflict; break the golden test and watch CI go red, then fix it	you are content to read memos and never edit code
3	M3	5, 6	Valuation online 5 to 8	None	Add a company to the watchlist; try to store a number without a vintage and watch Neon refuse; re-index one blog year	the watchlist has the names you care about
4	M3 done	7, 8	Corporate Finance online 4 to 11	None	Review and edit a memo; add a diagnostic warning in TypeScript	five PRs are merged, 20 names have approved memos, and you can explain every AGENTS.md rule aloud. A US-only system now works.
5	M4	9	Data Update 5, then 4	EODHD: test 20 NSE tickers free, then Fundamentals at \$60 a month	<code>value_company</code> TRENT IN shows tier and vintage	you do not need India
6	M4	12	Corporate Finance online 12, 13, 17, 18; Data Update 7	None	Add a chart in Hex; the sector-exposure and value-vs-price views	the dashboard answers your weekly questions

Week	Milestone	Guide chapters	Damodaran (chapter 15)	Accounts to open	Do in Cursor	You can stop here if
7	M4	11	Valuation online 9, 14 to 18; Document C chapter 6	None	Hand-label 100 real memos and answers; check the nightly eval issue	the advice-word test and the citation test are green
8	M4	10, 13	Corporate Life Cycle 1 to 7	None	The market-regime panel; update the five boards to match what exists	the panel reproduces ERP 4.09% and expected return 8.84%
9 to 10	Buffer	Re-read 0 and 8	Corporate Life Cycle 8 to 20	None	Reverse DCF and Monte Carlo review; a first opposing memo	the ratio in chapter 0, rule 8, is trending to zero
11 to 12	Buffer	17 as needed	Investment Philosophies; Document D chapter 5	None	Prove one engine, three consumers: the same value_company answer from Cursor, Cline and Claude	you can teach this chapter to someone else

Source: plan sections 0, 4 and 5 (2026-09-14); Valuation/docs/read-this-first.md section 10; bkm2kob5x.txt “12-week evening schedule” and section 13; guide chapter 15 week table

The plan schedules work only through week 8. Weeks 9 to 12 exist because things slip and because the Damodaran track runs twelve weeks. The first accuracy score needs 90 days of price history after the first stored valuation, so it lands just after this schedule ends.

Source: plan section 5, M4 item 4 (2026-09-14)

The five checkpoints

The report placed these at the end of weeks 4, 6, 8, 10 and 12 for a self-build. Because M2 delivers the code in week 1, your job is to prove each one yourself, earlier.

#	Must be true	Report week	Your week
1	The Almarai golden test is green in CI at 1e-6, and you have seen it go red	4	2
2	No number can be stored without a <code>vintage_id</code> ; the database rejects the attempt	6	3
3	Every displayed number traces to source, license class and vintage	8	4
4	The chat refuses “buy”, “sell” and “target price”, and always cites	10	7
5	One engine, three consumers, no duplicated formulas	12	12

If a checkpoint is false, stop and fix it before the next week.

Source: bkm2kob5x.txt, “Checkpoints that must be true”; bkdrkpfng.txt (golden value 7.187840270062114 versus price 72.28)

Learn it

- Damodaran’s self-paced class index, https://pages.stern.nyu.edu/~adamodar/New_Home_Page/onlineclass.htm. Web page, 15 minutes to bookmark. Free. It is the master list behind chapter 15’s session numbers.
- The milestone map board, <https://www.figma.com/board/LXcGUT7C2ArsRkwsUvRBWt>. FigJam board, 20 minutes. Free. It is this chapter as a picture; the PNG is at Valuation/docs/diagrams/png/milestone-map.png.
- The plan, Valuation/docs/plan.md, sections 0, 4 and 5. Markdown, 30 minutes. It is the contract this chapter is derived from.
- Document A, Valuation/docs/read-this-first.md, sections 10 and 14. Markdown, 20 minutes. Section 10 is the click-by-click order for week 1’s accounts.
- GitHub’s two-factor authentication page, <https://docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/about-two-factor-authentication>. Web page, 20 minutes. Free. The first task of week 1, evening 1.

Source: bkm2kob5x.txt sections 13 and 15 (2026-09-14)

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

1. Open the milestone map board and the calendar side by side.
2. Write a go date, or write "not yet" in your decisions log (chapter 0, rule 6). Both are valid.
3. Counting from that date, put three 1.5-hour blocks in week 1 and label them "evening 1, 2, 3" with the accounts from the table.
4. Add the five checkpoints as all-day events in weeks 2, 3, 4, 7 and 12.
5. Copy the three real stopping points into the log: end of week 1, end of week 4, end of week 8.

Done when

KEY IDEA

- A go date exists, or "not yet" is written down on purpose.
- Week 1's three evenings are on your calendar with their account lists.
- You can name the checkpoint that gates each week without looking.
- You can say in one sentence why the report's week 4 became your week 2.

Chapter 17. Glossary: every term in the guide and Document A, in plain words

OPTIONAL - FIRST PASS

Your lookup chapter. When a term interrupts your reading, find it here and return to the passage. No need to read the glossary straight through.

What it is (plain words and an analogy)

A glossary is an alphabetical list of terms, each with a short definition. This one covers the words used across the guide and Document A. It extends the glossary in Document A section 15 and never contradicts it. Where a term has a full chapter, the entry here is the short version and names the chapter.

The analogy is a phrase book you carry in a foreign country. You do not learn the language from it. You use it to stop being stuck, and then you go back to the lesson.

Source: Valuation/docs/read-this-first.md, section 15 (2026-09-14)

Why this tool now (for this project)

Two vocabularies arrive at once: software words like "migration" and finance words like "sales to capital". Each chapter defines its terms once, but the definitions are scattered across seventeen chapters and Document A. One searchable page saves you from re-reading a chapter to recover one word. It also keeps a memo, a database column and a chapter meaning the same thing by "vintage".

How it is used in this project

- The glossary uses the same names as the code and the database. `vintage` and `licence_class` are column names, not only ideas.
- When Cursor, a memo or a Damodaran post uses a word that is not here, look it up, then add it through a pull request, as chapter 1 shows.
- Entries about milestones after M0 say “will”. Nothing beyond M0 is built yet.

The glossary

Terms are alphabetical. Numbers sort before letters. A source line sits under each block.

3 to C

- **3P test.** Damodaran’s filter for a story: possible means it could happen, plausible means it could reasonably happen, probable means you would put weight on it today. Only the probable story sets the base case. See Document C chapter 1.
- **Batch mode.** Sending many model requests together and accepting a slower answer in exchange for a lower price. GLM-5.3-Flash in batch mode will do bulk cleaning at \$0.075 in and \$0.25 out per million tokens.
- **Beta.** A number that says how much a stock tends to move when the whole market moves. The engine uses an industry average, re-levered for the company’s debt.
- **Bias.** The average signed error of estimated value against a later price, across many valuations. Positive bias means the values ran too high on average.
- **Branch.** In git, a parallel draft of the code. In Neon, a parallel copy of the whole database.
- **CI.** Continuous integration. Checks that run automatically on every pull request, and `main` cannot change unless they pass.
- **Commit.** One saved snapshot of the files in a repo, with a short message. Chapter 1’s analogy: putting the pencil down and dating the margin.
- **Context window.** The amount of text a model can hold during one call, measured in tokens. A sub-agent gets its own.
- **Cost per task.** The dollars a model needs to finish one benchmark task. It maps to your bill better than price per token, because reasoning tokens are billed as output.
- **Country risk premium (CRP).** The extra return demanded for a company’s exposure to a riskier country, added on top of the mature-market ERP. India, January 2026 vintage: CRP 2.85%, total ERP 7.08%.
- **Crawler.** A program that starts at one page and follows links to list a whole site. A search API returns ranked pages for a question; a scraper fetches one known page.

Source: Valuation/docs/damodaran-essentials/c1-why-value-story-and-the-3p-test.md;

Valuation/docs/read-this-first.md sections 3.1, 3.4, 4.1, 6 and 15;

Valuation/docs/guide/01-git-github-and-pull-requests.md;

financeMD/damodaran/New_Home_Page/definitions.md, row “Beta (CAPM)”;

Valuation/docs/sources/bj0fbějiq.txt, CHECK 3 sections 3, 4 and 5; bafb6h9rb.txt India rows (2026-09-14)

D to H

- **Deploy.** Putting a new version of code where it actually runs. Here it means merging to `main` so the nightly robots use the new code, and applying any migration to Neon. The local reports do not define this word; this is general usage.

- **Diff.** The list of exact lines added and removed by a change. Chapter 0 says read the diff, not the summary.
- **Embedding.** A list of numbers that places a passage in a space where similar meanings sit close together. Also called a vector.
- **ERP (equity risk premium).** The expected return on the stock market minus the risk-free rate. It is multiplied by beta to get the cost of equity.
- **FCFF.** Free cash flow to the firm: after-tax operating income minus reinvestment. “To the firm” means before any debt payments, so it belongs to lenders and shareholders together.
- **Golden test.** A test that pins a known-correct answer so any drift is caught. Here the engine must reproduce Almarai’s value per share, 7.187840270062114, within 0.000001.
- **Harness.** The scaffolding around a model: prompt, tools, loop, permissions and interface. Cursor is a harness; the model inside it is not.

Source: Valuation/docs/read-this-first.md sections 3.1 and 15;

Valuation/docs/guide/00-how-to-learn-with-an-ai-tool.md;

financeMD/damodaran/New_Home_Page/definitions.md, rows “Equity Risk Premium (ERP)” and “Free Cash Flow to Firm (FCFF)”; Valuation/docs/sources/bkdrkpfnq.txt (2026-09-14)

I to M

- **Implied ERP.** The ERP solved backwards from today’s index level, expected cash flows and the risk-free rate. September 2026: 4.09%, with an expected return on US stocks of 8.84%.
- **Judge model.** A model used to grade another model’s answers during evaluations. Kimi K3 will be the judge here, and the judge must never be the model being judged.
- **Licence class.** Our label on every stored fact saying what you may do with it. Seven classes exist, three are refused from the librarian’s index, and the column is spelled `licence_class` in code.
- **LoRA.** Low-Rank Adaptation: fine-tuning by training a small adapter on top of frozen weights, which is why it is cheap. It changes form, not facts, and is not part of v1.
- **MCP.** Model Context Protocol, the standard plug that lets Cursor call our tools.
- **Migration.** A versioned script that changes the database’s structure, checked into git and applied in order. It is the database equivalent of a commit.
- **Model router.** One service that forwards a request to any of many models under one key and one bill. OpenRouter is the router here.
- **Monte Carlo.** Drawing each uncertain input many times from a range, running the calculator each time, and looking at the spread of results. The ranges come from Damodaran’s own industry quartiles.

Source: Valuation/docs/read-this-first.md sections 2, 3.3, 3.5, 4.1 and 15;

Valuation/docs/damodaran-essentials/c6-relative-valuation-and-what-the-market-prices-in.md;

financeMD/damodaran/New_Home_Page/definitions.md, row “Equity Risk Premium - Implied”;

financeMD/damodaran/pc/imlprem/ERPbymonth.md; Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3 sections 1c and 7 (2026-09-14)

N to R

- **OCR.** Optical character recognition: turning a picture of text into text. Mistral will do it once, for three image-only lecture decks.
- **Open-weight.** A model whose weights are public, so many hosts can serve it. Every runtime model here is open-weight.

- **PR (pull request).** A request to merge a branch into main, shown as a diff with a comment thread.
- **Pricing versus valuing.** Pricing asks what the crowd pays for similar assets today, using multiples. Valuing asks what cash flows, growth and risk are worth; the system shows both side by side, never blended.
- **Repo.** A folder of code with its full history, hosted on GitHub.
- **Reranker.** A second, slower model that reads the question and each candidate passage together and re-scores the top results.
- **Reverse DCF.** Solving the valuation backwards: what growth or margin does today's price require?
- **Risk-free rate.** The return on an investment with no default risk, and the base of every discount rate. For rupees it is the 10-year government bond yield minus India's default spread.

Source: Valuation/docs/read-this-first.md sections 2, 3.4, 4 and 15;

Valuation/docs/guide/01-git-github-and-pull-requests.md;

Valuation/docs/damodaran-essentials/c6-relative-valuation-and-what-the-market-prices-in.md;

Valuation/docs/sources/bj0fbeiqr.txt, CHECK 3 sections 2 and 6 (2026-09-14)

S to Z

- **Sales to capital.** Revenue produced per unit of capital invested. The workbook divides next year's extra revenue by it to get this year's reinvestment; Almarai's default, 1.7085, is an industry average.
- **Sub-agent.** A helper AI session spawned by another to do one sub-task in its own context window. It hands back a short summary.
- **Synthetic rating.** A bond rating estimated from interest coverage, meaning operating income divided by interest expense. It sets the company's default spread inside the cost of debt.
- **Terminal value.** One number that stands for every cash flow after year 10: terminal cash flow divided by terminal cost of capital minus terminal growth. The terminal cost of capital is the risk-free rate plus the mature-market ERP, whatever the sheet label says.
- **Token.** A word piece, roughly four characters. Models are billed per token, separately for what you send in and what they write out.
- **Variance.** The spread of the error between estimated value and later price. Bias says which way the estimates lean; variance says how wide they scatter.
- **Vector index.** The store that holds embeddings and finds the nearest ones to a question. Pinecone will hold about 60,000 passage vectors here.
- **Vintage.** Which dated snapshot of Damodaran's tables a number came from. Every run stores three vintage ids: market, country risk and industry.
- **WACC.** Weighted average cost of capital: the cost of equity and the after-tax cost of debt, weighted by market values. It is the rate that discounts FCFF.
- **Zero data retention.** A provider's promise not to store your prompts and outputs. It is a required setting on OpenRouter.

Source: Valuation/docs/damodaran-essentials/c3-the-dcf-chain.md, stations 4, 6 and 7;

Valuation/docs/sources/bkdrkpfnq.txt, synthetic rating and terminal cost of capital notes;

Valuation/docs/read-this-first.md sections 2, 3.2, 4.1, 6, 10 and 15;

financeMD/damodaran/New_Home_Page/definitions.md, row "Cost of Capital"; bj0fbeiqr.txt CHECK 3 section 4 (2026-09-14)

Learn it

#	Resource	Format	Hours	Why
1	Damodaran's definitions page, local copy at financeMD/damodaran/New_Home_Page/definitions.md (live page on pages.stern.nyu.edu , verify the link)	Table, one row per term	1	His own one-line definitions of beta, ERP, FCFF, WACC and reinvestment rate, with a "why it matters" column
2	Document C chapters 1, 3 and 6 in Valuation/docs/damodaran-essentials/	Markdown chapters	3	The full versions of the finance entries above, worked through on Almarai
3	https://docs.github.com/en/get-started/learning-about-github/github-glossary	Reference page	0.5	Official definitions of repo, branch, commit, PR, merge and check
4	https://www.pinecone.io/learn/series/rag/	Course, free	3	Embeddings, chunks, vector index and reranker explained in order, with code
5	https://artificialanalysis.ai/methodology/intelligence-benchmarking	Methodology page	1	Where cost per task, tokens and the index numbers in Document A section 4.1 come from
6	https://www.postgresql.org/docs/current/tutorial.html	Official tutorial	3	Tables, rows and constraints, the ground under migration, branch and licence class

Source: [Valuation/docs/sources/bj0fbeiqli.txt](https://github.com/damodaran-essentials/docs/sources/bj0fbeiqli.txt), CHECK 3 resource tables in sections 5, 6 and 7 (2026-09-14). Resource 3 is from general web knowledge, not covered by the local reports.

10-minute exercise

OPTIONAL - FIRST PASS

For the practical stage. Use this exercise when working on this chapter's tool. The following "Done when" checklist tells you how to judge completion.

1. Open the valuation chain board, <https://www.figma.com/board/SNaMV0lrKHeDRQgelrbPsp>, or the PNG at [Valuation/docs/diagrams/png/valuation-chain.png](https://github.com/damodaran-essentials/docs/diagrams/png/valuation-chain.png).
2. Pick five boxes on the board. Without opening this file, write one sentence for each term in your decisions log from chapter 0.
3. Open this file and compare. Mark each of your five as "same meaning", "close" or "wrong".
4. Find one word on the board, or in Document A, that this glossary lacks. Write its entry in the same style, on a branch, and open a PR titled docs: add <term> to glossary.

Done when

KEY IDEA

- You can say what vintage, licence class and golden test mean, out loud, in under fifteen seconds each.
- You can explain the difference between pricing and valuing, and between bias and variance, without the file open.
- At least one term you found missing has been merged into this chapter through a PR with passing checks.
- You have not written any sentence that tells a reader what to do with a stock in any entry you added.